



ugr

Universidad  
de Granada

TRABAJO FIN DE GRADO  
INGENIERÍA EN INFORMÁTICA

# Simulación de algoritmos bioinspirados en NetLogo

---

**Autor**

Raúl Caubera Jiménez

**Directora**

Rocío Romero Zaliz



Escuela Técnica Superior de Ingenierías Informática y de  
Telecomunicación

Granada, 18 de Junio de 2026

# **Simulación de algoritmos bioinspirados en NetLogo**

Raúl Caubera Jiménez

**Palabras clave:** Netlogo, Algoritmos Bioinspirados, Algoritmos de Ballenas, Algoritmos de Lobo Gris, Algoritmos de Camaleón, Simulación educativa, Problemas de ingeniería.

## **Resumen**

El objetivo de las disciplinas STEM es fomentar el pensamiento lógico-matemático a través de la informática. En nuestra sociedad actual, el uso de herramientas digitales ha surgido como una forma de promover, involucrar y acercar el conocimiento y las competencias necesarias para el desarrollo personal y profesional de los estudiantes.

Se plantea el uso de NetLogo, un entorno de modelado programable que puede simular fenómenos sociales y naturales. Permite a los estudiantes desarrollar, interactuar y visualizar modelos y objetos abstractos en dos y tres dimensiones. Los modelos NetLogo que se han desarrollado para mejorar la enseñanza de algoritmos bioinspirados son al algoritmo de ballenas (WOA), el algoritmo del lobo gris (GWO) y el algoritmo del camaleón (CSA).

Estos modelos promueven una nueva forma de interactuar con este tipo de algoritmos, que se pueden utilizar para resolver problemas de optimización básica y propia de la ingeniería. Esto mejora la comprensión y el uso adecuado de los diferentes parámetros de cada algoritmo por parte de los estudiantes.

# **Simulation of bio-inspired algorithms in NetLogo**

Raúl Caubera Jiménez

**Keywords:** NetLogo, Bioinspired Algorithms, Whale Algorithms, Gray Wolf Algorithms, Chameleon Algorithms, Educational Simulation, Engineering problems.

## **Abstract**

The objective of STEM disciplines is to promote logical-mathematical thinking through computer science. In our current society, the use of digital tools has emerged as a way to promote, involve and bring closer the knowledge and skills necessary for the personal and professional development of students.

The use of NetLogo is proposed, a programmable modeling environment that can simulate social and natural phenomena. It allows students to develop, interact and visualize abstract models and objects in two and three dimensions. The NetLogo models that have been developed to improve the teaching of bioinspired algorithms would be the whale algorithm (WOA), the gray wolf algorithm (GWO) and the chameleon algorithm (CSA).

These models promote a new way of interacting with this type of algorithms, which can be used to solve basic and engineering optimization problems. This improves students' understanding and proper use of the different parameters of each algorithm.

---

Yo, **Raúl Caubera Jiménez**, alumno de la titulación Grado en Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 44373742, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Raúl Caubera Jiménez

Granada a 18 de junio de 2026.

---

D. **Rocío Romero Zaliz**, Profesora del Departamento de Ciencias de la Computación e Inteligencia Artificial de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado ***Simulación de algoritmos bioinspirados en NetLogo***, ha sido realizado bajo su supervisión por **Raúl Caubera Jiménez**, y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada a 18 de junio de 2026.

**La directora:**

**Rocío Romero Zaliz**

### **Declaración sobre el uso de IA generativa y tecnologías asistidas por IA**

Durante la preparación de esta memoria, el autor ha utilizado herramientas de IA generativa, concretamente ChatGPT en su versión 5.5, como apoyo para la revisión ortográfica, la mejora de la redacción y la reformulación puntual de algunas frases.

El uso de esta herramienta se ha limitado a tareas de asistencia en la escritura y no sustituye el trabajo de análisis, desarrollo, implementación, experimentación ni validación realizado por el autor.

Tras utilizar dicha herramienta, el autor revisó, modificó y validó el contenido generado o sugerido según fue necesario, asumiendo plena responsabilidad sobre el contenido final de este trabajo fin de grado.

## ÍNDICE

|                                                                                |         |
|--------------------------------------------------------------------------------|---------|
| <a href="#"><u>Capítulo 1. Introducción</u></a>                                | pag.8   |
| <a href="#"><u>Capítulo 2. Gestión y planificación.</u></a>                    | pag.14  |
| <a href="#"><u>Capítulo 3. Estado del arte y fundamentos teóricos</u></a>      | pag.30  |
| <a href="#"><u>Capítulo 4. Análisis y diseño.</u></a>                          | pag.54  |
| <a href="#"><u>Capítulo 5. Implementación.</u></a>                             | pag.69  |
| <a href="#"><u>Capítulo 6. Pruebas y resultados.</u></a>                       | pag.92  |
| <a href="#"><u>Capítulo 7. Conclusiones y trabajo futuro.</u></a>              | pag.124 |
| <a href="#"><u>Capítulo 8. Bibliografía.</u></a>                               | pag.132 |
| <a href="#"><u>ANEXO A. Manual de usuario de los modelos desarrollados</u></a> | pag.134 |

# CAPITULO 1

## INTRODUCCIÓN

### 1.1 Contexto

No son pocos los problemas de optimización que aparecen constantemente en muchos ámbitos de la ingeniería, la informática y las ciencias aplicadas. Cuando optimizamos estamos buscando la mejor solución posible dentro de un conjunto de alternativas, normalmente sujetas a una serie de restricciones. Esto puede estar orientado a minimizar una magnitud (como el coste, el peso o el error) o a maximizar otra (como el rendimiento, la eficiencia o la calidad de una solución). Muchas veces, los problemas reales presentan espacios de búsqueda demasiado amplios, no lineales, discontinuos o con múltiples óptimos locales. Eso dificulta la aplicación directa de métodos exactos, deterministas o de fuerza bruta, especialmente cuando el coste computacional aumenta de forma significativa con el tamaño del problema. Es por ello que las metaheurísticas se han consolidado como una alternativa muy útil para obtener soluciones suficientemente buenas en tiempos razonables sin tener que dejar varios ordenadores durante años haciendo cálculos.

Dentro de estas metaheurísticas se encuentran los algoritmos bioinspirados, que toman como referencia procesos observados en la naturaleza (como el comportamiento colectivo de animales, la evolución biológica o las estrategias de búsqueda y adaptación de ciertos organismos). Estos algoritmos no nos dan necesariamente la solución óptima global, pero sí que nos permiten explorar de forma flexible espacios de búsqueda complejos y han sido aplicados con éxito en numerosos problemas de optimización.

En este trabajo se estudian tres algoritmos bioinspirados: el algoritmo de optimización de ballenas (WOA del inglés Whale Optimization Algorithm)[\[1\]](#), el algoritmo del lobo gris (GWO de Grey Wolf Optimizer)[\[2\]](#), y el algoritmo de enjambre de camaleones (CSA de Chameleon Swarm Algorithm)[\[3\]](#). Los tres comparten una estructura basada en poblaciones de soluciones candidatas, pero se diferencian en la forma en que actualizan dichas soluciones y en como equilibran como exploran el espacio de búsqueda con la explotación de las mejores zonas encontradas.



## **1.2 Simulación y visualización de algoritmos bioinspirados**

El estudio de los algoritmos de optimización se suele abordar mediante formulaciones matemáticas, pseudocódigo y experimentación numérica. Este enfoque es necesario ya que permite definir bien cómo funciona cada método. Pero también puede resultar un tanto frío o poco intuitivo para quien se enfrenta por primera vez a este tipo de algoritmos, sobre todo si no cuenta con una base sólida de programación o si el algoritmo incluye parámetros cuyo uso no se aprecia de forma inmediata al verlo en seco.

Y es aquí donde la simulación visual cobra un papel especialmente interesante. El hecho de ver cómo se mueve una población de soluciones sobre un espacio de búsqueda ayuda a comprender mejor lo que ocurre durante la ejecución. No es lo mismo leer una ecuación de actualización de posiciones que observar cómo los agentes se dispersan, se acercan a zonas prometedoras, se agrupan alrededor de una buena solución o, en algunos casos, se quedan atrapados en regiones que no son óptimas. Esa representación visual convierte un proceso abstracto en algo mucho más cercano y comprensible para más gente.

En este contexto, NetLogo resulta una herramienta especialmente adecuada. Se trata de un entorno de modelado basado en agentes que permite crear simulaciones interactivas de manera relativamente accesible. Su interfaz facilita la incorporación de botones, deslizadores, gráficas y visualizaciones, de forma que el usuario puede modificar parámetros y observar directamente cómo afectan al comportamiento del algoritmo. Además, NetLogo 3D permite representar superficies de búsqueda y poblaciones de agentes en un entorno tridimensional, lo que mejora la interpretación visual de los procesos de optimización.

Por tanto, el uso de modelos interactivos en NetLogo no tiene interés únicamente desde el punto de vista técnico. También tiene un valor para el docente evidente. Estos modelos permiten transformar algoritmos que pueden parecer demasiado abstractos, en sistemas observables, manipulables y más entendibles. El usuario no solo ve el resultado final, sino también el camino que sigue el algoritmo para llegar hasta él, con sus avances, sus oscilaciones y sus posibles dificultades.

### 1.3 Motivación

La motivación principal de este trabajo surge de la dificultad de comprender el funcionamiento interno de los algoritmos bioinspirados únicamente a partir de teoría, ecuaciones o implementaciones tradicionales no siempre es sencillo. En asignaturas relacionadas con optimización y computación inteligente (como en “*Optimización y Computación Inteligente*”, del Máster en Estructuras de la Universidad de Granada) estos algoritmos tienen un papel relevante por su aplicación a problemas de diseño y optimización en ingeniería.

Sin embargo, el alumnado puede encontrarse con importantes barreras al enfrentarse a estos métodos. Por un lado, muchos algoritmos bioinspirados utilizan conceptos que no siempre resultan inmediatos: *población, fitness, exploración, explotación, óptimos locales, convergencia, etc.* Por otro, su implementación puede requerir conocimientos de programación que no todos los estudiantes poseen al inicio. Esto puede hacer que el aprendizaje se centre demasiado en entender el código o las fórmulas, dejando en segundo plano la intuición sobre cómo funciona realmente el algoritmo.

Ante dicha situación, se propone desarrollar modelos visuales e interactivos que permitan observar el proceso de búsqueda de forma directa. En lugar de estudiar únicamente el resultado final de una ejecución, el usuario puede ver cómo se distribuyen inicialmente las soluciones, cómo evolucionan con el paso de las iteraciones y cómo influyen parámetros como el tamaño de la población, el presupuesto de evaluaciones o los coeficientes propios de cada algoritmo. Es una forma de acercar el algoritmo al estudiante, casi como si se pudiera “*ver pensar*” al método durante la búsqueda.

Además, la elección de WOA, GWO y CSA permite ampliar el repertorio de algoritmos disponibles en NetLogo con técnicas modernas inspiradas en comportamientos naturales muy diferentes: la estrategia de caza de las ballenas jorobadas, la organización jerárquica y cooperativa de los lobos grises, y los mecanismos de búsqueda y captura de presas de los camaleones. Esta variedad aporta riqueza al proyecto y permite comparar distintas formas de abordar un mismo problema: encontrar buenas soluciones dentro de un espacio de búsqueda.

#### **1.4 Antecedentes del proyecto**

El presente trabajo fin de grado se plantea como una continuación de una línea previa de desarrollo de modelos educativos en NetLogo para la enseñanza de algoritmos bioinspirados. En trabajos anteriores se abordó la implementación de otros algoritmos de optimización (algoritmos genéticos, de enjambre de partículas o de colonia de hormigas) y su aplicación a problemas básicos y posteriormente a otros específicos de ingeniería, con el objetivo de crear recursos interactivos que facilitaran la comprensión de estos métodos.

Partiendo de ello, este proyecto amplía el conjunto de modelos disponibles mediante la incorporación de tres nuevas metaheurísticas: WOA, GWO y CSA. La aportación principal del trabajo consiste en desarrollar modelos básicos que permitan visualizar el funcionamiento general de cada algoritmo y compararlos entre ellos, además de modelos específicos que apliquen dichos algoritmos a problemas de optimización más cercanos al ámbito de la ingeniería.

De esta forma, el proyecto no se limita a reproducir el comportamiento teórico de los algoritmos, también busca mostrar cómo pueden adaptarse a problemas concretos, con variables, restricciones y funciones objetivo diferentes. Esta doble perspectiva (modelo básico y modelo específico) resulta especialmente útil, ya que permite utilizar los modelos tanto para introducir los fundamentos de cada algoritmo como para estudiar su aplicación práctica en escenarios más realistas.

#### **1.5 Descripción del proyecto**

El proyecto desarrollado consiste en la implementación de seis modelos en NetLogo: tres modelos básicos que muestren el funcionamiento de WOA, GEO y CSA respectivamente, y tres modelos específicos de cada uno de los algoritmos en situaciones reales de ingeniería civil.

Los modelos básicos están orientados a mostrar el comportamiento general de cada algoritmo sobre espacios de búsqueda sencillos. En ellos se busca optimizar el mínimo tanto en un espacio homogéneo como es la función esfera  $f(x,y) = x^2 + y^2$ , como en paisajes aleatorios que permiten observar el comportamiento de los algoritmos en entornos más irregulares con múltiples regiones de interés. Estos modelos incorporan visualización tridimensional del espacio de búsqueda, representación de los agentes, gráficas de evolución del fitness y controles interactivos para modificar los principales parámetros.

Los modelos específicos para cada algoritmo son:

- Para el WOA la optimización estructural de una celosía espacial de 72 barras.[4] El objetivo es minimizar el peso total de una estructura tridimensional seleccionando secciones transversales discretas para grupos de barras, respetando restricciones de tensión y desplazamiento.
- Para el GWO un problema de regulación y reparto de agua en un canal de riego real en Irán.[5] Aquí cada solución representa un conjunto de aperturas de compuertas en un canal de riego, con el propósito de mejorar el reparto de agua entre distintos puntos de entrega.
- Para el CSA la optimización del domo espacial de 120 barras.[6] Cada diseño representará las áreas de los grupos de barras de un domo espacial, sujeto a restricciones sobre sus dos primeras frecuencias naturales

Esta combinación de modelos básicos y específicos permite que el proyecto tenga una doble utilidad. Por un lado, sirve como herramienta didáctica para entender el comportamiento de los algoritmos. Por otro, muestra cómo esos mismos algoritmos pueden adaptarse a problemas de ingeniería con restricciones y objetivos más complejos.

## 1.6 Objetivos

El objetivo general de este trabajo es desarrollar modelos interactivos en NetLogo que permitan simular, visualizar y analizar el comportamiento de distintos algoritmos bioinspirados aplicados a problemas de optimización.

A partir de este objetivo general, se plantean los siguientes objetivos específicos:

- Estudiar los fundamentos teóricos de los algoritmos WOA, GWO y CSA
- Implementar un modelo básico para cada algoritmo en NetLogo 3D
- Representar visualmente el espacio de búsqueda y la evolución de la población de soluciones
- Permitir la modificación de parámetros mediante la interfaz de usuario
- Implementar modelos específicos aplicados a problemas de optimización estructural e hidráulica
-

- Realizar experimentos con distintos tamaños de población, iteraciones y semillas aleatorias
- Analizar los resultados obtenidos y el comportamiento de cada algoritmo
- Documentar los modelos para facilitar su uso con fines docentes.

## 1.7 Estructura de la memoria

La presente memoria se organiza en varios capítulos.

En el *Capítulo 1* se introduce el contexto del trabajo, la motivación del proyecto, sus antecedentes, la descripción general y los objetivos perseguidos.

En el *Capítulo 2* se presenta la gestión y planificación del proyecto, incluyendo la metodología de desarrollo, las tareas realizadas, la planificación temporal, los recursos utilizados y la estimación de costes.

En el *Capítulo 3* se revisa el estado del arte relacionado con los algoritmos bioinspirados, las metaheurísticas de optimización y el uso de herramientas de simulación como NetLogo.

En el *Capítulo 4* se describen los fundamentos teóricos de los algoritmos implementados: WOA, GWO y CSA, así como los problemas de optimización utilizados en los modelos específicos.

En el *Capítulo 5* se expone el análisis y diseño de los modelos desarrollados, detallando los requisitos funcionales y no funcionales, la estructura común de los modelos y el diseño de las interfaces.

En el *Capítulo 6* se describe la implementación de los modelos básicos y específicos en NetLogo, explicando las decisiones tomadas para representar las soluciones, evaluar la función objetivo y actualizar la población de agentes.

En el *Capítulo 7* se presentan las pruebas realizadas y los resultados obtenidos, tanto en los modelos básicos como en los modelos específicos.

En el *Capítulo 8* se recogen las conclusiones del trabajo, las limitaciones encontradas y posibles líneas de trabajo futuro.

Finalmente, en el *Capítulo 9* se muestra la bibliografía utilizada para la elaboración de la memoria.

## **CAPITULO 2**

### **GESTIÓN Y PLANIFICACIÓN**

#### **2.1. Tareas a realizar.**

Recordemos que debemos desarrollar modelos interactivos en NetLogo 3D para simular y visualizar el comportamiento de WOA, GWO y CSA, tanto en entornos básicos de optimización como en problemas específicos de ingeniería. Además, se han de realizar experimentos para comparar los resultados entre ellos en el caso del básico que es común a los 3, y contra los papers de cada caso específico.

Cada objetivo debe contener una o varias tareas que lo lleven a cabo.

- Tarea 1. Implementar modelo básico para algoritmo de ballenas.
  - T 1.1. Investigación sobre el algoritmo de ballenas
  - T 1.2. Diseñar y desarrollar modelo básico del algoritmo de ballenas en NetLogo
  - T 1.3. Evaluación y pruebas del modelo básico de ballenas.
  
- Tarea 2. Implementar modelo básico para algoritmo del lobo gris.
  - T 2.1. Investigación sobre el algoritmo del lobo gris
  - T 2.2. Diseñar y desarrollar modelo básico del algoritmo del lobo gris en NetLogo
  - T 2.3. Evaluación y pruebas del modelo básico del lobo gris.
  
- Tarea 3. Implementar modelo básico para algoritmo de camaleones.
  - T 3.1. Investigación sobre el algoritmo de camaleones
  - T 3.2. Diseñar y desarrollar modelo básico del algoritmo de camaleones en NetLogo
  - T 3.3. Evaluación y pruebas del modelo básico de camaleones.

- Tarea 4. Implementar modelo específico para algoritmo de ballenas.
  - T 4.1. Investigación y búsqueda sobre problemas de ingeniería resueltos a través de algoritmo de ballenas.
  - T 4.2. Diseñar y desarrollar modelo específico de ballenas en NetLogo.
  
- Tarea 5. Implementar modelo específico para algoritmo de lobo gris.
  - T 5.1. Investigación y búsqueda sobre problemas de ingeniería resueltos a través de algoritmo de lobo gris.
  - T 5.2. Diseñar y desarrollar modelo específico de lobo gris en NetLogo.
  
- Tarea 6. Implementar modelo específico para algoritmo de camaleones.
  - T 6.1. Investigación y búsqueda sobre problemas de ingeniería resueltos a través de algoritmo de camaleones.
  - T 6.2. Diseñar y desarrollar modelo específico de camaleones en NetLogo.
  
- Tarea 7. Comparación de las soluciones que se muestran en artículos científicos con las que se han logrado a través de los modelos específicos implementados.
  - T 7.1. Comparación para ballenas específico.
  - T 7.2. Comparación para lobo gris específico.
  - T 7.3. Comparación para camaleón específico.
  
- Tarea 8. Crear un manual de usuario para facilitar el uso e introducción de la herramienta al alumnado.

### **2.3. Metodología de desarrollo.**

Antes de comenzar el desarrollo, resultaba necesario organizar el trabajo de una forma realista. El proyecto no consistía únicamente en implementar un algoritmo aislado, sino en construir seis modelos diferentes, documentarlos y comprobar su funcionamiento. Además, cada modelo tenía sus propias particularidades: algunos estaban orientados a la visualización básica del algoritmo y otros a problemas de optimización más concretos.

Por este motivo, se decidió seguir una forma de trabajo flexible, cercana a una metodología ágil. La idea principal era avanzar de forma incremental, dividiendo el trabajo en partes más pequeñas y manejables. Así, en lugar de intentar definir desde el principio todos los detalles del proyecto, se fue trabajando por bloques: primero el estudio de los algoritmos, después los modelos básicos, más adelante los modelos específicos y, finalmente, la fase de pruebas y documentación.

Este enfoque resultaba adecuado porque durante el desarrollo podían aparecer cambios o ajustes. Por ejemplo, al implementar un algoritmo en NetLogo, no siempre basta con trasladar directamente sus ecuaciones teóricas al código. También hay que adaptar la representación de los agentes, la visualización en 3D, los parámetros de la interfaz y la forma de mostrar los resultados. En ese sentido, una planificación demasiado rígida habría sido poco práctica.

Entre las distintas metodologías ágiles existentes, se tomó Scrum<sup>[7]</sup> como referencia, aunque adaptándolo a las características de un trabajo fin de grado realizado de forma individual. Scrum propone organizar el trabajo en iteraciones o sprints, cada una de ellas centrada en obtener un resultado funcional. Esta forma de trabajar encajaba bien con el proyecto, ya que cada modelo podía desarrollarse, probarse y mejorarse de manera progresiva.



### **2.3.1. SCRUM.**

Aunque Scrum está pensado originalmente para equipos de desarrollo, en este proyecto se ha utilizado como una guía de organización personal. Al tratarse de un trabajo individual, algunos elementos de la metodología no se aplicaron de forma estricta. Por ejemplo, no tenían sentido las reuniones diarias propias de un equipo Scrum, ni una división formal de roles como Scrum Master, Product Owner y equipo de desarrollo.

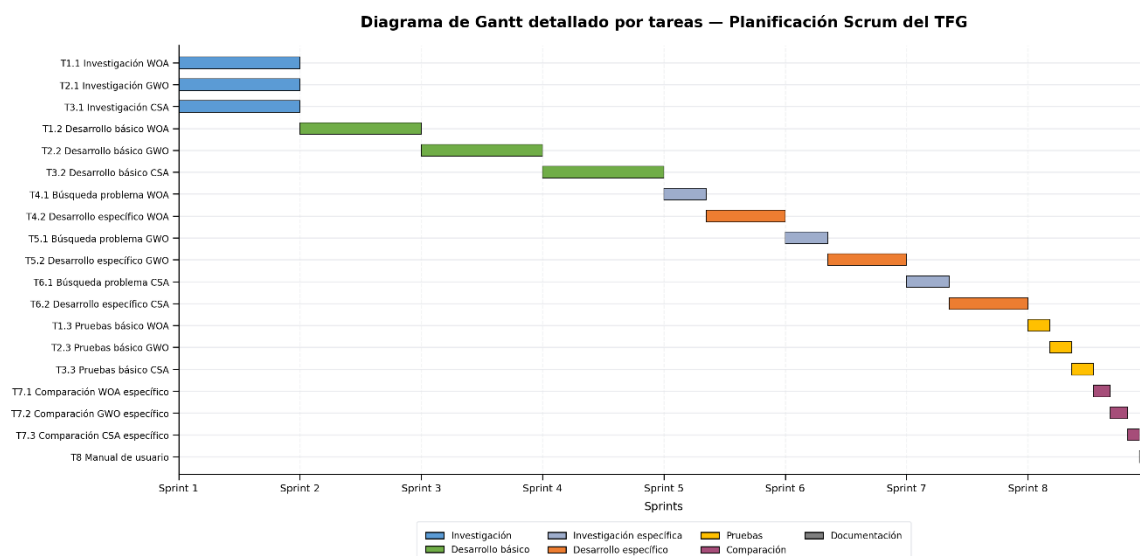
En este caso, el autor del proyecto asumió las tareas de análisis, diseño, implementación, pruebas y documentación. La tutora desempeñó un papel de orientación y revisión, ayudando a validar el enfoque seguido y a resolver dudas relacionadas con los algoritmos, los modelos y los problemas de optimización seleccionados.

La planificación se organizó en iteraciones centradas en entregables concretos. En cada una de ellas se trabajó sobre una parte específica del proyecto, intentando obtener siempre un resultado funcional antes de pasar a la siguiente fase. Esto permitió avanzar con más seguridad: primero se desarrollaron los modelos básicos, donde era más sencillo comprobar el comportamiento general de cada algoritmo, y posteriormente se abordaron los modelos específicos, que requerían una adaptación más compleja.

Los principales sprints del proyecto fueron los siguientes:

| Sprint   | Trabajo realizado                | Resultado                                 | Tareas                                  |
|----------|----------------------------------|-------------------------------------------|-----------------------------------------|
| Sprint 1 | Estudio de WOA, GWO y CSA        | Base teórica y selección de algoritmos    | T1.1, T2.1 y T3.1                       |
| Sprint 2 | Desarrollo del modelo básico WOA | Simulador WOA en NetLogo 3D               | T1.2                                    |
| Sprint 3 | Desarrollo del modelo básico GWO | Simulador GWO en NetLogo 3D               | T2.2.                                   |
| Sprint 4 | Desarrollo del modelo básico CSA | Simulador CSA en NetLogo 3D               | T3.2                                    |
| Sprint 5 | Modelo específico WOA            | Celosía espacial de 72 barras             | T4.1 y T4.2                             |
| Sprint 6 | Modelo específico GWO            | Canal de riego con turnouts y check gates | T5.1 y T5.2                             |
| Sprint 7 | Modelo específico CSA            | Domo espacial de 120 barras               | T6.1 y T6.2                             |
| Sprint 8 | Experimentos y documentación     | Resultados, memoria y manual              | T1.3, T2.3, T3.3, T7.1, T7.2, T7.3 y T8 |

En la siguiente imagen se puede ver más detalladamente el diagrama de Gantt, desglosando las tareas y subtareas incluidas en cada sprint:



## **2.4. Gestión de recursos.**

Para poder desarrollar el proyecto ha sido necesario contar con una serie de recursos humanos, hardware, software y documentales. Aunque se trata de un trabajo fin de grado realizado de forma individual, el proyecto ha requerido organizar distintos tipos de recursos, ya que no solo se ha desarrollado código, sino también modelos visuales, pruebas experimentales y documentación asociada.

En este apartado se describen los principales recursos empleados durante el desarrollo del trabajo.

### **2.4.1. Recursos humanos**

El principal recurso humano del proyecto ha sido el propio autor del trabajo, encargado de llevar a cabo las tareas de análisis, diseño, implementación, pruebas y documentación de los modelos desarrollados.

Durante el proyecto se han asumido varias funciones. En primer lugar, ha sido necesario estudiar los fundamentos teóricos de los algoritmos bioinspirados seleccionados: algoritmo de la ballena, algoritmo del lobo gris y algoritmo del camaleón. Después, se ha realizado la adaptación de estos algoritmos al entorno de NetLogo, tanto en sus versiones básicas como en sus aplicaciones específicas a problemas de ingeniería.

Además de la parte de programación, también se han llevado a cabo tareas de diseño de la interfaz, preparación de experimentos, análisis de resultados y redacción de la memoria. La verdad es que este tipo de proyecto obliga a cambiar bastante de rol: en algunos momentos se trabaja como programador, en otros como investigador, y en otros como usuario que intenta comprobar si el modelo realmente se entiende y se puede utilizar de forma cómoda.

La tutora del proyecto, Rocío Romero Zaliz, ha actuado como guía durante el desarrollo, orientando el enfoque general del trabajo y ayudando a validar las decisiones tomadas. Su papel ha sido especialmente importante en la selección y adaptación de los modelos, así como en la revisión de la evolución del proyecto.

Por tanto, los recursos humanos principales han sido:

- Autor del proyecto: Raúl Caubera Jiménez.
- Tutora del proyecto: Rocío Romero Zaliz.

#### **2.4.2. Recursos hardware**

El desarrollo del proyecto se ha llevado a cabo utilizando un ordenador personal. Este equipo ha sido suficiente para ejecutar NetLogo 3D, desarrollar los modelos, realizar pruebas y preparar la documentación del trabajo.

Aunque los modelos desarrollados no requieren una infraestructura hardware especialmente compleja, sí era necesario disponer de un equipo capaz de ejecutar simulaciones tridimensionales con un número razonable de agentes. En los modelos básicos, por ejemplo, se trabaja con poblaciones de partículas, lobos o camaleones que se desplazan sobre un espacio de búsqueda representado en 3D. Por ello, el rendimiento gráfico del equipo influye en la fluidez de la simulación, especialmente cuando se utilizan tamaños de población elevados o paisajes aleatorios más complejos.

Los principales recursos hardware utilizados han sido:

- Ordenador personal para el desarrollo y ejecución de los modelos. (Portatil ASUS Predator PH317-52 con procesador Intel Core i7-8750H, 32GB de memoria RAM, disco nvme de 256GB, tarjeta gráfica GeForce GTX 1050ti y sistema operativo Windows 11 Pro en su versión 25H2)
- Pantalla para la visualización de la interfaz y del entorno 3D de NetLogo (Monitor panorámico de 34" AOC modelo CU34G4 WQHD 180HZ).
- Unidad de almacenamiento local para guardar modelos, resultados y documentación, así como copias automáticas en disco externo.
- Periféricos habituales de trabajo, como teclado y ratón.

### **2.4.3. Recursos software**

El recurso software principal utilizado en este proyecto ha sido NetLogo 3D, ya que todos los modelos se han desarrollado en este entorno. NetLogo permite crear simulaciones basadas en agentes y proporciona una interfaz sencilla para incorporar controles, botones, interruptores, gráficas y monitores. Esto ha sido especialmente útil para el objetivo del proyecto, ya que no se buscaba únicamente implementar los algoritmos, sino también hacerlos visuales e interactivos.

En los modelos básicos, NetLogo se ha utilizado para representar el espacio de búsqueda, los agentes que se desplazan por él y la evolución del valor de fitness. Además, se han incorporado controles para modificar parámetros como el tamaño de la población, el número de iteraciones, el presupuesto de evaluaciones o los parámetros propios de cada algoritmo.

También se han utilizado como herramienta de edición de texto para la elaboración de la memoria y la documentación del proyecto el programa Word del paquete ofimático de Microsoft Office, debido a que en mi trabajo diario ya lo uso y me es más familiar y fácil e intuitivo para desarrollarlo.

Además, para el análisis de resultados se han utilizado archivos en formato CSV, generados desde los propios modelos de NetLogo mediante procedimientos de exportación. Para gestionar dichos archivos se ha hecho uso de Excel también incluido en el mismo paquete Office.

Los principales recursos software empleados han sido:

- NetLogo 3D, para el desarrollo y ejecución de los modelos.
- Herramienta de edición de documentos (Microsoft Word) para la redacción de la memoria.
- Herramientas de hoja de cálculo (Excel) para revisar o representar resultados experimentales.
- Sistema de almacenamiento local, para organizar las versiones de los modelos y la documentación.

Para la organización del código y las distintas versiones del proyecto se utilizó un repositorio en Github (<https://github.com/raulcaubera/algoritmos-bioinspirados>) lo que permitió mantener una copia ordenada de los modelos y reducir el riesgo de pérdida de información.

#### 2.4.4. Recursos documentales

Además de los recursos técnicos, el proyecto ha requerido una fase importante de documentación. Para poder implementar correctamente los algoritmos y adaptarlos a los distintos problemas de optimización, ha sido necesario consultar artículos científicos, documentación técnica y trabajos previos relacionados.

En primer lugar, se han utilizado como referencia los artículos originales o principales de los algoritmos implementados: WOA, GWO y CSA. Estos documentos han servido para comprender el funcionamiento matemático de cada método, sus fases principales, sus parámetros y la forma en que actualizan las soluciones candidatas.

También se han consultado referencias específicas para los problemas de optimización tratados en los modelos específicos. En el caso del modelo basado en GWO, se ha tomado como inspiración el trabajo relacionado con la planificación y regulación del agua en canales de riego. Para los modelos estructurales, se han utilizado problemas benchmark ampliamente conocidos, como la celosía espacial de 72 barras y el domo espacial de 120 barras.

Por otro lado, se ha tenido en cuenta el trabajo fin de grado anterior relacionado con algoritmos bioinspirados en NetLogo. Este trabajo ha servido como punto de partida metodológico y como referencia para mantener cierta continuidad en la línea de modelos educativos. Sin embargo, el presente proyecto amplía dicha línea mediante la incorporación de nuevos algoritmos y nuevos problemas específicos de ingeniería.

Los principales recursos documentales utilizados han sido:

- Artículos científicos sobre el Whale Optimization Algorithm.[\[1\]](#)
- Artículos científicos sobre el Grey Wolf Optimizer.[\[2\]](#)
- Artículos científicos sobre el Chameleon Swarm Algorithm.[\[3\]](#)
- Artículo relacionado con la aplicación de GWO a la planificación del riego.[\[5\]](#)
- Referencias sobre el problema estructural de la celosía espacial de 72 barras.[\[4\]](#)
- Referencias sobre el problema del domo espacial de 120 barras.[\[6\]](#)
- Documentación oficial y recursos de apoyo de NetLogo.[\[8\]](#)
- TFG previo sobre algoritmos bioinspirados en NetLogo.[\[9\]](#)

Estos recursos han sido necesarios no solo para implementar los algoritmos, sino también para entender cómo adaptar cada uno a un entorno visual e interactivo. En muchos casos, la dificultad no estaba únicamente en programar las fórmulas, sino en decidir cómo representar cada solución, cómo mostrar la evolución del algoritmo y cómo hacer que el modelo resultara comprensible para un usuario que lo utiliza con fines docentes.

Al tratarse de un proyecto principalmente software, el coste material ha sido reducido, ya que se han utilizado herramientas accesibles y un equipo personal con cierto tiempo y uso acumulado, aunque aun plenamente operativo. Aun así, el esfuerzo principal del proyecto se ha concentrado en el tiempo dedicado al estudio, implementación, experimentación y redacción de la memoria.

## **2.5. Gestión de costes.**

Aunque este proyecto se ha desarrollado en un contexto académico y no ha supuesto una contratación real, resulta útil realizar una estimación aproximada del coste que tendría si se valorase como un proyecto software profesional. Para ello se han tenido en cuenta principalmente las horas de trabajo dedicadas, el uso de recursos hardware y las herramientas software empleadas.

El coste más importante del proyecto no corresponde al material utilizado, sino al tiempo invertido en el estudio de los algoritmos, la implementación de los modelos, la realización de pruebas y la redacción de la documentación. Al tratarse de un Trabajo Fin de Grado de 12 créditos, puede estimarse una dedicación aproximada de entre 300 y 360 horas, considerando una carga habitual de entre 25 y 30 horas por crédito.

Estas horas se han distribuido entre distintas tareas: búsqueda de información, análisis de los algoritmos, desarrollo de los modelos básicos, adaptación a problemas específicos, pruebas experimentales, corrección de errores y elaboración de la memoria. La verdad es que no todas las horas han tenido el mismo tipo de trabajo; algunas han sido más teóricas, centradas en comprender artículos y modelos matemáticos, mientras que otras han sido puramente prácticas, dedicadas a programar, probar y ajustar los modelos en NetLogo.

### 2.5.1. Costes de recursos humanos

Para estimar el coste de recursos humanos se ha tomado como referencia un coste aproximado de 15 €/hora, correspondiente a un perfil junior de desarrollo o apoyo técnico. No se trata de un coste real pagado, sino de una valoración orientativa del esfuerzo dedicado al proyecto.

| Concepto                         | Estimación |
|----------------------------------|------------|
| Horas mínimas estimadas          | 300 h      |
| Horas máximas estimadas          | 360 h      |
| Coste/hora estimado              | 15 €/h     |
| Coste mínimo de recursos humanos | 4.500 €    |
| Coste máximo de recursos humanos | 5.400 €    |

Por tanto, el coste asociado al trabajo personal se situaría aproximadamente entre 4.500 € y 5.400 €.

### 2.5.2. Costes hardware

El desarrollo se ha realizado utilizando un ordenador portátil personal. El equipo tiene aproximadamente ocho años de antigüedad y, aunque en el momento de su compra tuvo un coste cercano a los 1.200 €, actualmente su valor es considerablemente inferior debido al uso y a la depreciación propia del hardware. Para esta estimación no se considera el precio original completo del portátil, ya que no ha sido adquirido específicamente para el proyecto. En su lugar, se puede considerar un valor residual aproximado de 200 €, asociado al uso del equipo durante el desarrollo.

Además, durante el proyecto se adquirió un monitor de 34 pulgadas, principalmente para mejorar la comodidad de trabajo. Este recurso ha sido útil para desarrollar los modelos, consultar documentación y redactar la memoria con varias ventanas abiertas al mismo tiempo, algo especialmente cómodo cuando se trabaja con NetLogo, artículos científicos y documentación de forma simultánea.



Como el monitor no se ha utilizado exclusivamente para el TFG, se puede imputar solo una parte de su coste al proyecto. Suponiendo un coste aproximado de 300 € y una imputación del 50 %, el coste asociado sería de 150 €.

| <b>Recurso hardware</b>                | <b>Coste estimado imputado</b> |
|----------------------------------------|--------------------------------|
| Ordenador portátil personal amortizado | 200 €                          |
| Monitor de 34 pulgadas                 | 150 €                          |
| Total hardware                         | 350 €                          |

### **2.5.3. Costes software**

En cuanto al software, el proyecto se ha desarrollado principalmente con herramientas gratuitas o ya disponibles. NetLogo es una herramienta accesible para el desarrollo de modelos basados en agentes, y no ha supuesto un coste adicional. Del mismo modo, la redacción de la memoria y el tratamiento de resultados se han realizado con herramientas ya disponibles para el autor (el paquete de Microsoft Office está disponible para el estudiantado de la Universidad de Granada).

| <b>Recurso software</b>                    | <b>Coste estimado</b> |
|--------------------------------------------|-----------------------|
| NetLogo / NetLogo 3D                       | 0 €                   |
| Editor de documentos (Word)                | 0 €                   |
| Herramientas de hoja de cálculo (Excel)    | 0 €                   |
| Almacenamiento local / copias de seguridad | 0 €                   |
| Total software                             | 0 €                   |

#### 2.5.4. Presupuesto total estimado

Sumando los costes de recursos humanos, hardware y software, se obtiene la siguiente estimación global:

| Concepto         | Coste mínimo |
|------------------|--------------|
| Recursos humanos | 4.500 €      |
| Hardware         | 350 €        |
| Software         | 0 €          |
| Total estimado   | 4.850 €      |

Por tanto, el coste total estimado del proyecto se situaría aproximadamente entre 4.850 € y 5.750 €.

Cabe destacar que esta cifra representa una valoración orientativa del esfuerzo y los recursos empleados. En la práctica, el proyecto se ha desarrollado con medios propios y dentro del contexto académico del trabajo fin de grado, por lo que no ha supuesto un gasto directo equivalente a dicha cantidad.

#### 2.6. Gestión de riesgos.

Durante el desarrollo del proyecto se identificaron una serie de riesgos que podían afectar al avance del trabajo, tanto desde el punto de vista técnico como desde el punto de vista organizativo. Aunque se trata de un trabajo fin de grado individual, el proyecto tenía cierta complejidad, ya que no se limitaba a implementar un único algoritmo, sino que incluía seis modelos diferentes: tres modelos básicos y tres modelos específicos aplicados a problemas de optimización.

Además, el desarrollo se realizó compaginando el TFG con una jornada laboral completa (en jornada partida para mas inri). Esto supuso una dificultad añadida en la planificación, ya que el tiempo disponible para trabajar en el proyecto era más limitado y debía organizarse en bloques más reducidos. Por este motivo, fue importante dividir el trabajo en tareas pequeñas y avanzar de forma incremental, evitando dejar partes demasiado grandes sin cerrar.

La gestión de riesgos no consistió en eliminar por completo todos los problemas posibles, algo poco realista, sino en identificarlos con antelación y establecer medidas para reducir su impacto. En la siguiente tabla se recogen los principales riesgos detectados durante el desarrollo del proyecto.

### 2.6.1. Identificación de riesgos

| Riesgo                                                                         | Probabilidad | Impacto | Medida de mitigación                                                                                           |
|--------------------------------------------------------------------------------|--------------|---------|----------------------------------------------------------------------------------------------------------------|
| Disponibilidad limitada por compaginar el TFG con una jornada laboral completa | Alta         | Alto    | Dividir el trabajo en tareas pequeñas, priorizar los modelos principales y mantener una planificación flexible |
| Dificultad para adaptar algoritmos matemáticos a NetLogo                       | Media        | Alto    | Implementar primero versiones básicas sobre la función esfera antes de pasar a problemas específicos           |
| Errores en la visualización 3D                                                 | Media        | Medio   | Separar, en la medida de lo posible, la lógica del algoritmo de la parte visual del modelo                     |
| Convergencia prematura a óptimos locales                                       | Media        | Medio   | Realizar pruebas con distintos tamaños de población, iteraciones y semillas aleatorias                         |
| Dificultad en la implementación de los modelos específicos                     | Alta         | Alto    | Validar previamente el comportamiento de los modelos básicos y adaptar los problemas por fases                 |
| Falta de resultados comparables                                                | Media        | Medio   | Utilizar artículos científicos y problemas benchmark como referencia                                           |
| Pérdida de código o documentación                                              | Baja         | Alto    | Mantener copias de seguridad y organizar los archivos por versiones                                            |
| Sobrecarga de alcance del proyecto                                             | Media        | Alto    | Separar claramente los modelos básicos de los específicos y evitar añadir funcionalidades no necesarias        |
| Errores difíciles de detectar en la función objetivo o restricciones           | Media        | Alto    | Revisar los cálculos por partes y probar casos simples antes de ejecutar experimentos completos                |

### **2.6.2. Descripción de los principales riesgos**

Uno de los riesgos más importantes fue la disponibilidad limitada de tiempo. Al realizar el proyecto al mismo tiempo que una jornada laboral completa, no siempre era posible dedicar sesiones largas y continuadas al TFG. Esto podía provocar retrasos o pérdida de continuidad, especialmente en tareas que requerían concentración, como la depuración de errores o la adaptación de los algoritmos a problemas específicos. Para reducir este riesgo, se intentó dividir el desarrollo en tareas pequeñas y cerradas, de forma que cada avance dejara una parte del proyecto más estable.

Otro riesgo relevante fue la adaptación de algoritmos matemáticos a NetLogo. Los algoritmos WOA, GWO y CSA se describen normalmente mediante ecuaciones y pseudocódigo, pero llevarlos a un entorno visual basado en agentes requiere tomar decisiones adicionales. Por ejemplo, no basta con actualizar vectores de soluciones; también hay que representar los agentes, mostrar su posición, calcular el fitness y actualizar la visualización en cada iteración. Para evitar problemas mayores, se implementaron primero modelos básicos sobre funciones sencillas, como la función esfera.

También existía el riesgo de que los algoritmos presentaran convergencia prematura. Este comportamiento puede aparecer cuando la población se concentra demasiado pronto en una región del espacio de búsqueda y deja de explorar otras zonas potencialmente mejores. Para analizar este aspecto, se realizaron pruebas con distintos tamaños de población, diferentes números de iteraciones y varias semillas aleatorias.

Los modelos específicos supusieron otra fuente importante de riesgo. A diferencia de los modelos básicos, estos no se limitan a optimizar una función sencilla, sino que incorporan variables, restricciones y funciones objetivo asociadas a problemas de ingeniería. En el caso de la celosía de 72 barras, por ejemplo, era necesario tener en cuenta restricciones de tensión y desplazamiento; en el canal de riego, las aperturas de compuertas y restricciones de peticiones; y en el domo de 120 barras, restricciones sobre frecuencias naturales. Por ello, se decidió abordar estos modelos después de haber validado primero el comportamiento general de cada algoritmo.

Por último, también se tuvo en cuenta el riesgo de sobrecargar el alcance del proyecto. Al trabajar con tres algoritmos y seis modelos, existía la tentación de

añadir más funcionalidades, más gráficas o más comparativas. Sin embargo, esto podía hacer que el proyecto se volviera demasiado grande para el tiempo disponible. Para controlar este riesgo, se mantuvo una separación clara entre modelos básicos y modelos específicos, priorizando que cada uno funcionara correctamente y estuviera documentado.

### **2.6.3. Riesgos materializados**

A lo largo del desarrollo, algunos de estos riesgos llegaron a materializarse parcialmente. El más evidente fue la limitación de tiempo disponible, ya que compatibilizar el TFG con el trabajo obligó a organizar el avance de forma más cuidadosa. En algunos momentos, esto hizo que ciertas tareas se alargaran más de lo previsto, especialmente las relacionadas con depuración y documentación. También aparecieron dificultades técnicas durante la adaptación de los algoritmos a NetLogo 3D. Algunos comportamientos que en teoría resultaban sencillos de expresar mediante fórmulas necesitaban ajustes para funcionar correctamente dentro del entorno de simulación. Esto ocurrió especialmente al combinar la lógica del algoritmo con la representación visual del espacio de búsqueda.

En cualquier caso, estos problemas pudieron gestionarse mediante un desarrollo incremental. Primero se buscó que cada modelo funcionara de forma básica; después se añadieron mejoras visuales, controles de interfaz y procedimientos de experimentación. Esta forma de trabajar permitió avanzar sin bloquear el proyecto completo ante un único problema técnico.

### **2.6.4. Conclusión sobre la gestión de riesgos**

La identificación de riesgos permitió afrontar el proyecto de forma más realista. No todos los problemas podían preverse desde el inicio, pero sí era posible reducir su impacto mediante una planificación flexible, pruebas progresivas y una división clara del trabajo.

En particular, el hecho de compaginar el desarrollo del TFG con una jornada laboral completa hizo necesario organizar el proyecto en objetivos alcanzables y evitar una planificación demasiado rígida. Gracias a esta forma de trabajo, fue posible completar los modelos básicos y específicos, realizar pruebas y documentar el funcionamiento de la herramienta desarrollada.

## **CAPITULO 3 ESTADO DEL ARTE**

### **3.1 Introducción**

Los algoritmos bioinspirados han ganado bastante importancia en los últimos años dentro del campo de la optimización. Esto se debe, sobre todo, a que muchos problemas reales no pueden resolverse de forma sencilla mediante métodos exactos, ya sea por el tamaño del espacio de búsqueda, por la presencia de restricciones o por la existencia de múltiples óptimos locales.

En este trabajo se estudian tres algoritmos bioinspirados: Whale Optimization Algorithm, Grey Wolf Optimizer y Chameleon Swarm Algorithm. Los tres pertenecen al grupo de algoritmos basados en población, por lo que trabajan con un conjunto de soluciones candidatas que evolucionan a lo largo de las iteraciones.

Además, el proyecto no se limita a explicar los algoritmos de forma teórica. También se implementan en NetLogo 3D mediante modelos básicos y modelos específicos. Los modelos básicos permiten visualizar su comportamiento sobre funciones sencillas, mientras que los modelos específicos aplican cada algoritmo a un problema de ingeniería.

En este capítulo se presentan tanto el contexto general de las metaheurísticas como los fundamentos teóricos de los algoritmos seleccionados, las herramientas consideradas y los problemas de optimización empleados.

### **3.2 Metaheurísticas y algoritmos bioinspirados**

Las metaheurísticas son estrategias generales de búsqueda diseñadas para resolver problemas de optimización complejos. A diferencia de los métodos exactos, no garantizan necesariamente encontrar la mejor solución posible, pero sí buscan obtener soluciones suficientemente buenas en un tiempo razonable. Esta característica las hace especialmente interesantes en problemas donde evaluar todas las posibles soluciones sería inviable.

Dentro de las metaheurísticas se encuentran los algoritmos bioinspirados. Estos métodos toman como referencia comportamientos observados en la naturaleza y los transforman en reglas de búsqueda. Algunos se basan en la evolución de las especies, otros en el movimiento colectivo de enjambres, en la cooperación entre animales o en estrategias de caza y supervivencia. La verdad es que esta idea resulta bastante atractiva: usar comportamientos naturales, que han demostrado ser eficaces en entornos reales, como inspiración para resolver problemas matemáticos y de ingeniería.

En este tipo de algoritmos, normalmente se trabaja con una población de soluciones candidatas. Cada individuo de la población representa una posible solución del problema, y su calidad se mide mediante una función objetivo o función de aptitud. A lo largo de las iteraciones, las soluciones se modifican siguiendo ciertas reglas, intentando mejorar progresivamente el resultado obtenido.

Un aspecto importante de estas técnicas es el equilibrio entre exploración y explotación. La exploración permite recorrer zonas amplias del espacio de búsqueda para evitar quedarse demasiado pronto en una solución local. La explotación, en cambio, se centra en mejorar las soluciones prometedoras que ya se han encontrado. Un buen algoritmo debe combinar ambas fases de forma adecuada: explorar lo suficiente al principio y afinar la búsqueda conforme avanza la ejecución.

### **3.3 Herramientas para simulación educativa**

El desarrollo y análisis de algoritmos de optimización puede realizarse mediante distintas herramientas. La elección de una u otra depende del objetivo del trabajo. No es lo mismo buscar el máximo rendimiento computacional que construir una herramienta visual para comprender el comportamiento de un algoritmo.

En este proyecto, el objetivo principal no era competir con implementaciones optimizadas en lenguajes de bajo nivel, sino crear modelos interactivos y comprensibles desde el punto de vista docente. Por ello, se consideró especialmente importante que la herramienta permitiera representar agentes, visualizar el espacio de búsqueda y modificar parámetros de forma sencilla.

### **3.3.1 NetLogo**

NetLogo es un entorno de modelado basado en agentes. Permite definir individuos, reglas de comportamiento y entornos sobre los que estos individuos interactúan. Esta característica encaja muy bien con los algoritmos bioinspirados basados en población, ya que cada solución candidata puede representarse como un agente que se desplaza por el espacio de búsqueda.

Una de las principales ventajas de NetLogo es su interfaz interactiva. Mediante botones, interruptores, deslizadores y gráficas, el usuario puede ejecutar el modelo, modificar parámetros y observar la evolución del sistema. Además, NetLogo 3D permite representar superficies y agentes en un entorno tridimensional, lo que resulta especialmente útil para visualizar funciones objetivo sencillas o paisajes generados de forma aleatoria.

Como inconveniente, NetLogo no está pensado principalmente para realizar cálculos numéricos de alto rendimiento. Por tanto, puede no ser la opción más eficiente para problemas de gran tamaño o experimentos masivos. Sin embargo, para un objetivo docente y visual como el de este proyecto, ofrece un equilibrio muy adecuado entre sencillez, interacción y capacidad de representación.

### **3.3.2 Python**

Python es una de las herramientas más utilizadas actualmente para implementar algoritmos de optimización. Cuenta con numerosas bibliotecas científicas y permite realizar cálculos, generar gráficas, gestionar datos y automatizar experimentos de forma muy flexible.

Su principal ventaja es la potencia y versatilidad. Es posible implementar algoritmos complejos, procesar resultados y generar comparativas de manera relativamente cómoda. Sin embargo, para estudiantes sin experiencia previa en programación, puede resultar menos visual al principio. Aunque existen bibliotecas para crear interfaces o animaciones, estas suelen requerir trabajo adicional.



### 3.3.3 MATLAB

MATLAB es otra herramienta muy habitual en ingeniería y optimización. Ofrece un entorno orientado al cálculo numérico y dispone de funciones avanzadas para trabajar con matrices, representar datos y resolver problemas matemáticos.

Su uso es frecuente en investigación y docencia técnica, especialmente en áreas relacionadas con estructuras, control, señales y optimización. No obstante, presenta algunas limitaciones para el objetivo de este trabajo. Por un lado, no está tan orientado al modelado basado en agentes como NetLogo. Por otro, su licencia puede suponer una barrera en determinados contextos.

### 3.3.4 Simuladores específicos

También existen simuladores o aplicaciones específicas para determinados algoritmos o problemas de optimización. Estas herramientas pueden resultar útiles cuando están muy bien adaptadas a un caso concreto, ya que permiten probar algoritmos, modificar parámetros y observar visualmente el proceso de búsqueda.

Un ejemplo es *PSO Simulator*, una aplicación web dedicada al algoritmo Particle Swarm Optimization. Esta herramienta permite ejecutar PSO sobre distintas funciones benchmark, como Sphere, Rastrigin, Rosenbrock o Griewank, modificando parámetros como el tamaño de población o los coeficientes del algoritmo. <https://thiagodnf.github.io/psa-simulator/>

Otro ejemplo es *Swarm Wave*, una plataforma educativa orientada al estudio de algoritmos de inteligencia de enjambre. Esta herramienta incluye soporte para algoritmos como Ant Colony Optimization y Particle Swarm Optimization, y ofrece visualización en tiempo real, ajuste de parámetros y seguimiento del rendimiento. <https://www.swarmwave.app/dashboard>

También puede citarse *Genetic Algorithm Visualization Tool*, una aplicación web centrada en algoritmos genéticos. Esta herramienta permite visualizar problemas como el viajante, la mochila, N-Reinas o aproximación de imágenes, mostrando cómo evoluciona una población de soluciones a lo largo de generaciones. <https://www.geneticalgorithms.online/>

Aun así, este tipo de simuladores suelen estar centrados en algoritmos o problemas concretos. Por ello, aunque son útiles desde el punto de vista educativo, pueden resultar menos flexibles si se desea modificar el algoritmo, cambiar el problema o incorporar visualizaciones personalizadas. En este proyecto se buscaba precisamente una herramienta más adaptable, que permitiera implementar varios algoritmos, diseñar modelos básicos y específicos, y construir interfaces propias.

| Herramienta      | Ventajas                            | Inconvenientes                                        |
|------------------|-------------------------------------|-------------------------------------------------------|
| NetLogo          | Visual, sencilla, basada en agentes | Menos potente que Python/MATLAB para cálculos masivos |
| Python           | Flexible, muchas librerías          | Requiere más conocimientos de programación            |
| MATLAB           | Muy usado en optimización           | Licencia y menor orientación educativa visual         |
| Sim. específicos | Accesibles                          | Menos adaptables                                      |

### 3.4 Justificación de uso de NetLogo

El uso de NetLogo en este proyecto está especialmente relacionado con su valor didáctico. Los algoritmos bioinspirados pueden resultar difíciles de entender si se presentan únicamente mediante ecuaciones o pseudocódigo. En cambio, cuando se representan mediante agentes que se mueven sobre un espacio de búsqueda, muchos conceptos se vuelven más claros.

Por ejemplo, el concepto de población puede visualizarse directamente como un conjunto de agentes. La función objetivo puede representarse como una superficie. La convergencia puede observarse cuando los agentes se agrupan alrededor de una región. Del mismo modo, los óptimos locales pueden

entenderse mejor cuando la población queda atrapada en una zona que parece buena, pero no es la mejor del espacio completo.

Además, la posibilidad de modificar parámetros desde la interfaz facilita la experimentación. El usuario puede cambiar el tamaño de la población, activar o desactivar un paisaje aleatorio, variar el número de iteraciones o modificar parámetros específicos del algoritmo, observando después cómo cambia el comportamiento del modelo.

Por esta razón, NetLogo no se utiliza aquí únicamente como un lenguaje de programación, sino como una herramienta para convertir algoritmos abstractos en procesos visibles y manipulables.

### **3.4 Algoritmos seleccionados**

En este trabajo se han seleccionado tres algoritmos bioinspirados: el algoritmo de ballenas (Whale Optimization Algorithm), el algoritmo del lobo gris (Grey Wolf Optimizer) y el algoritmo del camaleón (Chameleon Swarm Algorithm). Los tres pertenecen al grupo de algoritmos basados en población y comparten la idea de que varias soluciones candidatas evolucionan de forma conjunta dentro de un espacio de búsqueda.

Aun así, cada algoritmo utiliza una inspiración distinta y, por tanto, una forma diferente de actualizar sus soluciones. Esta variedad permite observar distintos comportamientos de búsqueda y comparar cómo cada técnica equilibra la exploración y la explotación.

#### **3.4.1 Algoritmo de optimización de ballenas (WOA)**

##### Inspiración biológica

El WOA es un algoritmo metaheurístico inspirado en el comportamiento de caza de las ballenas jorobadas. En concreto, toma como referencia una estrategia conocida como alimentación mediante red de burbujas. Durante este proceso, las ballenas rodean a sus presas siguiendo trayectorias circulares o en espiral, reduciendo progresivamente el espacio disponible hasta concentrarlas en una zona determinada.

Esta idea se traslada al campo de la optimización de una forma bastante intuitiva. Cada ballena representa una posible solución del problema, y la mejor solución

encontrada hasta el momento se interpreta como la presa. A partir de ahí, el resto de soluciones actualizan sus posiciones intentando aproximarse a esa mejor solución o explorando nuevas regiones del espacio de búsqueda.

La parte interesante del algoritmo está precisamente en ese equilibrio. Si todas las soluciones se acercasen demasiado rápido a la mejor encontrada, el algoritmo podría quedarse atrapado en un óptimo local. En cambio, si todas exploraran sin control, la convergencia sería lenta o incluso inestable. WOA intenta combinar ambos comportamientos mediante distintos mecanismos de actualización.

### Funcionamiento general del algoritmo

El funcionamiento del WOA se basa principalmente en dos mecanismos:

- encerramiento de la presa
- movimiento en espiral alrededor de la presa.

En el primer caso, las ballenas actualizan su posición tomando como referencia la mejor solución conocida. Esta fase representa la explotación del espacio de búsqueda, ya que las soluciones se dirigen hacia una región que parece prometedora.

De forma simplificada, si  $X(t)$  representa la posición actual de una ballena y  $X^*(t)$  la mejor solución encontrada, el algoritmo calcula una nueva posición a partir de la distancia entre ambas. Para ello utiliza coeficientes que introducen aleatoriedad y permiten variar la intensidad del movimiento.

El segundo mecanismo imita el movimiento en espiral de las ballenas jorobadas durante la caza. En este caso, la nueva posición se calcula siguiendo una trayectoria espiral alrededor de la mejor solución. Este comportamiento permite acercarse a la presa de manera progresiva, pero sin hacerlo siempre en línea recta.

En cada iteración, el algoritmo decide aleatoriamente si aplica el movimiento de encerramiento o el movimiento en espiral. Esta decisión permite que las ballenas no sigan todas exactamente el mismo patrón, lo que ayuda a mantener cierta diversidad dentro de la población.

A continuación se muestra un pseudocódigo para este funcionamiento del mismo:

1. Inicializar aleatoriamente la población de ballenas  $X_i$

2. Evaluar el fitness de cada ballena

3. Seleccionar la mejor ballena como  $X^*$

4. Para  $t = 1$  hasta  $T$  hacer:

    Actualizar el parámetro  $a$  (decrece de 2 a 0)

    Para cada ballena  $X_i$  hacer:

        Generar números aleatorios  $r$ ,  $p$  y  $l$

        Calcular:

$$A = 2 \cdot a \cdot r - a$$

$$C = 2 \cdot r$$

        Si  $p < 0.5$  entonces:

            Si  $|A| < 1$  entonces: // Explotación: encerramiento de la presa

                Calcular la distancia entre  $X_i$  y  $X^*$

                Actualizar  $X_i$  acercándola a  $X^*$

            Si  $|A| \geq 1$  entonces: // Exploración: búsqueda en otra zona

                Seleccionar una ballena aleatoria  $X_{rand}$

                Calcular la distancia entre  $X_i$  y  $X_{rand}$

                Actualizar  $X_i$  usando  $X_{rand}$  como referencia

        Si  $p \geq 0.5$  entonces: // Ataque mediante red de burbujas

            Calcular la distancia entre  $X_i$  y  $X^*$

            Actualizar  $X_i$  mediante movimiento en espiral

            Ajustar  $X_i$  a los límites del espacio de búsqueda

    Fin

Evaluar el fitness de todas las ballenas

Si alguna ballena mejora a  $X^*$ :

    Actualizar  $X^*$

5. Devolver  $X^*$

### Exploración y explotación en WOA

Uno de los elementos más importantes del WOA es el parámetro “ $a$ ”, que disminuye de forma lineal a lo largo de las iteraciones. Este parámetro influye sobre el coeficiente “ $A$ ”, utilizado para decidir si una ballena se comporta de forma más exploratoria o más explotadora.

Cuando el valor absoluto de “ $A$ ” es menor que 1, la ballena tiende a acercarse a la mejor solución global. En este caso, el algoritmo favorece la explotación, ya que intenta mejorar soluciones cercanas a la mejor posición encontrada.

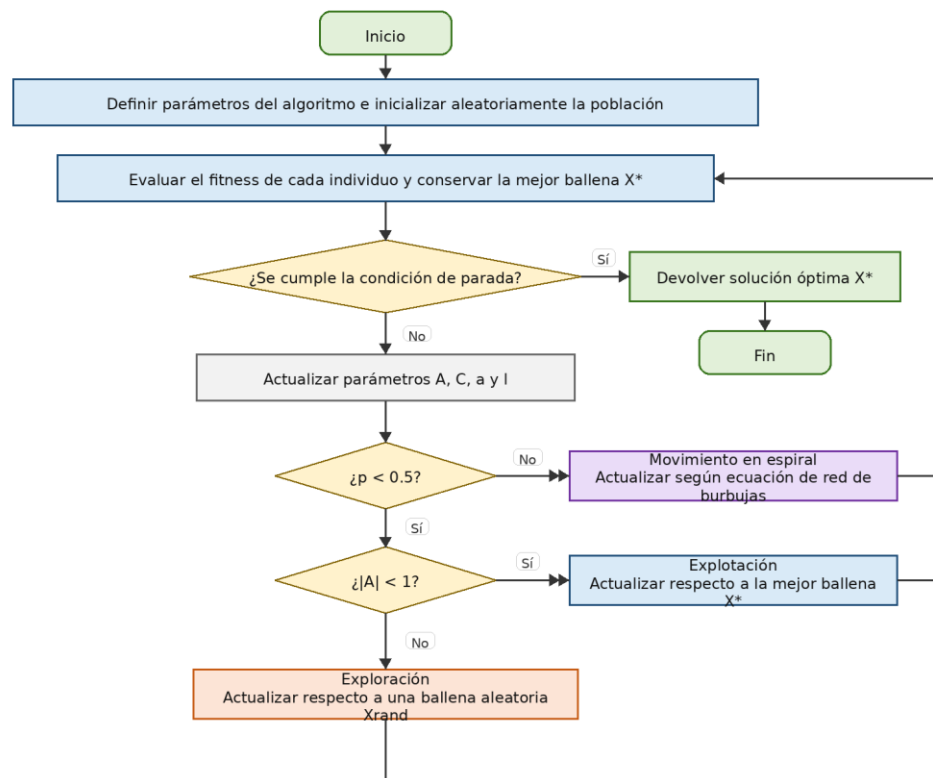
Por el contrario, cuando el valor absoluto de “ $A$ ” es mayor o igual que 1, la ballena puede actualizar su posición tomando como referencia una ballena aleatoria de la población. Este comportamiento favorece la exploración, ya que permite buscar en zonas más alejadas del espacio de soluciones.

De forma resumida, WOA combina tres comportamientos principales:

- aproximación a la mejor solución conocida
- movimiento en espiral alrededor de dicha solución
- exploración mediante individuos aleatorios de la población.

Esta combinación hace que el algoritmo sea relativamente sencillo de interpretar y, al mismo tiempo, adecuado para representarlo visualmente mediante agentes en un entorno como NetLogo.

Para facilitar la comprensión del algoritmo, se presenta un diagrama de flujo que resume las fases principales y su adaptación general al modelo que implementaremos:



### 3.4.2 Algoritmo del lobo gris (GWO)

#### Inspiración biológica y jerarquía social

El GWO es un algoritmo bioinspirado basado en la jerarquía social y el comportamiento de caza de los lobos grises. En la naturaleza, estos animales se organizan en grupos con distintos niveles de liderazgo. El algoritmo toma esta idea y la adapta al proceso de búsqueda de soluciones.

En GWO, la población de soluciones se divide de forma jerárquica. Las tres mejores soluciones encontradas en cada momento reciben nombres específicos:

- alfa ( $\alpha$ ): mejor solución de la población
- beta ( $\beta$ ): segunda mejor solución
- delta ( $\delta$ ): tercera mejor solución
- omega ( $\omega$ ): resto de soluciones.

La idea principal es que los lobos omega actualizan sus posiciones guiándose por los tres líderes: alfa, beta y delta. En lugar de seguir únicamente a la mejor solución, como ocurre en otros algoritmos, GWO utiliza la información de las tres mejores soluciones para orientar la búsqueda.

Este planteamiento resulta bastante útil, porque reduce la dependencia de una única solución. Si la mejor solución actual no se encuentra cerca del óptimo global, la influencia de beta y delta puede aportar diversidad y evitar, en cierta medida, una convergencia demasiado prematura. Jerarquía alfa, beta, delta y omega.

#### Funcionamiento general del algoritmo

El algoritmo GWO simula el proceso de caza de los lobos grises mediante tres fases generales:

- búsqueda de la presa
- rodeo de la presa
- ataque final

En términos de optimización, la presa representa la solución óptima o una región prometedora del espacio de búsqueda. Como esa solución no se conoce de antemano, el algoritmo utiliza las mejores soluciones actuales (alfa, beta y delta) como aproximación.

En cada iteración, cada lobo calcula nuevas posiciones candidatas en función de su distancia respecto a alfa, beta y delta. Después, su nueva posición se obtiene como el promedio de esas tres posiciones estimadas. De esta forma, el movimiento del lobo queda influido por los tres líderes de la población.

Este mecanismo se puede interpretar como una forma cooperativa de búsqueda. La población no se mueve de manera completamente aleatoria, pero tampoco depende de un único individuo. Cada lobo ajusta su posición teniendo en cuenta varias referencias, lo que permite combinar estabilidad y exploración.



Podríamos definir un pseudocódigo para este funcionamiento como el siguiente:

1. Inicializar aleatoriamente la población de lobos.

2. Evaluar el fitness de cada lobo.

3. Identificar las tres mejores soluciones:

$X\alpha$  = mejor solución

$X\beta$  = segunda mejor solución

$X\delta$  = tercera mejor solución

4. Repetir hasta alcanzar el número máximo de iteraciones:

Actualizar el parámetro de control  $a$ .

Para cada lobo de la población:

Calcular la influencia de  $X\alpha$  sobre el lobo.

Calcular la influencia de  $X\beta$  sobre el lobo.

Calcular la influencia de  $X\delta$  sobre el lobo.

Actualizar la posición del lobo combinando la información de los tres líderes.

Ajustar la nueva posición a los límites del problema.

Evaluar de nuevo el fitness de la población.

Actualizar  $X\alpha$ ,  $X\beta$  y  $X\delta$  si aparecen mejores soluciones.

5. Devolver  $X\alpha$ .

### Actualización de posiciones mediante alfa, beta y delta

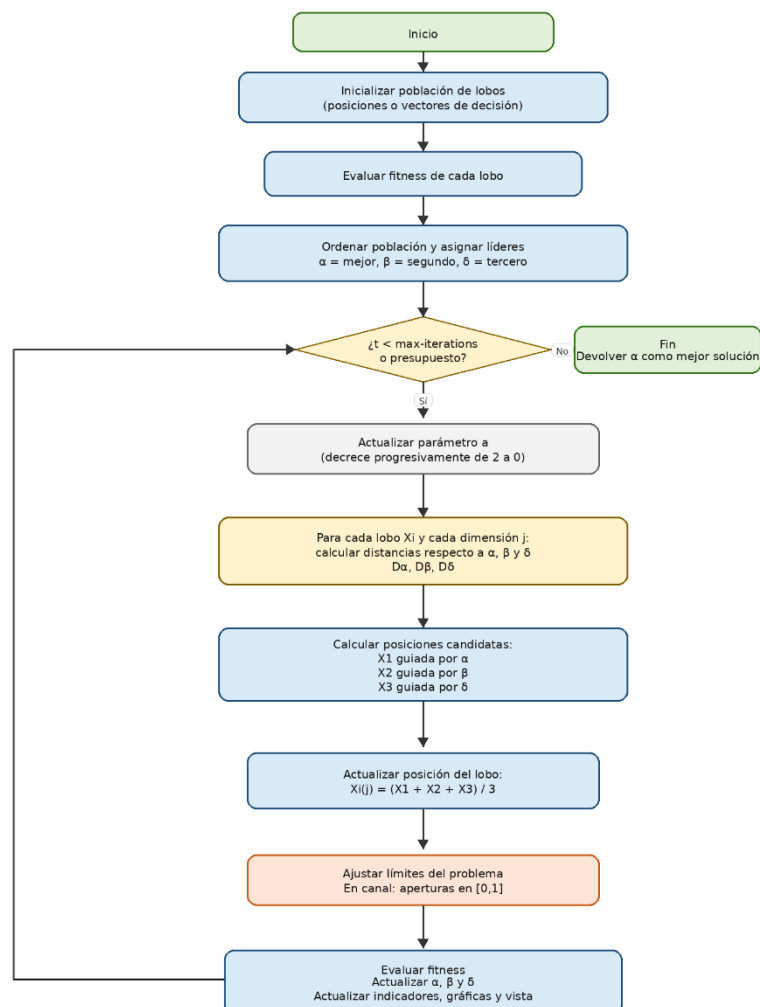
El parámetro “ $a$ ” desempeña un papel importante dentro del GWO. Al igual que en otros algoritmos bioinspirados, se utiliza para controlar el equilibrio entre exploración y explotación. Su valor disminuye progresivamente durante la ejecución, de forma que al principio los lobos tienden a explorar regiones más amplias y, conforme avanzan las iteraciones, se concentran alrededor de las mejores soluciones encontradas.

Durante las primeras iteraciones, los movimientos suelen ser más amplios. Esto

ayuda a recorrer distintas zonas del espacio de búsqueda. Más adelante, el algoritmo reduce la amplitud de los desplazamientos y los lobos comienzan a converger hacia regiones más prometedoras.

En los modelos desarrollados en este trabajo, este comportamiento puede observarse de forma visual. En los modelos básicos, los lobos se desplazan sobre una función o paisaje de búsqueda. En el modelo específico del canal de riego, cada lobo representa una combinación de aperturas de compuertas, y la jerarquía alfa-beta-delta guía la mejora progresiva del reparto de agua.

Para facilitar la comprensión del algoritmo, se presenta un diagrama de flujo que resume las fases principales y su adaptación general al modelo que implementaremos:



### 3.4.3 Algoritmo del camaleón (CSA)

#### Inspiración biológica

El CSA es una metaheurística inspirada en el comportamiento de los camaleones durante la búsqueda y captura de presas. Aunque es menos conocido que otros algoritmos bioinspirados más clásicos, resulta interesante porque divide el comportamiento de búsqueda en varias fases diferenciadas.

El CSA toma como referencia tres capacidades características de los camaleones:

- la percepción del entorno para localizar posibles presas
- la rotación ocular para explorar visualmente distintas zonas
- la proyección rápida de la lengua para capturar la presa.

En el algoritmo, cada camaleón representa una solución candidata. Al igual que ocurre en otros métodos basados en población, cada individuo conserva información sobre su mejor posición personal y el enjambre mantiene también una mejor posición global.

Esta estructura permite que cada solución combine experiencia individual y conocimiento colectivo. Dicho de otra manera, cada camaleón no se mueve únicamente al azar, sino que utiliza tanto lo que ha encontrado él mismo como lo que ha encontrado el conjunto de la población.

#### Fases del algoritmo

El CSA utilizado en este proyecto se puede describir mediante tres fases principales.

La *primera fase* es la de búsqueda o percepción. En ella, cada camaleón genera una nueva posición candidata utilizando información de su mejor posición personal, la mejor posición global y una componente aleatoria de exploración. Esta fase permite recorrer el espacio de búsqueda y evita que todas las soluciones se desplacen exactamente de la misma manera.

La *segunda fase* es la de rotación ocular. En la inspiración biológica, los camaleones pueden mover sus ojos de forma independiente y observar distintas zonas del entorno. En el modelo algorítmico, esta idea se traduce en una modificación adicional de la dirección de búsqueda. En la implementación

desarrollada, esta fase se representa mediante una rotación del desplazamiento en un plano del espacio de diseño. La intención es introducir diversidad y evitar trayectorias demasiado rígidas.

La *tercera fase* es la de lengua o ataque. Esta fase representa un movimiento más dirigido hacia zonas prometedoras. Para ello se utiliza una actualización basada en velocidad, donde intervienen la mejor posición personal, la mejor posición global y factores aleatorios. Si la nueva posición mejora la solución actual, se acepta; si no, se descarta.

Este esquema hace que CSA sea un algoritmo especialmente interesante desde el punto de vista docente, ya que su comportamiento puede explicarse de forma gradual: primero explora, después modifica la dirección de búsqueda y finalmente intenta mejorar mediante un movimiento más dirigido.

Podríamos definir un pseudocódigo para este funcionamiento como el siguiente:

1. Inicializar aleatoriamente la población de camaleones.
2. Inicializar la velocidad de cada camaleón.
3. Evaluar el fitness de cada camaleón.
4. Guardar para cada camaleón su mejor posición personal.
5. Seleccionar la mejor solución inicial como global-best.
6. Repetir hasta alcanzar el número máximo de iteraciones:

Actualizar los parámetros adaptativos del algoritmo.

Para cada camaleón de la población:

Fase 1: búsqueda o percepción

- Generar una nueva posición candidata usando información del mejor personal, del mejor global y una componente aleatoria.
- Aceptar la nueva posición si mejora el fitness.

Fase 2: rotación ocular

- Modificar la dirección de búsqueda mediante una rotación del desplazamiento.
- Aceptar la nueva posición si mejora el fitness.

Fase 3: lengua o ataque

- Actualizar la velocidad del camaleón.
- Generar una nueva posición orientada hacia zonas prometedoras.
- Aceptar la nueva posición si mejora el fitness.

Ajustar la posición final a los límites del problema.

Actualizar los mejores personales de los camaleones.

Actualizar global-best si aparece una solución mejor.

7. Devolver global-best.

### Parámetros principales del CSA

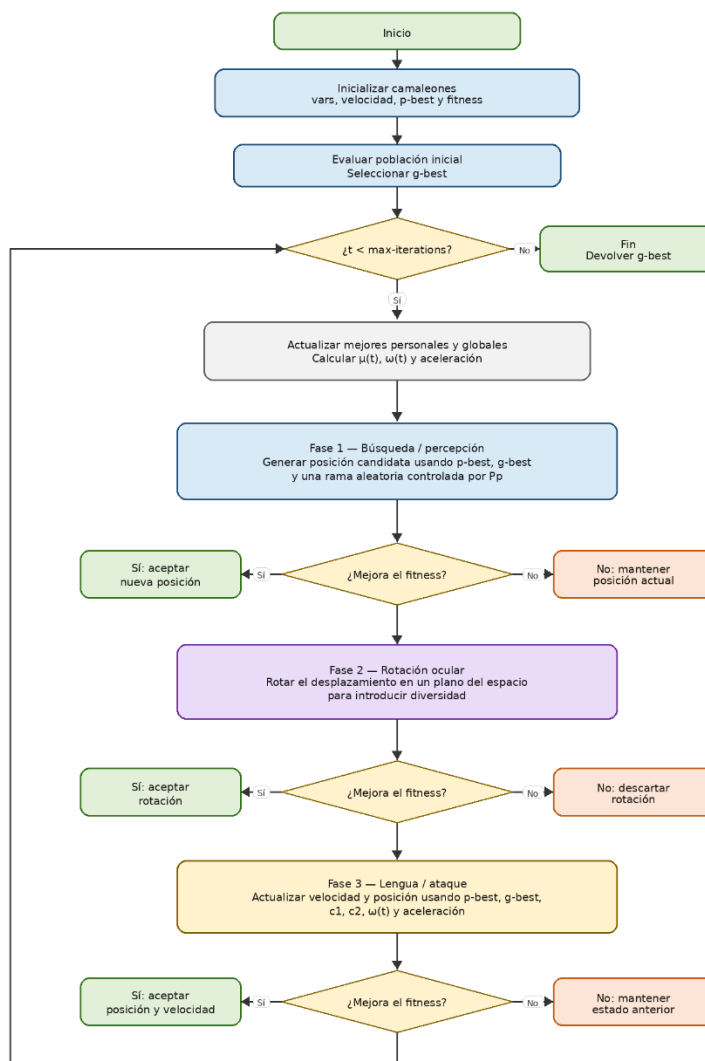
El comportamiento del CSA depende de varios parámetros que controlan la intensidad de sus fases de búsqueda. En los modelos desarrollados, los más relevantes son:

| Parámetro      | Descripción                                                           |
|----------------|-----------------------------------------------------------------------|
| csa-Pp         | Controla la probabilidad asociada a la exploración aleatoria.         |
| csa-p1         | Peso asociado al mejor personal durante la fase de búsqueda.          |
| csa-p2         | Peso asociado al mejor global durante la fase de búsqueda.            |
| csa-alpha      | Controla la forma del decaimiento del parámetro adaptativo $\mu(t)$ . |
| csa-beta       | También participa en el decaimiento de $\mu(t)$ .                     |
| max-iterations | Número máximo de iteraciones del algoritmo.                           |

El parámetro  $\mu(t)$  tiene un papel importante, ya que regula la amplitud de ciertos movimientos de exploración. Al principio de la ejecución, el algoritmo tiende a moverse con mayor libertad. Conforme avanzan las iteraciones, el movimiento se va refinando y la búsqueda se concentra en regiones más prometedoras.

En los modelos básicos, estos parámetros permiten observar cómo cambia la convergencia sobre la función esfera o sobre paisajes aleatorios. Por ejemplo, como se verá en el modelo específico del domo de 120 barras, estos parámetros afectan a la búsqueda de áreas estructurales que reduzcan el peso del domo sin incumplir las restricciones de frecuencia.

Para facilitar la comprensión del algoritmo, se presenta un diagrama de flujo que resume las fases principales y su adaptación general al modelo que implementaremos:



### 3.5 Problema básico de optimización empleado

Antes de aplicar los algoritmos a problemas específicos de ingeniería, se ha hecho de un problema básico para comprobar y visualizar su comportamiento general. Este problema permite observar si el algoritmo converge, cómo se distribuye la población y cómo afectan los parámetros principales a la búsqueda.

En este caso se ha buscado implementar una solución para minimizar la función esfera, que es una función clásica de prueba en optimización continua.

Para que visualmente sea más efectiva, aparte de buscar el óptimo dentro de una misma esfera, se ha recreado el uso de un paisaje aleatoria, así el alumnado podrá seguir mejor la evolución del mismo.

#### 3.5.1. Espacio de búsqueda uniforme en esfera

En dos dimensiones se define como la función esfera se define como:

$$f(x,y) = x^2 + y^2$$

Su mínimo global se encuentra en el punto:

$$(x,y)=(0,0)$$

con valor:

$$f(0,0)=0$$

Se trata de una función sencilla, suave y con un único mínimo global. Precisamente por eso resulta útil como primera prueba. Si un algoritmo funciona correctamente, lo esperable es que la población de soluciones se desplace progresivamente hacia el origen, reduciendo el valor de la función objetivo.

Aunque la función esfera no representa un problema complejo de ingeniería, permite comprobar aspectos importantes:

- si el algoritmo actualiza correctamente las posiciones
- si la población converge hacia una región óptima
- si los parámetros tienen un efecto visible
- si la visualización 3D refleja adecuadamente la evolución de la búsqueda.

Además, al ser una función fácil de representar, encaja bien con el objetivo docente del proyecto. El usuario puede ver directamente cómo los agentes se mueven sobre la superficie y cómo se acercan al mínimo.

### **3.5.2. Espacio de búsqueda en paisajes aleatorios**

Además de buscar en una esfera, los modelos básicos permiten trabajar con paisajes aleatorios. En este caso, el espacio de búsqueda se genera asignando valores de elevación a distintas zonas y suavizando posteriormente la superficie. El resultado es un entorno irregular, con varias regiones atractivas y posibles óptimos locales.

Este tipo de paisaje no tiene una expresión matemática cerrada como la función esfera, pero resulta interesante desde el punto de vista visual. Permite observar cómo se comportan los algoritmos cuando el espacio de búsqueda es menos regular y más impredecible.

En estos paisajes, el objetivo puede plantearse como la búsqueda de la zona de mayor elevación. Esto permite estudiar un comportamiento diferente al de la función esfera: en lugar de minimizar una función suave con un único mínimo, los agentes deben localizar regiones favorables dentro de un entorno con varios máximos locales.

Este escenario ayuda a mostrar algunos problemas habituales en las metaheurísticas, como:

- convergencia prematura
- estancamiento en regiones subóptimas
- importancia del tamaño de población
- influencia del número de iteraciones
- necesidad de mantener diversidad durante la búsqueda.

Por tanto, los paisajes aleatorios complementan a la función esfera y hacen que los modelos básicos sean más ricos desde el punto de vista didáctico.



### 3.6. Problemas específicos de ingeniería

Además de los modelos básicos, este trabajo incluye tres modelos específicos aplicados a problemas de ingeniería. La finalidad de estos modelos es mostrar cómo los algoritmos bioinspirados pueden adaptarse a problemas más cercanos a situaciones reales, donde ya no basta con minimizar una función sencilla, sino que aparecen variables de diseño, restricciones y funciones objetivo más elaboradas.

Los tres problemas específicos desarrollados son:

- optimización estructural de una celosía de 72 barras mediante WOA
- optimización del reparto de agua en un canal de riego mediante GWO
- optimización del domo espacial de 120 barras mediante CSA.

#### 3.6.1. Optimización de la celosía espacial de 72 barras

El primer modelo específico aplica el WOA a la optimización estructural de una celosía espacial de 72 barras. Este problema es un benchmark clásico dentro de la optimización estructural y se utiliza habitualmente para evaluar algoritmos bajo restricciones de tensión y desplazamiento.

La estructura considerada representa una torre espacial tridimensional formada por:

- 20 nodos
- 72 barras
- 4 módulos verticales
- apoyos en los nodos de la base
- dos casos de carga externos.

El objetivo del problema es minimizar el peso total de la estructura. Para ello, el algoritmo debe seleccionar áreas transversales para distintos grupos de barras. En este caso, el diseño está formado por 16 variables:

$$A1, A2, A3, \dots A16$$

Cada variable representa el área de un grupo de barras. En lugar de permitir cualquier valor continuo, las áreas se seleccionan de un conjunto discreto de secciones disponibles. Esto hace que el problema sea más realista, ya que en ingeniería estructural las secciones comerciales suelen pertenecer a catálogos o listas discretas.

La función objetivo puede expresarse de forma general como la minimización del peso:

$$W = \sum_{i=1}^{72} \rho A_i L_i$$

donde “ $\rho$ ” es la densidad del material, “ $A_i$ ” es el área de la barra correspondiente y “ $L_i$ ” su longitud.

Sin embargo, no cualquier diseño ligero es válido. El diseño debe cumplir restricciones estructurales, principalmente:

$$|\sigma_i| \leq \sigma_{\text{adm}}$$

$$|d_j| \leq d_{\text{adm}}$$

donde “ $\sigma_i$ ” representa la tensión axial de cada barra y “ $d_j$ ” los desplazamientos de los nodos controlados.

Para tratar las restricciones se utiliza una función objetivo penalizada. La idea es que, si un diseño incumple los límites de tensión o desplazamiento, su valor de fitness aumenta, aunque su peso sea bajo. De esta forma, el algoritmo no solo busca diseños ligeros, sino también diseños factibles.

En el modelo desarrollado, la estructura se analiza mediante una formulación de celosía espacial 3D. Se calcula la matriz de rigidez global, se resuelven los desplazamientos para los casos de carga y, a partir de ellos, se obtienen las tensiones axiales de las barras. Además, el modelo incluye una comparación con un diseño benchmark de referencia, cuyo peso publicado se sitúa aproximadamente en 389,33 lb.

Este problema resulta especialmente adecuado para el proyecto porque combina un algoritmo bioinspirado con un caso realista de optimización estructural, donde aparecen variables discretas, restricciones y análisis numérico.

### 3.6.2. Optimización del reparto de agua en un canal de riego

El segundo modelo específico aplica el GWO a un problema de reparto y regulación de agua en un canal de riego. El modelo se inspira en trabajos relacionados con la planificación del riego mediante GWO, pero se implementa en NetLogo mediante una simulación simplificada.

El problema consiste en encontrar una configuración adecuada de aperturas de compuertas que mejore el reparto de agua entre varios puntos de entrega del canal. En el modelo se consideran:

- 6 turnouts o puntos de entrega de agua
- 3 check gates o compuertas de control intermedias.

Cada solución candidata, es decir, cada lobo, se representa mediante un vector de 9 componentes:

$$[TO_1, TO_2, TO_3, TO_4, TO_5, TO_6, CH_1, CH_2, CH_3]$$

Los valores de este vector se encuentran en el intervalo  $[0,1]$ , donde cada componente representa el grado de apertura de una compuerta.

El modelo simula de forma simplificada el movimiento del agua desde aguas arriba hacia aguas abajo. Las compuertas de control influyen en el reparto por tramos y los turnouts determinan la cantidad de agua que puede entregarse a cada punto. Además, se considera una evolución temporal simplificada para poder evaluar la estabilidad del reparto.

El objetivo no se reduce únicamente a entregar la mayor cantidad de agua posible, sino a conseguir un reparto equilibrado y eficiente. Para ello se utilizan varios indicadores:

- PA, adecuación: mide en qué grado se satisface la demanda
- PF, eficiencia: penaliza la sobreentrega
- PE, equidad: mide la desigualdad espacial del reparto
- PD, fiabilidad: mide la variabilidad temporal del servicio.

La función de fitness que se minimiza se define como:

$$FF = (1 - PA) + (1 - PF) + PE + PD$$

Cuanto menor sea el valor de esta función, mejor será la solución encontrada. Una buena solución debería satisfacer la demanda en la medida de lo posible, evitar entregas excesivas, repartir el agua de forma equilibrada y mantener cierta estabilidad temporal.

Es importante señalar que este modelo no pretende sustituir a un simulador hidráulico profesional. En lugar de utilizar un simulador completo como ICSS, se ha implementado una aproximación simplificada en NetLogo. El objetivo es representar la lógica general del problema y permitir estudiar cómo el algoritmo GWO busca configuraciones de compuertas en un entorno visual e interactivo. Este problema aporta variedad al proyecto, ya que introduce un caso de optimización diferente a los estructurales. En lugar de minimizar peso bajo restricciones mecánicas, aquí se optimiza una distribución de recursos limitada, algo muy común en problemas reales de planificación y control.

### **3.6.3. Optimización del domo espacial de 120 barras**

El tercer modelo específico aplica el CSA al problema del domo espacial de 120 barras. Este benchmark pertenece al ámbito de la optimización estructural y resulta especialmente interesante porque incorpora restricciones dinámicas basadas en frecuencias naturales.

El modelo representa un domo tridimensional formado por:

- 49 nodos
- 120 barras
- 7 grupos de diseño
- apoyos en el anillo exterior
- masas nodales no estructurales en nodos seleccionados.

Cada diseño se define mediante 7 variables:

$$A1, A2, A3, \dots, A7$$

Cada una de estas variables representa el área de un grupo de barras del domo.

A partir de estas áreas se calcula el peso total de la estructura.

El objetivo principal es reducir el peso del domo, pero respetando restricciones dinámicas sobre sus dos primeras frecuencias naturales:

$$f_1 \geq 9 \text{ Hz}$$

$$f_2 \geq 11 \text{ Hz}$$

Estas restricciones son importantes porque un diseño demasiado ligero puede presentar frecuencias naturales bajas y, por tanto, un comportamiento dinámico no deseado. En otras palabras, no basta con hacer la estructura lo más ligera posible: también debe conservar una rigidez suficiente.

De forma general, el fitness del modelo puede interpretarse como:

$$\text{fitness} = \text{peso} + \text{penalización}$$

La penalización aparece cuando alguna de las frecuencias calculadas no alcanza el valor mínimo requerido. Si el diseño cumple las dos restricciones, la penalización es cero y el fitness coincide con el peso real.

Para evaluar los diseños se construyen matrices de rigidez y masa de la estructura. A partir de ellas se obtiene una aproximación de las primeras frecuencias naturales. Este cálculo puede resultar costoso, especialmente si se realiza muchas veces durante el proceso de optimización. Por este motivo, el modelo incorpora distintos niveles de evaluación:

- RÁPIDO, pensado para observar el comportamiento general del algoritmo de forma ágil
- MIXTO, que utiliza una aproximación intermedia entre rapidez y realismo
- PRECISO, que emplea un cálculo modal más completo y resulta más adecuado para validar resultados finales.

Este planteamiento permite trabajar con el modelo de forma interactiva sin que cada ejecución sea excesivamente lenta, pero manteniendo la posibilidad de realizar evaluaciones más rigurosas cuando sea necesario.

Desde el punto de vista visual, el domo permite representar de forma clara cómo cambian las áreas de los grupos de barras. Las barras pueden mostrarse con distintos colores o grosores en función del área asignada, lo que ayuda a interpretar qué zonas de la estructura está reforzando el algoritmo durante la búsqueda.

## **CAPITULO 4**

### **ANALISIS Y DISEÑO**

#### **4.1 Introducción**

Una vez descritos los fundamentos teóricos de los algoritmos y de los problemas de optimización utilizados, en este capítulo se presenta el análisis y diseño de los modelos desarrollados en NetLogo.

Como el proyecto se ha planteado siguiendo una metodología ágil basada en Scrum, el diseño no se ha entendido como una fase cerrada al inicio del trabajo, sino como un proceso incremental. En cada sprint se ha desarrollado una parte funcional del sistema: primero los modelos básicos de los algoritmos, después los modelos específicos y finalmente las pruebas, comparativas y documentación.

En este capítulo se explica cómo se ha organizado el backlog del producto, qué historias de usuario se han considerado y qué criterios de aceptación se han utilizado para validar los modelos. También se describe la estructura común de los modelos, la representación de las soluciones candidatas y las principales decisiones de diseño tomadas antes y durante la implementación.

Se diferencian dos tipos de modelos. Por un lado, los modelos básicos, orientados a mostrar el comportamiento general de los algoritmos WOA, GWO y CSA sobre espacios de búsqueda sencillos. Por otro lado, los modelos específicos, donde cada algoritmo se aplica a un problema de ingeniería.

Esta separación resulta importante porque cada tipo de modelo tiene una finalidad distinta. Los modelos básicos buscan ser visuales, intuitivos y fáciles de manipular. En cambio, los modelos específicos deben representar problemas más complejos, con variables de diseño, restricciones, funciones objetivo y cálculos internos más elaborados.

## 4.2 Backlog del producto e historias de usuario

En una metodología ágil, el análisis del sistema no se plantea únicamente como una lista cerrada de requisitos iniciales. En su lugar, el trabajo se organiza mediante un backlog del producto, formado por funcionalidades que aportan valor al usuario final.

En este proyecto, el usuario objetivo es principalmente alumnado o profesorado que quiera utilizar los modelos para comprender el funcionamiento de algoritmos bioinspirados. Por este motivo, las funcionalidades se han expresado como historias de usuario, siguiendo la estructura:

*“Como usuario, quiero realizar una acción, para conseguir un objetivo”.*

Estas historias han servido para guiar el desarrollo de los modelos durante los distintos sprints.

#### 4.2.1 Historias de usuario principales

| ID    | Historia de usuario                                                                                                                                  | Relación con tareas |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| HU-01 | Como usuario, quiero ejecutar un modelo básico de WOA, para observar cómo se comporta el algoritmo de ballenas en un espacio de búsqueda sencillo.   | T1                  |
| HU-02 | Como usuario, quiero ejecutar un modelo básico de GWO, para observar cómo la jerarquía alfa-beta-delta guía la búsqueda.                             | T2                  |
| HU-03 | Como usuario, quiero ejecutar un modelo básico de CSA, para visualizar sus fases de búsqueda, rotación ocular y ataque.                              | T3                  |
| HU-04 | Como usuario, quiero modificar parámetros como población o iteraciones, para comprobar cómo afectan al comportamiento del algoritmo.                 | T1, T2, T3          |
| HU-05 | Como usuario, quiero ejecutar los modelos paso a paso o de forma continua, para estudiar la evolución de la población con distinto nivel de detalle. | T1, T2, T3          |
| HU-06 | Como usuario, quiero ver gráficas de fitness, para analizar si el algoritmo mejora durante la ejecución.                                             | T1, T2, T3          |
| HU-07 | Como usuario, quiero aplicar WOA a una celosía de 72 barras, para estudiar un caso estructural con restricciones de tensión y desplazamiento.        | T4                  |
| HU-08 | Como usuario, quiero aplicar GWO a un canal de riego, para analizar el reparto de agua mediante aperturas de compuertas.                             | T5                  |
| HU-09 | Como usuario, quiero aplicar CSA a un domo de 120 barras, para estudiar un problema estructural con restricciones de frecuencia.                     | T6                  |
| HU-10 | Como usuario, quiero comparar los resultados obtenidos con los artículos de referencia, para validar el comportamiento de los modelos específicos.   | T7                  |
| HU-11 | Como usuario, quiero disponer de un manual de uso, para poder utilizar la herramienta sin conocer el código interno.                                 | T8                  |



Estas historias de usuario no se implementaron todas de golpe. Se fueron abordando de forma incremental a lo largo de los sprints. Primero se trabajaron las historias asociadas a los modelos básicos y, una vez validadas, se pasó a los modelos específicos.

#### 4.2.2 Criterios de aceptación

Para validar cada historia de usuario se definieron criterios de aceptación. Estos criterios permiten comprobar si la funcionalidad desarrollada cumple lo esperado al final del sprint.

| Historia            | Criterios de aceptación                                                                                      |
|---------------------|--------------------------------------------------------------------------------------------------------------|
| HU-01, HU-02, HU-03 | El modelo se inicializa correctamente, crea la población, evalúa fitness y actualiza posiciones sin errores. |
| HU-04               | Los parámetros modificados desde la interfaz afectan realmente a la ejecución del modelo.                    |
| HU-05               | El modelo puede ejecutarse paso a paso y de forma continua.                                                  |
| HU-06               | La gráfica de fitness se actualiza durante la simulación y permite observar la convergencia.                 |
| HU-07               | El modelo de 72 barras calcula peso, tensiones, desplazamientos, violaciones y factibilidad.                 |
| HU-08               | El modelo del canal calcula entregas medias e indicadores PA, PF, PE y PD.                                   |
| HU-09               | El modelo del domo calcula peso, frecuencias, márgenes y violación total.                                    |
| HU-10               | Se obtienen resultados comparables con los valores publicados en los artículos de referencia.                |
| HU-11               | El manual explica cómo ejecutar cada modelo, qué parámetros modificar y cómo interpretar los resultados.     |

### **4.2.3 Requisitos de calidad del producto**

Además de las historias de usuario, se tuvieron en cuenta varios criterios de calidad del producto. Estos criterios no corresponden a una funcionalidad concreta, sino a características generales que debían respetarse:

- Claridad visual: los modelos deben ser comprensibles para un usuario que quiera observar el comportamiento del algoritmo.
- Facilidad de uso: la interfaz debe permitir ejecutar los modelos sin necesidad de modificar directamente el código.
- Orientación docente: los modelos deben servir como apoyo para entender conceptos como población, fitness, exploración, explotación y convergencia.
- Modularidad: cada modelo debe separar, en la medida de lo posible, la lógica del algoritmo, la evaluación del problema y la visualización.
- Reproducibilidad: los experimentos deben poder repetirse utilizando parámetros y semillas concretas.
- Rendimiento razonable: los modelos deben poder ejecutarse en un ordenador personal, aunque algunos problemas específicos sean más costosos.
- Transparencia: en los modelos simplificados, (como en el del canal de riego o los distintos modos del domo de 120 barras), debe quedar claro que se trata de una aproximación académica y no de una simulación profesional exacta y completa.

### **4.3 Diseño incremental de los modelos**

El diseño de los modelos se realizó de forma incremental, siguiendo la planificación por sprints. En lugar de desarrollar primero todo el sistema completo, se fue construyendo el proyecto por bloques funcionales.

En los primeros sprints se diseñaron los modelos básicos de WOA, GWO y CSA. Estos modelos permitieron validar la lógica general de cada algoritmo sobre espacios de búsqueda sencillos. Una vez comprobado su funcionamiento, se avanzó hacia los modelos específicos, donde cada algoritmo se adaptó a un problema de ingeniería.

A pesar de las diferencias entre modelos, todos comparten una estructura común:

- Inicialización del entorno.
- Generación de la población inicial.
- Evaluación de cada solución candidata.
- Identificación de las mejores soluciones.
- Actualización de la población mediante el algoritmo correspondiente.
- Actualización de monitores, gráficas y visualización.
- Repetición hasta alcanzar el criterio de parada.

Esta estructura común facilitó el desarrollo incremental, ya que cada nuevo modelo aprovechaba ideas ya probadas en los sprints anteriores.

#### **4.3.1. Incremento 1: modelos básicos**

El primer incremento del producto estuvo formado por los modelos básicos de los tres algoritmos. Estos modelos se desarrollaron en los sprints 2, 3 y 4, una vez completada la fase inicial de estudio teórico.

El objetivo de este incremento era disponer de simuladores visuales sencillos que permitieran comprobar el funcionamiento general de WOA, GWO y CSA antes de aplicarlos a problemas de ingeniería. Los modelos básicos están pensados para mostrar el comportamiento de cada algoritmo sobre un espacio de búsqueda visual. Para ello se utiliza principalmente la función esfera  $f(x,y) = x^2 + y^2$

También se incluye la opción de generar paisajes aleatorios, que permiten observar el comportamiento del algoritmo en entornos menos regulares y con posibles óptimos locales.

En estos modelos, cada solución candidata se representa mediante una posición en el plano (x,y)

La función objetivo proporciona el valor de fitness asociado a esa posición. Este

valor se representa visualmente mediante la altura de la superficie en la vista 3D. El objetivo de los modelos básicos no es resolver un problema de ingeniería complejo, sino facilitar la comprensión del funcionamiento de los algoritmos. Por eso se prioriza la claridad visual, la interacción con parámetros y la posibilidad de observar trayectorias y convergencia.

#### **4.3.2. Incremento 2: modelos específicos**

El segundo incremento del producto consistió en adaptar cada algoritmo a un problema específico de ingeniería. Este trabajo se realizó en los sprints 5, 6 y 7. A diferencia de los modelos básicos, en los modelos específicos una solución candidata ya no es simplemente un punto  $(x,y)$ , sino un vector de variables asociado a un diseño o configuración concreta. En la celosía de 72 barras, el vector contiene 16 áreas de grupos de barras.

- En el canal de riego, el vector contiene 9 aperturas de compuertas (6-3).
- En el domo de 120 barras, el vector contiene 7 áreas de grupos de barras.

La evaluación de estos vectores requiere cálculos adicionales. En el caso estructural, se calculan pesos, desplazamientos, tensiones o frecuencias. En el caso del canal de riego, se simula un reparto simplificado de agua y se calculan indicadores de rendimiento.

Por tanto, el diseño de los modelos específicos se ha planteado con tres partes diferenciadas:

- Representación del diseño.
- Evaluación del diseño mediante una función objetivo.
- Aplicación del algoritmo de optimización para mejorar las soluciones.

#### **4.3.3. Representación de soluciones candidatas**

La representación de las soluciones se fue refinando en cada sprint, según el problema abordado. En los modelos básicos se utilizó una representación sencilla, mientras que en los modelos específicos se emplearon vectores de variables más complejos.

En los modelos básicos, una solución candidata se representa como:

$$[x,y]$$

En el modelo estructural de 72 barras, una solución se representa como:

$$[A1,A2,...,A16]$$

donde cada variable corresponde al área transversal de un grupo de barras. El problema de 72 barras divide los elementos en 16 grupos por simetría, con restricciones de tensión y desplazamiento, y utiliza secciones discretas disponibles para las áreas de diseño.

En el modelo del canal de riego, una solución se representa como:

$$[TO1,TO2,TO3,TO4,TO5,TO6,CH1,CH2,CH3]$$

donde los seis primeros valores corresponden a aperturas de puntos de entrega y los tres últimos a compuertas de control. El artículo de referencia modela el canal E1R1 con seis turnouts y tres check gates, y plantea el ajuste de sus aperturas como variables de decisión del problema.

En el modelo del domo de 120 barras, una solución se representa como:

$$[A1,A2,...,A7]$$

donde cada variable representa el área de un grupo de barras. El benchmark del domo agrupa las 120 barras en siete grupos por simetría, con áreas entre 1 cm<sup>2</sup> y 129,3 cm<sup>2</sup>, e impone restricciones de frecuencia  $f_1 \geq 9\text{Hz}$  y  $f_2 \geq 11\text{Hz}$ .

#### **4.4 Diseño de los modelos básicos**

Los modelos básicos fueron desarrollados como primeros incrementos funcionales del proyecto. Cada uno permitió validar una parte del backlog asociada al algoritmo correspondiente.

##### **4.4.1. Modelo básico WOA**

El modelo básico del algoritmo WOA se diseñó para representar visualmente el comportamiento de las ballenas durante el proceso de optimización. Cada agente representa una ballena, es decir, una solución candidata dentro del espacio de búsqueda.

El diseño del modelo incluye dos modos principales:

- búsqueda sobre esfera
- búsqueda sobre paisaje aleatorio.

En la esfera, el objetivo es minimizar el valor de  $f(x,y)$ , por lo que las ballenas deben desplazarse hacia el origen. En el paisaje aleatorio, el objetivo se adapta para buscar zonas de mayor elevación, lo que permite observar el comportamiento del algoritmo en un entorno más irregular.

El modelo debía reflejar los mecanismos principales del WOA:

- aproximación a la mejor solución global
- exploración mediante una ballena aleatoria
- movimiento en espiral alrededor de la mejor solución.

El diseño de este modelo se apoya en la formulación original del WOA, donde el algoritmo alterna entre el encierro de la presa, el movimiento en espiral y la búsqueda exploratoria mediante agentes aleatorios.

#### **4.4.2. Modelo básico GWO**

El modelo básico del GWO se diseñó en torno a la jerarquía propia del algoritmo. En este caso, cada agente representa un lobo, y las mejores soluciones de la población se clasifican como alfa, beta y delta.

El diseño del modelo debía permitir observar cómo el resto de lobos actualizan sus posiciones a partir de la influencia de estos tres líderes. Esta decisión encaja con el funcionamiento del algoritmo, donde los tres mejores individuos estiman la posición de la presa y guían el movimiento de la población.

Al igual que en el modelo WOA, se incluyeron dos tipos de espacio de búsqueda:

- esfera
- paisaje aleatorio.

El objetivo era que el usuario pudiera ver con claridad cómo los lobos se distribuyen inicialmente por el espacio, cómo se identifican los mejores individuos y cómo la población converge progresivamente hacia zonas prometedoras.

#### **4.4.3. Modelo básico CSA**

El modelo básico del CSA se diseñó para representar las tres fases principales del algoritmo:

- búsqueda o percepción
- rotación ocular 360 grados
- lanzamiento de lengua o ataque.

Cada agente representa un camaleón, con una posición actual, una velocidad, un mejor personal y una referencia al mejor global. Esta estructura se basa en el planteamiento original del CSA, que modela la búsqueda de presa, la rotación de ojos y el ataque mediante lengua como mecanismos de actualización de posiciones.

El modelo debía permitir observar cómo los camaleones exploran el espacio, modifican sus trayectorias y aceptan nuevas posiciones cuando mejoran la solución actual.

En este caso, se prestó especial atención a que los parámetros propios del CSA fueran configurables desde la interfaz, ya que afectan directamente al equilibrio entre exploración y explotación.

#### **4.5 Diseño de los modelos específicos**

Una vez validados los modelos básicos, se abordaron los modelos específicos. Cada uno de estos modelos se desarrolló como un incremento

##### **4.5.1. Modelo WOA para la celosía espacial de 72 barras**

El modelo específico basado en WOA se diseñó para resolver el problema de optimización estructural de la celosía espacial de 72 barras. El objetivo es minimizar el peso total de la estructura seleccionando áreas transversales para grupos de barras, respetando restricciones de tensión y desplazamiento.

La estructura del benchmark tiene 20 nodos, 72 barras y dos condiciones de carga independientes; las barras se agrupan en 16 grupos debido a la simetría estructural, y las secciones se seleccionan de una lista discreta de 64 valores posibles.

Desde el punto de vista del diseño del modelo, se decidió que cada ballena representara un vector de 16 variables  $[A1, A2, \dots, A16]$ .

Cada variable se asocia a un grupo de barras. Una vez generado un diseño, el modelo lo expande internamente para asignar áreas a las 72 barras reales.

La función de evaluación del modelo debe calcular:

- peso total de la estructura
- desplazamientos nodales
- tensiones axiales
- violaciones de restricciones
- fitness penalizado.

El problema original impone que las tensiones máximas no superen  $\pm 25$  ksi y que los desplazamientos de los nodos superiores no superen  $\pm 0,25$  pulgadas. Para manejar estas restricciones, el diseño del modelo incorpora una función de penalización. De esta manera, los diseños ligeros pero no factibles reciben un fitness peor que diseños algo más pesados pero válidos.

La visualización del modelo se planteó para mostrar la estructura tridimensional y colorear las barras según su nivel de tensión. Esto permite que el usuario no solo vea el resultado numérico, sino también qué zonas de la estructura están más solicitadas.

#### **4.5.2. Modelo GWO para el canal de riego**

El modelo específico basado en GWO se diseñó tomando como referencia el problema de programación y reparto de agua en canales de riego. El artículo de referencia aplica GWO al canal E1R1 de la red Dez, utilizando seis turnouts y tres check gates como estructuras regulables.

En el modelo desarrollado, cada lobo representa una configuración de aperturas:

$[TO1, TO2, TO3, TO4, TO5, TO6, CH1, CH2, CH3]$



Cada valor está acotado entre 0 y 1. Los turnouts representan puntos de entrega de agua y los check gates representan compuertas de control intermedias.

El artículo original utiliza ICSS para simular el comportamiento hidráulico del canal y calcular caudales y profundidades de agua, que después se emplean en la función de fitness. En este proyecto, debido a la complejidad de integrar un simulador hidráulico completo en NetLogo, se diseñó una versión simplificada del canal. Esta versión conserva la lógica general del problema, pero no pretende reproducir con exactitud el comportamiento hidráulico real.

La función de fitness utilizada se basa en los cuatro indicadores clásicos de rendimiento del reparto de agua:

$$FF = (1 - PA) + (1 - PF) + PE + PD$$

El artículo de referencia define estos indicadores como adecuación, eficiencia, equidad y fiabilidad, y los combina en una función lineal que se minimiza.

En el diseño del modelo se incluyeron distintos patrones de demanda, como demanda uniforme, aleatoria, mayor demanda aguas arriba o mayor demanda aguas abajo. Esto permite probar cómo cambia el comportamiento del algoritmo en distintos escenarios.

La interfaz visual se diseñó como un esquema simplificado del canal, mostrando:

- el canal principal
- las compuertas de control
- los puntos de entrega
- las mejores aperturas encontradas
- las entregas medias obtenidas.

#### **4.5.3. Modelo CSA para el domo espacial de 120 barras**

El modelo específico basado en CSA se diseñó para resolver el problema del domo espacial de 120 barras. Este benchmark se caracteriza por tener restricciones dinámicas, lo que lo diferencia del modelo de la celosía de 72 barras.

El problema del domo agrupa sus 120 barras en siete grupos por simetría. El material tiene densidad 7971,810 kg/m<sup>3</sup> y módulo de elasticidad 210 GPa. Además, se consideran masas no estructurales en nodos libres: 3000 kg en el

nodo superior, 500 kg en determinados nodos del anillo superior y 100 kg en el resto de nodos libres.

Cada camaleón representa un diseño formado por siete áreas  $[A_1, A_2, \dots, A_7]$

El objetivo es minimizar el peso real del domo, cumpliendo las restricciones de frecuencias  $f_1 \geq 9$  Hz y  $f_2 \geq 11$  Hz.

Estas restricciones se corresponden con las indicadas en el benchmark del domo de 120 barras.

Desde el punto de vista de diseño, este modelo presenta una dificultad añadida: el cálculo de frecuencias naturales es más costoso que la evaluación de una función matemática simple. Por ello, se plantearon distintos modos de evaluación:

- RÁPIDO, para observar el comportamiento del algoritmo con bajo coste computacional.
- MIXTO, para utilizar una aproximación más representativa sin llegar al coste completo del cálculo preciso.
- PRECISO, para validar resultados mediante un cálculo modal más completo.

Esta decisión se tomó para mantener el modelo utilizable en NetLogo, ya que una evaluación precisa en cada iteración puede hacer que la simulación sea demasiado lenta.

La visualización del domo se diseñó para mostrar los grupos de barras mediante colores y grosores asociados al área asignada. Así, el usuario puede observar qué zonas del domo son reforzadas por el algoritmo y cómo evoluciona el mejor diseño encontrado.

#### **4.6 Diseño de la interfaz**

La interfaz también se fue ajustando de forma iterativa. A medida que se probaban los modelos, se añadieron o reorganizaron controles, monitores y gráficas para facilitar el uso por parte del usuario.

#### **4.6.1. Controles de ejecución**

Todos los modelos incluyen controles básicos de ejecución:

- botón de inicialización
- botón de ejecución continua
- botón de ejecución paso a paso.

El botón de inicialización prepara el entorno, genera la población inicial y calcula los primeros valores de fitness. La ejecución continua permite observar la evolución completa del algoritmo, mientras que el modo paso a paso resulta útil para estudiar el comportamiento iteración por iteración.

#### **4.6.2. Parámetros configurables**

Cada modelo incorpora parámetros propios del algoritmo y del problema tratado.

En los modelos básicos, los parámetros más importantes son:

- tamaño de población
- número de iteraciones o presupuesto de evaluación
- activación del espacio aleatorio
- suavizado del paisaje
- número de colinas
- modo de trazado.

En los modelos específicos, se añaden parámetros del problema:

- límites de tensión, desplazamiento, densidad y módulo de Young en la celosía de 72 barras
- caudal disponible, demanda base, patrón de demanda y heterogeneidad del canal en el modelo de riego
- factores de rigidez, densidad, masa no estructural y frecuencia objetivo en el domo de 120 barras.

#### **4.6.3. Monitores y gráficas**

Los monitores muestran información resumida del estado del algoritmo.

Dependiendo del modelo, pueden incluir:

- iteración actual
- mejor fitness
- mejor vector de variables
- peso real
- factibilidad
- indicadores de reparto
- frecuencias naturales
- violación total de restricciones.

Las gráficas permiten observar la evolución del algoritmo. En los modelos básicos se representa principalmente la evolución del fitness. En el canal de riego se incluyen además indicadores como PA, PE y PD. En los modelos estructurales se representa la convergencia del mejor diseño y, cuando procede, el valor medio de la población.

#### **4.6.4. Visualización 3D**

La visualización 3D es uno de los elementos más importantes del proyecto.

En los modelos básicos, la vista 3D representa el espacio de búsqueda y la posición de los agentes sobre la superficie.

En el modelo de 72 barras, la vista 3D muestra la estructura espacial y permite observar el estado de las barras mediante colores relacionados con la tensión.

En el modelo del canal, la visualización representa un esquema del canal, las compuertas y los puntos de entrega, mostrando las aperturas y entregas asociadas a la mejor solución.

En el modelo del domo, la vista 3D muestra la geometría del domo de 120 barras. Los colores y grosores de las barras ayudan a interpretar las áreas asignadas a cada grupo de diseño.

## **CAPITULO 5 IMPLEMENTACION**

### **5.1. Introducción**

En este capítulo se describe la implementación de los modelos desarrollados en NetLogo. Después de haber definido los algoritmos, los problemas de optimización y el diseño general de los modelos, toca explicar cómo se ha llevado todo eso a código.

El desarrollo se ha dividido en dos grandes bloques. Por una parte, se han implementado tres modelos básicos, uno para cada algoritmo: WOA, GWO y CSA. Estos modelos están pensados para visualizar el comportamiento general de los algoritmos sobre espacios de búsqueda sencillos, como la función esfera o paisajes aleatorios.

Por otra parte, se han desarrollado tres modelos específicos, donde cada algoritmo se aplica a un problema de ingeniería diferente: la celosía espacial de 72 barras, el canal de riego y el domo espacial de 120 barras.

La implementación no ha consistido únicamente en traducir fórmulas a código. Además, también se incluye una propuesta de actividades docentes derivadas de los modelos desarrollados, pensadas para que el alumnado pueda utilizar la herramienta de forma guiada, modificando parámetros, comparando resultados y relacionando el comportamiento de los agentes con los conceptos del algoritmo.

### **5.2. Herramienta utilizada: NetLogo 3D**

Todos los modelos se han implementado en NetLogo 3D. La elección de esta herramienta viene motivada por su orientación al modelado basado en agentes, lo que encaja bastante bien con los algoritmos bioinspirados utilizados.

En estos algoritmos se trabaja con poblaciones de soluciones candidatas: ballenas, lobos o camaleones. En NetLogo, este tipo de comportamiento puede representarse de forma natural mediante agentes que se desplazan por un entorno y modifican su estado en cada iteración.

Además, NetLogo permite construir interfaces interactivas con botones, deslizadores, interruptores, monitores y gráficas. Esto ha sido importante porque uno de los objetivos del proyecto no era solo obtener resultados numéricos, sino

también poder observar el proceso de optimización de forma visual.

En los modelos básicos se ha usado la vista 3D para representar superficies de búsqueda y agentes desplazándose sobre ellas. En los modelos específicos se ha usado la visualización para representar estructuras o esquemas del problema: la torre de 72 barras, el canal de riego y el domo de 120 barras.

### **5.3. Estructura común de implementación**

Aunque cada algoritmo y cada problema tienen sus propias particularidades, todos los modelos comparten una estructura parecida. Esta estructura común ha ayudado a mantener cierta coherencia entre los modelos y a no desarrollar cada uno como si fuese completamente independiente.

De forma general, cada modelo se organiza alrededor de dos procedimientos principales:

- *setup*, encargado de inicializar el modelo
- *go*, encargado de ejecutar una iteración del algoritmo.

Esta estructura es muy habitual en NetLogo y resulta cómoda para separar la preparación inicial del proceso iterativo.

#### **5.3.1. Inicialización de los modelos**

El procedimiento *setup* se encarga de preparar todo lo necesario antes de comenzar la simulación. Dependiendo del modelo, esto puede incluir:

- limpiar el entorno anterior
- generar el espacio de búsqueda
- crear la población inicial
- calcular el fitness inicial
- identificar la mejor solución inicial
- preparar la visualización
- reiniciar el contador de iteraciones.

En los modelos básicos, esta inicialización suele ser más sencilla. Se genera la superficie de búsqueda, se colocan los agentes de forma aleatoria y se calcula el valor inicial de la función objetivo.

En los modelos específicos, la inicialización es más pesada. Por ejemplo, en el modelo de la celosía de 72 barras se construye la geometría estructural, la conectividad, los apoyos, las cargas y las longitudes de las barras. En el modelo del domo de 120 barras también se generan nodos, barras, grupos de diseño, masas no estructurales y grados de libertad.

### **5.3.2. Representación de agentes y soluciones**

En NetLogo, los agentes se han representado mediante breed. En los modelos básicos, las soluciones candidatas se representan como partículas que se mueven sobre un espacio de búsqueda. Aunque visualmente puedan aparecer como puntos rojos, conceptualmente cada partícula representa una solución del problema.

En los modelos básicos WOA y GWO, una solución se corresponde con una posición en el plano:

$[x,y]$

En el modelo básico CSA, también se trabaja con una representación vectorial de la posición:

$vars=[x,y]$

En los modelos específicos, la representación cambia bastante:

- En la celosía de 72 barras, cada partícula representa un diseño estructural de 16 variables.
- En el canal de riego, cada lobo representa un vector de 9 aperturas de compuertas.
- En el domo de 120 barras, cada camaleón representa un diseño de 7 áreas de grupos de barras.

Esto hace que, aunque visualmente todos sean agentes, internamente no todos significan lo mismo. En los básicos son puntos en un espacio sencillo. En los específicos son diseños completos o configuraciones del problema.

### 5.3.3. Evaluación del fitness

La evaluación del fitness es una de las partes más importantes de todos los modelos. Es la función que dice si una solución es buena o mala.

En los modelos básicos, el fitness se calcula principalmente mediante la función esfera:

$$f(x,y) = x^2 + y^2$$

En este caso, el objetivo es minimizar el valor de la función, por lo que los mejores agentes son los que se acercan al origen.

También se ha incluido la posibilidad de usar paisajes aleatorios. En ese modo, el espacio de búsqueda se genera asignando elevaciones aleatorias y suavizándolas. Esto permite observar cómo se comportan los algoritmos en un entorno menos limpio y con más zonas engañosas.

En los modelos específicos, el fitness depende del problema:

- En la celosía de 72 barras, el fitness se calcula a partir del peso de la estructura y las penalizaciones por incumplir tensión o desplazamiento.
- En el canal de riego, el fitness se calcula combinando indicadores de adecuación, eficiencia, equidad y dependabilidad.
- En el domo de 120 barras, el fitness se calcula a partir del peso del domo y una penalización si no se cumplen las restricciones de frecuencia.

En todos los casos, el algoritmo intenta minimizar el fitness.

### 5.3.4. Actualización de mejores soluciones

Los tres algoritmos necesitan guardar información sobre las mejores soluciones encontradas.

En el modelo *WOA* se guarda una mejor solución global, que representa la presa hacia la que se mueven las ballenas. Además, en los modelos básicos se mantiene también la mejor posición personal de cada partícula.

En el modelo *GWO* se mantiene una jerarquía de tres líderes:

- alpha, mejor solución
- beta, segunda mejor solución
- delta, tercera mejor solución.

El resto de lobos actualizan su posición en función de estos tres líderes.



En el modelo CSA se guarda:

- la mejor posición personal de cada camaleón
- la mejor posición global del enjambre.

Esta información se actualiza en cada iteración, siempre que se encuentra una solución mejor.

### **5.3.5. Visualización y gráficos**

Una parte importante del proyecto ha sido la visualización. La idea no era simplemente ejecutar el algoritmo y obtener un número final, sino poder observar qué está pasando durante la búsqueda.

En los modelos básicos, la visualización muestra:

- la superficie de búsqueda
- la posición de las partículas
- las trayectorias, si se activa el modo de rastro
- la evolución del fitness.

En los modelos específicos, la visualización se adapta al problema:

- En la celosía de 72 barras se muestra la estructura 3D y las barras cambian de color según el nivel de tensión.
- En el canal de riego se muestra un esquema del canal, con compuertas y puntos de entrega.
- En el domo de 120 barras se muestra la estructura del domo, con colores y grosores asociados al área de los grupos de barras.

Además, se han añadido monitores y gráficas para mostrar datos relevantes, como mejor fitness, peso, factibilidad, indicadores hidráulicos, frecuencias naturales o vector de mejor solución.

## **5.4. Implementación de los modelos básicos**

### **5.4.1. Modelo básico WOA**

El modelo básico WOA implementa el comportamiento general del algoritmo de optimización de ballenas. Cada partícula representa una ballena situada en el espacio de búsqueda.

La implementación sigue las fases principales del WOA:

- actualización del mejor personal y global
- cálculo del parámetro “ $a$ ”, que decrece con las iteraciones
- generación de los coeficientes aleatorios “ $A$ ” y “ $C$ ”
- selección del tipo de movimiento mediante una probabilidad “ $p$ ”
- actualización de la posición de la ballena.

Cuando  $p < 0.5$ , el algoritmo aplica el mecanismo de encerramiento. Si  $A < 1$ , la ballena se mueve hacia la mejor solución global. Si  $A \geq 1$ , la ballena se mueve tomando como referencia otra ballena aleatoria, lo que favorece la exploración.

Cuando  $p \geq 0.5$ , se aplica el movimiento en espiral alrededor de la mejor solución global. Este movimiento intenta imitar la maniobra de caza con red de burbujas de las ballenas jorobadas.

En la interfaz se permite modificar parámetros como el tamaño de población, el presupuesto de evaluación, el modo de espacio aleatorio y el parámetro asociado a la espiral. También se incluye la opción de activar o desactivar el rastro de las partículas.

### **5.4.2. Modelo básico GWO**

El modelo básico GWO implementa el comportamiento del algoritmo del lobo gris. En este caso, cada partícula representa un lobo que se desplaza por el espacio de búsqueda.

La implementación se basa en la jerarquía del algoritmo:

- el mejor lobo se almacena como  $\alpha$
- el segundo mejor como  $\beta$
- el tercero como  $\delta$ .

En cada iteración, todos los lobos calculan una nueva posición a partir de la influencia de esos tres líderes. Para cada líder se calcula una posición candidata y, finalmente, la nueva posición del lobo se obtiene como el promedio de las tres.

El parámetro “ $a$ ” disminuye linealmente durante la ejecución. Al principio, los movimientos son más amplios y el algoritmo explora más. Conforme avanzan las iteraciones, los lobos se acercan más a las mejores soluciones y el algoritmo entra en una fase más explotadora.

En la implementación también se ha tenido cuidado con los límites del mundo de NetLogo, para evitar que los agentes se salgan del espacio representado. Esto es importante porque los movimientos calculados por el algoritmo pueden producir coordenadas fuera del rango visible.

#### **5.4.3. Modelo básico CSA**

El modelo básico CSA tiene una implementación algo distinta a los anteriores, ya que el algoritmo trabaja de forma más claramente vectorial. Cada camaleón mantiene:

- su posición actual
- su velocidad
- su mejor posición personal
- su fitness actual
- su mejor valor personal.

La iteración principal del CSA se divide en tres fases:

##### **1.- Búsqueda o percepción.**

El camaleón genera una nueva posición usando información de su mejor personal, del mejor global y de una componente aleatoria.

##### **2.- Rotación ocular.**

Se modifica el desplazamiento mediante una rotación en el espacio de búsqueda. En la implementación se ha simplificado esta idea realizando una rotación en un plano aleatorio.

##### **3.- Lengua o ataque.**

Se actualiza la velocidad y se calcula una nueva posición candidata orientada hacia zonas prometedoras.

Después de cada fase se aplica una aceptación tipo greedy: la nueva posición solo se acepta si mejora el valor actual. Esto evita que el camaleón empeore innecesariamente su solución.

El modelo CSA incluye varios parámetros configurables desde la interfaz, como son *csa-Pp*, *csa-p1*, *csa-p2*, *csa-alpha* y *csa-beta*. Estos parámetros permiten modificar el equilibrio entre exploración y explotación.

## **5.5. Implementación de los modelos específicos**

### **5.5.1. WOA aplicado a la celosía espacial de 72 barras**

El modelo específico WOA se ha implementado para resolver el problema de la celosía espacial de 72 barras. En este caso, cada ballena ya no representa una posición  $[x,y]$ , sino un diseño completo de la estructura.

Cada diseño se representa mediante un vector de 16 variables:

$$[A1,A2,...,A16]$$

Cada variable corresponde a un grupo de barras. Como el problema trabaja con secciones discretas, cada valor del diseño se ajusta a la sección permitida más cercana. Esto es importante porque, aunque el movimiento del WOA genera valores continuos, el problema real solo permite seleccionar secciones de una lista concreta.

La implementación del modelo estructural incluye:

- definición de las coordenadas de los 20 nodos
- definición de la conectividad de las 72 barras
- agrupación de barras en 16 grupos
- cálculo de longitudes
- definición de apoyos
- definición de dos casos de carga
- ensamblaje de la matriz de rigidez
- resolución de desplazamientos
- cálculo de tensiones axiales
- cálculo del peso total
- cálculo de penalizaciones.

La función de evaluación calcula primero el peso real de la estructura. Después resuelve los dos casos de carga y comprueba si se superan los límites de tensión o desplazamiento. Si hay violaciones, se añade una penalización al fitness.

De forma simplificada:

$$\text{fitness} = \text{peso} \cdot (1 + \text{violación})^{p(t)}$$

donde  $p(t)$  aumenta con las iteraciones. Con esto se consigue que al principio el algoritmo pueda explorar más libremente y, según avanza la ejecución, se vuelva más exigente con las restricciones.

En la visualización 3D se muestra la torre estructural. Los nodos apoyados y superiores se destacan con colores, y las barras se colorean según su nivel de tensión. También se muestra el peso del mejor diseño actual.

### **5.5.2. GWO aplicado al canal de riego**

El modelo específico GWO se ha implementado para un problema de reparto de agua en un canal de riego. En este modelo, cada lobo representa una configuración de aperturas de compuertas.

La solución candidata se representa como:

$$[TO1, TO2, TO3, TO4, TO5, TO6, CH1, CH2, CH3]$$

Los seis primeros valores corresponden a los puntos de entrega de agua y los tres últimos a las compuertas de control. Todos los valores se mantienen en el intervalo  $[0, 1]$ .

El modelo no utiliza un simulador hidráulico profesional. En su lugar, se ha implementado una simulación simplificada del canal dentro de NetLogo. Esta decisión se tomó porque integrar un simulador completo habría aumentado muchísimo la complejidad del proyecto. Aun así, el modelo intenta mantener la idea principal del problema: repartir un caudal limitado entre varios puntos de entrega, teniendo en cuenta el efecto de las compuertas.

La implementación del canal incluye:

- generación de demandas
- definición del caudal disponible
- simulación temporal del reparto
- cálculo de entregas medias
- cálculo de indicadores de rendimiento
- actualización de las mejores aperturas encontradas.

La función de fitness se basa en:

$$FF = (1 - PA) + (1 - PF) + PE + PD$$

donde:

- PA mide la adecuación
- PF mide la eficiencia
- PE mide la equidad
- PD mide la fiabilidad.

Cuanto menor es el valor de esta función, mejor es la configuración encontrada. En la interfaz se muestran las mejores aperturas de los turnouts y check gates, las entregas medias y los indicadores principales. Además, la vista del canal se actualiza para mostrar visualmente la configuración de la mejor solución.

### **5.5.3. CSA aplicado al domo espacial de 120 barras**

El modelo específico CSA se ha implementado para optimizar el domo espacial de 120 barras. Este modelo ha sido uno de los más delicados, porque combina optimización con cálculo estructural dinámico.

Cada camaleón representa un diseño formado por siete áreas:

$$[A1, A2, \dots, A7]$$

Cada área corresponde a un grupo de barras del domo. Internamente, ese vector se expande para asignar áreas a las 120 barras reales.

El modelo estructural del domo incluye:

- 49 nodos
- 120 barras
- 7 grupos de barras
- apoyos en el anillo exterior
- masas no estructurales
- matriz de rigidez
- matriz de masa
- cálculo aproximado o preciso de frecuencias naturales.

El objetivo es minimizar el peso del domo cumpliendo:

$$f_1 \geq 9 \text{ Hz}$$

$$f_2 \geq 11 \text{ Hz}$$

La función de evaluación calcula el peso real y añade una penalización si alguna frecuencia no alcanza el valor mínimo requerido.

Uno de los problemas de este modelo es que el cálculo modal puede ser bastante costoso. Por eso se implementaron varios modos de evaluación:

- RÁPIDO, para pruebas visuales y ejecuciones ágiles.
- MIXTO, para una evaluación aproximada más representativa.
- PRECISO, para validar resultados con un cálculo modal más completo.

Esta decisión fue importante porque, si todas las evaluaciones fueran precisas desde el principio, el modelo sería demasiado lento para usarlo de forma interactiva (hablamos de horas para llegar al resultado del paper).

En la vista 3D se representa el domo. Las barras cambian de color y grosor según el área asignada al grupo correspondiente. Esto permite ver de un vistazo qué partes del domo está reforzando el algoritmo.

## **5.6. Exportación y ejecución de experimentos**

Además de la ejecución interactiva desde la interfaz, en los modelos básicos se implementaron procedimientos para lanzar experimentos por lotes y exportar resultados en formato CSV y así poder realizar comparaciones en igualdad de condiciones entre ellos.

Esta parte resulta útil para comparar ejecuciones con distintos parámetros sin tener que hacerlas manualmente una por una. En concreto, se prepararon pruebas variando:

- tamaño de población
- número de iteraciones
- semilla aleatoria.

Los resultados exportados incluyen información como:

- algoritmo utilizado
- tamaño de población
- semilla
- número de iteraciones
- mejor posición encontrada
- mejor valor de fitness
- valor medio de la población.

Este sistema permite analizar de forma más ordenada el comportamiento de los algoritmos y preparar tablas o gráficas para el capítulo de resultados.

En los modelos específicos, la experimentación es más costosa, sobre todo en los modelos estructurales, porque cada evaluación requiere más cálculos. Por eso, en estos casos se ha dado más importancia a la validación del modelo comparándolo con los papers originales y a la observación de la evolución del mejor diseño.

### **5.7. Propuesta de actividades docentes y pruebas guiadas**

Además de implementar los modelos, se ha considerado importante que la herramienta pueda utilizarse como apoyo docente. Por ello, a partir del estudio de los algoritmos y de las pruebas realizadas, se han planteado una serie de actividades recomendadas para el alumnado.

Estas actividades tienen como objetivo que el usuario no se limite a ejecutar el modelo y observar una animación, sino que pueda modificar parámetros, comparar comportamientos y sacar conclusiones sobre conceptos como exploración, explotación, convergencia, factibilidad, sensibilidad ante restricciones o coste computacional.

Las actividades propuestas en este apartado están relacionadas con los seis modelos desarrollados: los tres modelos básicos y los tres modelos específicos. En el Anexo A se incluye además un manual de usuario más detallado para facilitar la ejecución de cada modelo.



### 5.7.1. Actividad propuesta para los modelos básicos

Una primera actividad docente consiste en comparar los tres algoritmos básicos, WOA, GWO y CSA, sobre la función esfera. Esta función tiene un mínimo global conocido en el punto (0,0), por lo que resulta sencilla de interpretar.

La configuración inicial propuesta sería:

- Función objetivo: esfera
- Población: 50 agentes
- Iteraciones: 50
- Espacio aleatorio: desactivado

En los modelos que utilicen evaluation-budget en lugar de max-iterations, se puede calcular el presupuesto como:

$$\text{evaluation-budget} = \text{population-size} \times \text{iteraciones}$$

Por tanto, para esta primera prueba:

```
population-size = 50  
iteraciones = 50  
evaluation-budget = 2500  
random-search-space = false
```

El alumnado ejecutaría los tres modelos básicos con una configuración equivalente y anotaría el mejor valor de fitness obtenido en cada caso. Además, debería observar visualmente cómo se mueve la población de agentes durante la simulación.

Después, se repetiría la prueba aumentando la población a 100 agentes, manteniendo el mismo número de iteraciones:

```
population-size = 100  
iteraciones = 50  
evaluation-budget = 5000  
random-search-space = false
```

Con esto se puede estudiar si aumentar la población mejora la búsqueda o si simplemente aumenta el coste de ejecución sin aportar una mejora clara.

Las preguntas que se pueden plantear son:

- ¿Qué algoritmo obtiene el menor valor de fitness?
- ¿Qué algoritmo converge visualmente más rápido hacia el mínimo?
- ¿Cuál mantiene mayor dispersión de la población durante las primeras iteraciones?
- ¿Se observa algún caso de convergencia prematura?
- ¿Cómo afecta aumentar la población de 50 a 100 agentes?
- ¿Coinciden las observaciones visuales con los resultados numéricos?

Esta actividad permite trabajar conceptos comunes a las metaheurísticas, como población, fitness, exploración, explotación y convergencia.

### **5.7.2. Actividad propuesta para el modelo básico WOA**

En el caso del modelo básico WOA, una actividad interesante es estudiar el efecto del tamaño de población y del paisaje de búsqueda.

La prueba propuesta sería ejecutar el modelo en dos situaciones:

- Caso A: función esfera  
population-size = 50  
evaluation-budget = 2500  
random-search-space = false  
woa-spiral-b = 0.5
  
- Caso B: paisaje aleatorio  
population-size = 50  
evaluation-budget = 2500  
random-search-space = true  
woa-spiral-b = 0.5

Después, se puede repetir la prueba aumentando la población:

population-size = 100

evaluation-budget = 5000

En ambos casos se mantiene el mismo número aproximado de iteraciones. El alumnado debería observar cómo cambia el comportamiento de las ballenas cuando el espacio de búsqueda es suave y regular frente a cuando el paisaje tiene zonas irregulares.

Las preguntas propuestas son:

- ¿Convergen las ballenas más rápido en la función esfera o en el paisaje aleatorio?
- ¿Aparecen zonas donde las ballenas se agrupan aunque no sean la mejor solución global?
- ¿Qué ocurre al aumentar el número de ballenas?
- ¿Se aprecia visualmente la diferencia entre exploración y explotación?
- ¿El movimiento en espiral ayuda a concentrar la búsqueda alrededor de la mejor solución?

Con esta actividad, el alumnado puede relacionar el comportamiento del WOA con sus mecanismos principales: aproximación a la mejor solución, exploración mediante individuos aleatorios y movimiento en espiral.

### **5.7.3. Actividad propuesta para el modelo básico GWO**

En el modelo básico GWO, la actividad docente puede centrarse en la jerarquía alfa-beta-delta. La idea es que el alumnado observe cómo los lobos actualizan sus posiciones siguiendo a los tres líderes principales.

Una configuración sencilla sería:

```
population-size = 50  
evaluation-budget = 2500  
random-search-space = false
```

Después, se repetiría la prueba con un mayor tamaño de población y un paisaje aleatorio:

```
population-size = 100  
evaluation-budget = 5000  
random-search-space = true
```

Si se desea mantener la actividad más sencilla, landscape-smoothness, hills-number y trace-mode pueden dejarse con los valores por defecto de la interfaz.

Las preguntas propuestas son:

- ¿Cómo se comporta la población cuando se identifican alfa, beta y delta?
- ¿Los lobos se concentran rápidamente alrededor de los líderes?
- ¿Qué ocurre si se utiliza un paisaje aleatorio?
- ¿La población mantiene diversidad o se agrupa demasiado pronto?
- ¿Cómo afecta el presupuesto de evaluación al resultado final?

Esta actividad permite entender de forma visual por qué GWO suele converger de forma estable en problemas sencillos, ya que no depende únicamente de una solución, sino de la influencia conjunta de las tres mejores.

#### 5.7.4. Actividad propuesta para el modelo básico CSA

En el modelo básico CSA, la actividad puede centrarse en observar sus tres fases principales: búsqueda, rotación ocular y ataque mediante lengua.

La configuración inicial recomendada sería:

```
population-size = 50
max-iterations = 50
random-search-space = false
csa-Pp = 0.10
csa-p1 = 0.85
csa-p2 = 0.85
csa-alpha = 3.5
csa-beta = 3.0
```

Después se pueden realizar variaciones sobre esa configuración base. Por ejemplo:

Prueba 1: mayor exploración

```
csa-Pp = 0.30
```

Prueba 2: mayor influencia del mejor global

```
csa-p1 = 0.60
```

```
csa-p2 = 0.95
```

Prueba 3: comparación con paisaje aleatorio

```
random-search
```

El objetivo no es que el alumnado encuentre una combinación perfecta de parámetros, sino que observe cómo cambia el comportamiento del enjambre cuando se modifican.

Preguntas propuestas:

- ¿Qué ocurre si se aumenta o reduce csa-Pp?
- ¿Cómo cambia el comportamiento al modificar csa-p1 y csa-p2?
- ¿El algoritmo se mueve de forma más exploratoria o más dirigida?
- ¿Se aprecia la influencia del mejor personal y del mejor global?
- ¿El CSA converge de forma estable o presenta más variabilidad que WOA y GWO?

Esta actividad ayuda a entender que CSA tiene más parámetros ajustables y que su comportamiento puede variar bastante según la configuración elegida.

#### **5.7.5. Actividad propuesta para la celosía espacial de 72 barras**

En el modelo específico WOA aplicado a la celosía de 72 barras, la actividad docente puede centrarse en la relación entre peso, restricciones y factibilidad.

Una primera prueba puede realizarse con una configuración moderada:

```
population-size = 20  
max-iterations = 1000  
woa-spiral-b = 0.5  
stress-limit = 25000  
displacement-limit = 0.25  
young-modulus = 10000000  
density = 0.10
```

El alumnado debería ejecutar el modelo y observar:

- el peso del mejor diseño;
- si el diseño es factible;
- la violación total de restricciones;
- el desplazamiento máximo;
- la tensión máxima;
- el vector de áreas obtenido.

Después, se puede repetir la prueba aumentando la población:

```
population-size = 100  
max-iterations = 1000  
woa-spiral-b = 0.5  
stress-limit = 25000  
displacement-limit = 0.26  
young-modulus = 10000000  
density = 0.10
```

Esta última prueba no debe interpretarse como una comparación estricta con el benchmark original, ya que modifica el límite de desplazamiento. Su utilidad docente está en observar cómo una pequeña variación en una restricción puede afectar al peso y a la factibilidad.

Preguntas propuestas:

- ¿El diseño más ligero es siempre factible?
- ¿Qué restricción condiciona más el resultado: tensión o desplazamiento?
- ¿Qué ocurre si se aumenta el tamaño de población?
- ¿Cómo cambia el peso si se modifica el límite de desplazamiento?
- ¿Por qué una solución con poco peso puede no ser válida?

Esta actividad permite trabajar una idea importante en optimización estructural: no basta con minimizar el peso, sino que el diseño debe cumplir todas las restricciones.

#### **5.7.6. Actividad propuesta para el canal de riego**

En el modelo GWO aplicado al canal de riego, la actividad docente puede orientarse a comparar situaciones con caudal suficiente y situaciones de escasez.

Se proponen dos configuraciones:

Caso A: caudal suficiente

num-turnouts = 6

population-size = 9

evaluation-budget = 10000

base-demand = 0.10

available-flow = 1.0

section-decay = 0.040

demand-pattern = uniform

Caso B: escasez

num-turnouts = 6

population-size = 9

evaluation-budget = 10000

base-demand = 0.12

available-flow = 0.7

section-decay = 0.040

demand-pattern = uniform

En cada caso, el alumnado debe observar:

- mejor fitness;
- aperturas de los turnouts;
- aperturas de las check gates;
- entregas medias;
- PA;
- PE;
- PD.

Preguntas propuestas:

- ¿Qué ocurre con el fitness cuando hay caudal suficiente?
- ¿Qué ocurre cuando aparece escasez?
- ¿PA disminuye cuando no se puede entregar toda la demanda?
- ¿PE y PD aumentan en escenarios más difíciles?
- ¿El algoritmo reparte el agua de forma equilibrada o favorece algunos puntos?
- ¿Qué papel tienen las compuertas de control en el reparto?



Esta actividad permite relacionar el algoritmo GWO con un problema de reparto de recursos, donde la mejor solución no depende solo de maximizar una entrega, sino de equilibrar varios indicadores.

#### **5.7.7. Actividad propuesta para el domo espacial de 120 barras**

En el modelo CSA aplicado al domo de 120 barras, la actividad docente más interesante es comparar los distintos modos de evaluación: rápido, mixto y preciso.

Una primera prueba puede realizarse en modo mixto:

```
evaluation-mode = MIXTO
population-size = 10
max-iterations = 300
accurate-rotations = 12000
csa-Pp = 0.10
csa-p1 = 0.85
csa-p2 = 0.85
csa-alpha = 3.5
csa-beta = 3.0
young-modulus-factor = 1.0
density-factor = 1.0
nonstructural-mass-factor = 1.0
frequency-target-factor = 1.0
```

Después, se puede validar una solución en modo preciso:

```
evaluation-mode = PRECISO
population-size = 20
```

max-iterations = 500  
accurate-rotations = 20000  
csa-Pp = 0.10  
csa-p1 = 0.85  
csa-p2 = 0.85  
csa-alpha = 3.5  
csa-beta = 3.0  
young-modulus-factor = 1.0  
density-factor = 1.0  
nonstructural-mass-factor = 1.0  
frequency-target-factor = 1.0

El alumnado debería observar:

- peso real;
- fitness;
- frecuencia f1;
- frecuencia f2;
- margen de frecuencia;
- violación total;
- factibilidad.

Preguntas propuestas:

- ¿Qué diferencia hay entre un resultado rápido y uno preciso?
- ¿Por qué el modo mixto puede ser demasiado optimista?
- ¿Qué ocurre cuando una frecuencia queda muy cerca del límite?
- ¿Un diseño ligero siempre cumple las restricciones dinámicas?
- ¿Cómo afecta el coste computacional al uso del modelo?
- ¿Por qué es necesario validar los resultados finales con mayor precisión?

Esta actividad permite trabajar un aspecto muy importante de los modelos de ingeniería: el equilibrio entre rapidez de cálculo y fiabilidad de los resultados.

#### **5.7.8. Valor docente de las actividades propuestas**

Estas actividades muestran que el trabajo realizado no se limita a implementar algoritmos, sino que también proporciona una base para utilizarlos como herramienta de aprendizaje. El estudio de WOA, GWO y CSA ha permitido identificar qué parámetros son más interesantes, qué comportamientos conviene observar y qué comparaciones pueden ayudar a entender mejor cada método.

En los modelos básicos, las actividades se centran principalmente en comprender el comportamiento de los algoritmos. En los modelos específicos, las actividades permiten analizar cómo cambian los resultados cuando aparecen restricciones, indicadores de rendimiento o cálculos más complejos.

De esta forma, el alumnado puede pasar de una visión teórica de los algoritmos a una experiencia más práctica e interactiva. En lugar de estudiar únicamente ecuaciones o pseudocódigo, puede modificar parámetros, observar la evolución de los agentes y razonar sobre los resultados obtenidos.

## CAPITULO 6

### PRUEBAS Y RESULTADOS

#### 6.1 Introducción

En este capítulo se presentan las pruebas realizadas sobre los modelos desarrollados y los resultados obtenidos. El objetivo principal no es únicamente comprobar que los modelos se ejecutan sin errores, sino también analizar si el comportamiento de los algoritmos se corresponde con lo esperado.

Para organizar mejor el análisis, las pruebas se han dividido en dos partes. En primer lugar, se comparan los tres *modelos básicos* —WOA, GWO y CSA— utilizando una función matemática sencilla y conocida. En segundo lugar, se analizan los *modelos específicos* aplicados a problemas de ingeniería.

En este primer bloque se estudia el comportamiento de los algoritmos sobre la función esfera, ya que el paisaje aleatorio al ser totalmente aleatorio no lo podemos reproducir igual en todas las pruebas y no sería comparable de forma equitativa. Esta prueba permite comparar la capacidad de convergencia de los tres métodos en un entorno controlado, donde el mínimo global se conoce de antemano.

#### 6.2. Metodología de comparación de los modelos básicos

Para comparar los tres algoritmos de optimización se ha utilizado la función esfera, definida como:

$$f(x,y) = x^2 + y^2$$

Esta función tiene un mínimo global conocido en el punto:

$$(x,y)=(0,0)$$

donde el valor óptimo es:

$$f(0,0)=0$$

Por tanto, el objetivo de los algoritmos es encontrar valores de  $x$  e  $y$  que hagan que el valor de la función sea lo más pequeño posible. Aunque se trata de una función sencilla, resulta muy útil como primera prueba porque permite comprobar si los algoritmos son capaces de converger hacia una solución conocida.

Los algoritmos comparados han sido:

- WOA, algoritmo de ballenas (Whale Optimization Algorithm)
- GWO, algoritmo del lobo gris (Grey Wolf Optimizer)
- CSA, algoritmo del camaleón (Chameleon Swarm Algorithm)

Para que la comparación fuese lo más equilibrada posible, se utilizaron las mismas configuraciones de población, iteraciones y semillas aleatorias en los tres algoritmos. En concreto, se probaron poblaciones de 50, 100 y 200 individuos, con 10, 50 y 100 iteraciones. Cada combinación se repitió con cinco semillas distintas, de la 1 a la 5. En total se realizaron 45 ejecuciones por algoritmo, es decir, 135 ejecuciones entre los tres métodos.

La configuración experimental queda resumida en la siguiente tabla:

| Parámetro                           | Valores utilizados |
|-------------------------------------|--------------------|
| Algoritmos                          | WOA, GWO, CSA      |
| Función objetivo                    | Función esfera     |
| Tamaño de población                 | 50, 100, 200       |
| Iteraciones                         | 10, 50, 100        |
| Semillas                            | 1, 2, 3, 4, 5      |
| Número de ejecuciones por algoritmo | 45                 |
| Número total de ejecuciones         | 135                |
| Métrica principal                   | best_val           |

La métrica principal utilizada para comparar los algoritmos ha sido *best\_val*, que representa el mejor valor encontrado en cada ejecución. Como el problema consiste en minimizar la función esfera, cuanto menor sea este valor, mejor será el resultado.

Además, para no depender únicamente de una ejecución concreta, se calculó la media de *best\_val* para cada combinación de algoritmo, población e iteraciones. Esta media permite comparar el comportamiento general de cada algoritmo de forma más estable.

### 6.3. Resultados obtenidos en la función esfera

A partir de los resultados obtenidos, se observa que los tres algoritmos son capaces de aproximarse al mínimo de la función esfera. Sin embargo, no todos lo hacen con la misma precisión ni con la misma rapidez.

En general, los mejores resultados se reparten entre GWO y CSA, mientras que WOA queda por detrás en la mayoría de combinaciones analizadas. Considerando las nueve combinaciones de población e iteraciones, GWO obtiene el mejor resultado medio en siete de ellas. Por su parte, CSA obtiene el mejor resultado en dos combinaciones, ambas con población 50. WOA no obtiene el mejor resultado medio en ninguna combinación, aunque mejora de forma clara cuando aumenta el número de iteraciones.

La siguiente tabla resume el algoritmo ganador en cada configuración:

| Población | Iteraciones | Mejor algoritmo | Media de <i>best_val</i> |
|-----------|-------------|-----------------|--------------------------|
| 50        | 10          | CSA             | $4.815 \cdot 10^{-7}$    |
| 50        | 50          | GWO             | $1.279 \cdot 10^{-32}$   |
| 50        | 100         | CSA             | $1.081 \cdot 10^{-65}$   |
| 100       | 10          | GWO             | $9.326 \cdot 10^{-11}$   |
| 100       | 50          | GWO             | $2.248 \cdot 10^{-51}$   |
| 100       | 100         | GWO             | $2.317 \cdot 10^{-81}$   |
| 200       | 10          | GWO             | $1.430 \cdot 10^{-11}$   |
| 200       | 50          | GWO             | $1.039 \cdot 10^{-59}$   |
| 200       | 100         | GWO             | $1.198 \cdot 10^{-101}$  |

Estos resultados muestran una tendencia bastante clara: al aumentar el número de iteraciones, los tres algoritmos mejoran, pero GWO es el que consigue los valores medios más bajos en la mayoría de casos.

### 6.3.1. Resultados con población 50

Con una población de 50 individuos, los resultados están más repartidos entre GWO y CSA.

Con 10 iteraciones, CSA obtiene el mejor resultado medio, con un valor aproximado de:

$$4.815 \cdot 10^{-7}$$

mientras que GWO obtiene:

$$1.183 \cdot 10^{-6}$$

y WOA queda más alejado, con:

$$1.293 \cdot 10^{-3}$$

En este primer caso, CSA consigue una mejor aproximación al mínimo con pocas iteraciones. Esto puede indicar que, para poblaciones pequeñas y ejecuciones cortas, su mecanismo de búsqueda puede ser bastante eficaz.

Con 50 iteraciones, GWO pasa a ser el mejor algoritmo, alcanzando una media de:

$$1.279 \cdot 10^{-32}$$

CSA también obtiene un valor muy bajo:

$$4.664 \cdot 10^{-32}$$

mientras que WOA mejora de forma notable, pero queda por detrás:

$$6.495 \cdot 10^{-11}$$

Aquí ya se aprecia que WOA necesita más iteraciones para acercarse al óptimo, mientras que GWO y CSA convergen de forma mucho más agresiva.

Con 100 iteraciones, CSA vuelve a obtener el mejor resultado medio:

$$1.081 \cdot 10^{-65}$$

GWO obtiene:

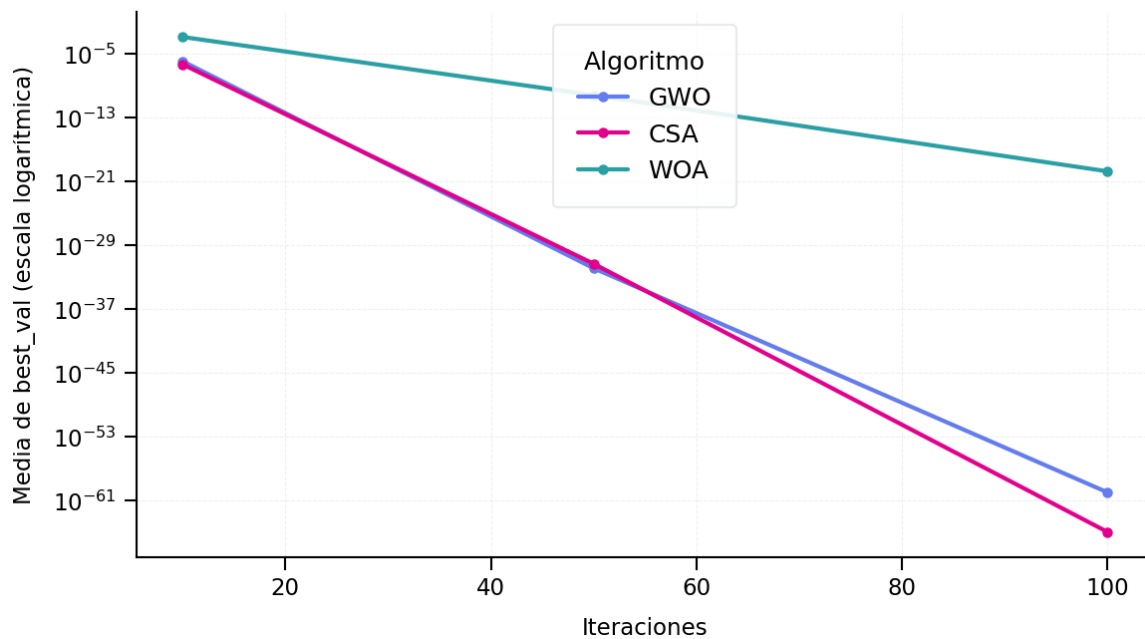
$$1.001 \cdot 10^{-60}$$

y WOA alcanza:

$$1.813 \cdot 10^{-20}$$

Aunque GWO también obtiene un valor extremadamente cercano a cero, en esta configuración concreta CSA logra la mejor media.

### Comparativa de convergencia — Población = 50



#### 6.3.2. Resultados con población 100

Con población 100, el comportamiento cambia y GWO obtiene los mejores resultados en las tres configuraciones de iteraciones.

Con 10 iteraciones, GWO consigue una media aproximada de:

$$9.326 \cdot 10^{-11}$$

CSA obtiene:

$$1.086 \cdot 10^{-8}$$

y WOA:

$$7.737 \cdot 10^{-4}$$

En este caso, GWO ya consigue desde pocas iteraciones un valor muy próximo al mínimo global.

Con 50 iteraciones, GWO baja hasta:

$$2.248 \cdot 10^{-51}$$

CSA obtiene:

$$2.539 \cdot 10^{-36}$$

y WOA:

$$1.607 \cdot 10^{-14}$$



Los tres mejoran respecto a 10 iteraciones, pero la diferencia entre GWO y los otros dos algoritmos se hace bastante grande.

Con 100 iteraciones, GWO alcanza:

$$2.317 \cdot 10^{-81}$$

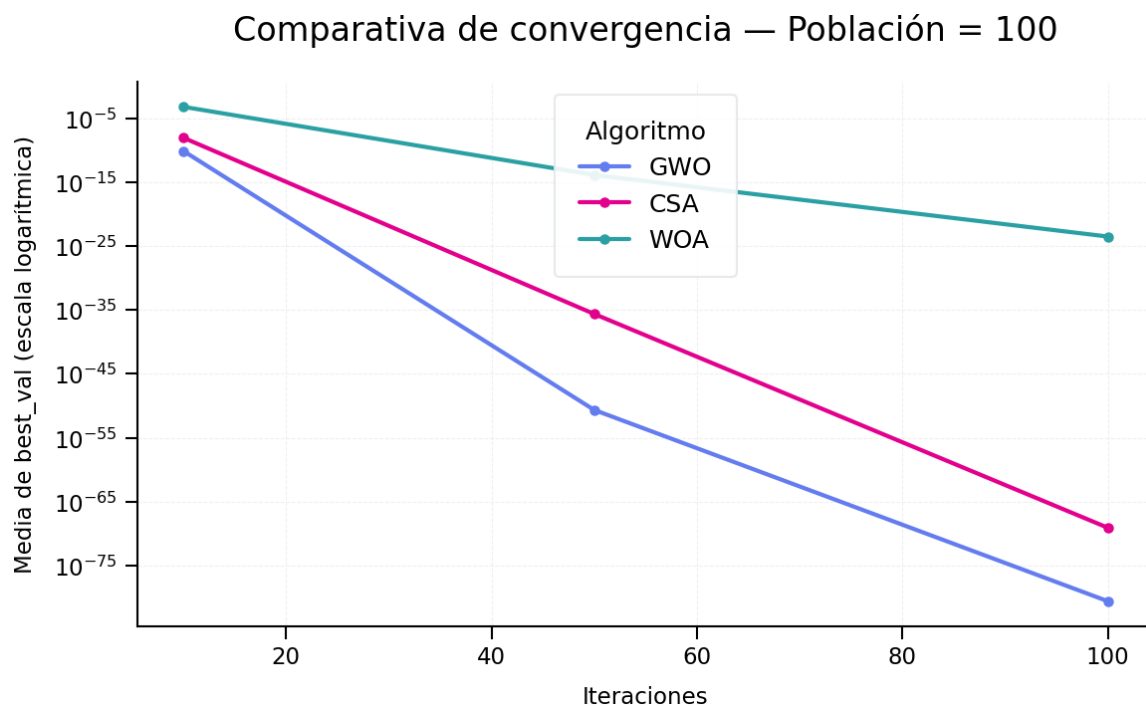
CSA obtiene:

$$7.620 \cdot 10^{-70}$$

y WOA:

$$3.505 \cdot 10^{-24}$$

De nuevo, GWO consigue el mejor valor medio. CSA también se mantiene muy competitivo, mientras que WOA mejora, pero no llega a la misma precisión.



### 6.3.3. Resultados con población 200

Con población 200, GWO mantiene el mejor rendimiento en todas las configuraciones.

Con 10 iteraciones, GWO obtiene:

$$1.430 \cdot 10^{-11}$$

CSA obtiene:

$$4.101 \cdot 10^{-10}$$

y WOA:

$$6.067 \cdot 10^{-5}$$

Aunque CSA también consigue un valor bajo, GWO vuelve a situarse por delante

Con 50 iteraciones, GWO alcanza:

$$1.039 \cdot 10^{-59}$$

CSA obtiene:

$$6.723 \cdot 10^{-40}$$

y WOA:

$$1.151 \cdot 10^{-15}$$

La diferencia entre GWO y el resto vuelve a ser clara.

Finalmente, con 100 iteraciones, GWO consigue el mejor valor medio de toda la comparativa:

$$1.198 \cdot 10^{-101}$$

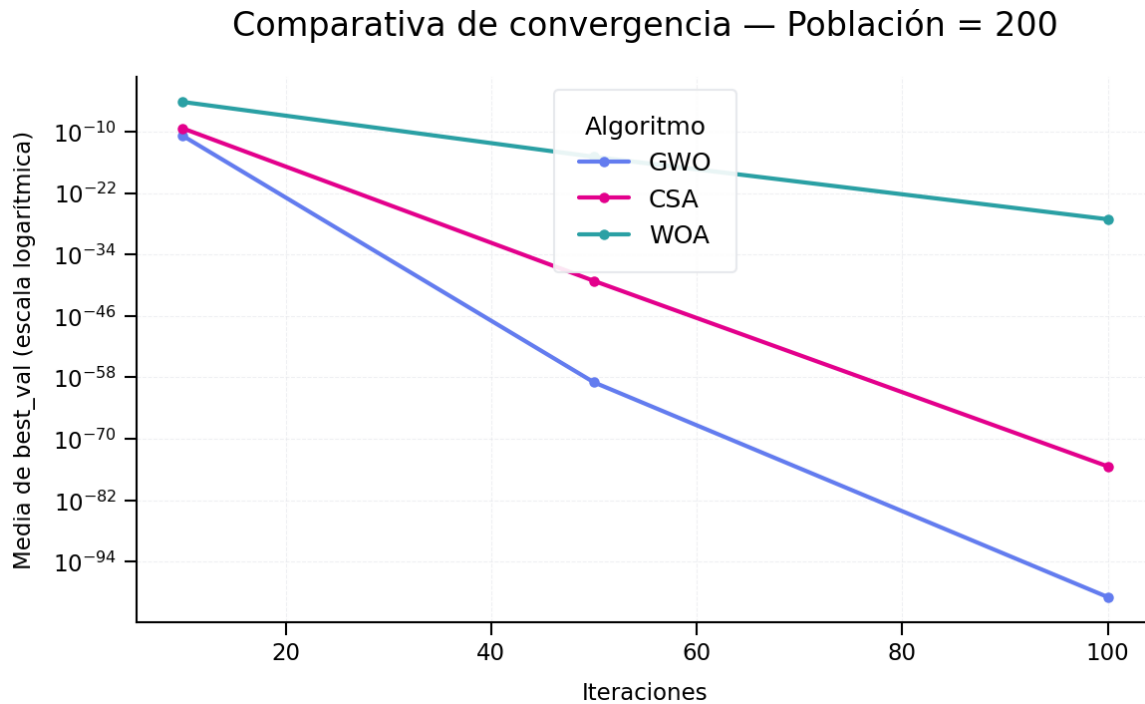
CSA obtiene:

$$4.038 \cdot 10^{-76}$$

y WOA:

$$7.416 \cdot 10^{-28}$$

Esta configuración muestra el mejor resultado global del experimento.



#### 6.3.4. Comparación general entre WOA, GWO y CSA

De forma global, los resultados muestran que GWO es el algoritmo más sólido en esta comparativa. Obtiene el mejor resultado medio en siete de las nueve combinaciones estudiadas, lo que indica una convergencia muy rápida y precisa hacia el mínimo global de la función esfera.

Este comportamiento tiene sentido si se tiene en cuenta la naturaleza del problema. La función esfera es suave, continua y tiene un único mínimo global. En este tipo de función, la estrategia de GWO, basada en seguir la influencia conjunta de los tres mejores individuos de la población, resulta muy efectiva.

CSA también presenta un comportamiento muy competitivo. Aunque no gana en la mayoría de configuraciones, sí consigue los mejores resultados en dos casos con población 50. Esto indica que CSA puede alcanzar soluciones de muy alta precisión, especialmente en algunas configuraciones concretas.

WOA queda como el algoritmo con resultados más modestos dentro de esta comparación. Aun así, no se puede decir que funcione mal. De hecho, sus valores mejoran mucho al aumentar el número de iteraciones. El problema es que, en esta función concreta, GWO y CSA convergen con mucha más precisión.

También hay que tener en cuenta una limitación importante: en esta comparación se han igualado población, iteraciones y semillas, pero no necesariamente el número total de evaluaciones internas de la función objetivo. Por tanto, estos resultados deben entenderse como una comparación inicial y controlada del comportamiento de los tres modelos básicos, no como una conclusión absoluta sobre el rendimiento general de los algoritmos en cualquier problema.

#### **6.3.5. Conclusiones de la comparativa básica**

A partir de los experimentos realizados sobre la función esfera, se pueden extraer varias conclusiones.

En primer lugar, los tres algoritmos implementados son capaces de aproximarse al mínimo global de la función. Esto permite validar que los modelos básicos funcionan correctamente y que las reglas de actualización de cada algoritmo producen una mejora progresiva del fitness.

En segundo lugar, GWO obtiene el mejor rendimiento global en esta comparativa. Consigue el mejor valor medio en siete de las nueve combinaciones de población e iteraciones, y alcanza el mejor resultado global con población 200 y 100 iteraciones.

En tercer lugar, CSA se sitúa como el segundo algoritmo más competitivo. Aunque gana solo en dos configuraciones, sus valores son muy cercanos a cero y en algunos casos supera a GWO, especialmente con población 50.

Por último, WOA queda en tercera posición para esta prueba concreta. No obtiene el mejor resultado en ninguna combinación, pero mejora de forma clara al aumentar el número de iteraciones. Esto indica que el algoritmo funciona, aunque en la función esfera no converge con tanta precisión como GWO o CSA.

En resumen, para esta comparativa concreta, el orden general observado es:

1º GWO

2º CSA

3º WOA

Esta conclusión debe interpretarse dentro de las condiciones del experimento: función esfera, poblaciones de 50, 100 y 200 individuos, 10, 50 y 100 iteraciones, y cinco semillas por combinación. Como trabajo futuro, sería interesante ampliar la comparación con más funciones objetivo, más semillas y una comparación basada también en el número total de evaluaciones de la función.

#### **6.4. Resultados de los modelos específicos**

Una vez analizados los modelos básicos sobre la función esfera, se realizaron pruebas con los modelos específicos desarrollados en el proyecto. En estos casos, el objetivo no era únicamente observar la convergencia del algoritmo, sino comprobar cómo se comportaba cada metaheurística al aplicarse a un problema de ingeniería con variables, restricciones y una función objetivo más elaborada.

Los tres modelos específicos desarrollados fueron:

- WOA aplicado a la celosía espacial de 72 barras.
- GWO aplicado al reparto de agua en un canal de riego.
- CSA aplicado al domo espacial de 120 barras.

##### **6.4.1. WOA aplicado a la celosía espacial de 72 barras**

El primer modelo específico evaluado fue el correspondiente a la optimización estructural de la celosía espacial de 72 barras mediante el algoritmo de ballenas (Whale Optimization Algorithm). El objetivo de este modelo es minimizar el peso total de la estructura seleccionando áreas transversales para 16 grupos de barras, respetando las restricciones de tensión y desplazamiento.

Como referencia se tomó el benchmark clásico de la celosía de 72 barras, cuyo diseño óptimo publicado tiene un peso aproximado de 389.33 lb. Antes de realizar las pruebas de optimización, se comprobó que el modelo implementado en NetLogo era capaz de reproducir correctamente el peso y la factibilidad del diseño benchmark. Esta comprobación permitió validar que la geometría, la conectividad, las cargas, las restricciones y el cálculo del peso estaban correctamente definidos.

Para las pruebas se utilizaron los parámetros físicos del problema original:

| Parámetro                | Valor                   |
|--------------------------|-------------------------|
| Límite de tensión        | 25000 psi               |
| Límite de desplazamiento | 0.25 in                 |
| Módulo de Young          | 10000000 psi            |
| Densidad                 | 0.10 lb/in <sup>3</sup> |
| Parámetro espiral $b$    | 0.5                     |

A continuación, se muestran tres ejecuciones representativas del modelo, realizadas con distintas configuraciones de población e iteraciones.

#### Caso 1: población 20 y 1000 iteraciones

En la primera prueba se utilizó una población de 20 ballenas y 1000 iteraciones. Esta configuración es cercana a una parametrización básica del algoritmo, con un coste computacional moderado.

| Parámetro       | Valor |
|-----------------|-------|
| population-size | 20    |
| max-iterations  | 1000  |
| woa-spiral-b    | 0.5   |
| ticks           | 1000  |

El resultado obtenido fue un diseño factible, con un peso de:

$$\text{Weight} = 527.2984 \text{ lb}$$

El vector de áreas encontrado fue:

$$A1=1$$

$$A2=0.563$$

$$A3=0.111$$

$$A4=1$$

$$A5=1.62$$

$$A6=0.563$$

$A7=0.111$   
 $A8=1$   
 $A9=1$   
 $A10=1$   
 $A11=0.111$   
 $A12=0.25$   
 $A13=0.563$   
 $A14=0.563$   
 $A15=0.25$   
 $A16=1$

Los resultados obtenidos con dicha solución fueron:

| Métrica    | Valor       |
|------------|-------------|
| g-best-val | 527.2984    |
| Peso real  | 527.2984 lb |
| Factible   | Sí          |

Este resultado cumple las restricciones del problema, aunque queda por encima del peso de referencia de 389.33 lb. Aun así, sirve como primera comprobación de que el algoritmo es capaz de generar diseños válidos.

#### Caso 2: población 30 y 2000 iteraciones

En la segunda prueba se aumentó la población a 30 ballenas y el número de iteraciones a 2000.

| Parámetro       | Valor |
|-----------------|-------|
| population-size | 30    |
| max-iterations  | 2000  |
| woa-spiral-b    | 0.5   |
| ticks           | 2000  |

El resultado obtenido también fue factible, con un peso de:

Weight=639.2428 lb

El vector de áreas encontrado fue:

A1=0.766

A2=0.766

A3=0.766

A4=0.766

A5=1

A6=0.766

A7=0.766

A8=0.766

A9=1

A10=0.766

A11=0.766

A12=0.391

A13=0.766

A14=0.766

A15=0.766

A16=0.391

Los resultados obtenidos con dicha solución fueron:

| Métrica    | Valor       |
|------------|-------------|
| g-best-val | 639.2428    |
| Peso real  | 639.2428 lb |
| Factible   | Sí          |

Aunque esta configuración utiliza más población e iteraciones que el caso anterior, el peso obtenido es mayor. Esto muestra una característica habitual de las metaheurísticas: una ejecución con más recursos no garantiza necesariamente una solución mejor, ya que el resultado depende también de la población inicial, de la trayectoria de búsqueda y de la convergencia hacia una región concreta del espacio de soluciones.



### Caso 3: población 100 y 1000 iteraciones con restricción de desplazamiento relajada

En la tercera prueba se utilizó una configuración con una población bastante mayor que en los casos anteriores. Además, en esta ejecución se modificó ligeramente el límite de desplazamiento, pasando de 0.25 in a 0.26 in. La intención de esta prueba era observar si, al permitir una pequeña relajación de la restricción de desplazamiento, el algoritmo era capaz de acercarse más al peso de referencia publicado en la literatura.

| Parámetro          | Valor                   |
|--------------------|-------------------------|
| stress-limit       | 25000 psi               |
| displacement-limit | 0.26 in                 |
| young-modulus      | 10000000 psi            |
| density            | 0.10 lb/in <sup>3</sup> |
| population-size    | 100                     |
| woa-spiral-b       | 0.5                     |
| max-iterations     | 1000                    |
| ticks              | 1000                    |

El resultado obtenido fue un diseño factible dentro de esta configuración, con un peso de:

$$\text{Weight} = 415.6720 \text{ lb}$$

El vector de áreas obtenido fue:

$$\begin{aligned} A1 &= 1.8 \\ A2 &= 0.563 \\ A3 &= 0.141 \\ A4 &= 0.141 \\ A5 &= 1.8 \\ A6 &= 0.563 \end{aligned}$$

A7=0.141  
 A8=0.141  
 A9=0.563  
 A10=0.563  
 A11=0.141  
 A12=0.141  
 A13=0.563  
 A14=0.563  
 A15=0.141  
 A16=0.563

Los resultados obtenidos con dicha solución fueron:

| Métrica                      | Valor               |
|------------------------------|---------------------|
| g-best-val                   | 415.6720            |
| Peso real                    | 415.6720 lb         |
| Factible                     | Sí                  |
| Violación total              | 0                   |
| Violación por tensión        | 0                   |
| Violación por desplazamiento | 0                   |
| Tensión máxima caso 1        | 8468.2261 psi       |
| Tensión máxima caso 2        | 487.5061 psi aprox. |
| Desplazamiento máximo caso 1 | 0.003260 in         |
| Desplazamiento máximo caso 2 | 0.258878 in         |

Este caso es el que más se aproxima al peso de referencia del paper, que es aproximadamente 389.33 lb. La diferencia relativa es:

$$\frac{415.6720 - 389.33}{389.33} \cdot 100 \approx 6.77\%$$

Por tanto, el resultado queda aproximadamente un 6.77 % por encima del valor de referencia. Es una mejora clara respecto a los casos anteriores.

No obstante, hay que tener en cuenta un detalle importante: este caso se ha obtenido usando un límite de desplazamiento de 0.26 in, mientras que el benchmark original utiliza 0.25 in. De hecho, el desplazamiento máximo del segundo caso de carga es:

$$d_{\max}=0.258878 \text{ in}$$

Este valor cumple la restricción relajada de 0.26 in, pero no cumpliría estrictamente el límite original de 0.25 in. Por ese motivo, este resultado no debe interpretarse como una reproducción exacta del óptimo del benchmark, sino como una prueba adicional para analizar cómo afecta la relajación de la restricción al peso obtenido.

#### Comparación de resultados WOA

La siguiente tabla resume los tres casos analizados:

| Caso | Población | Iteraciones | Límite desplazamiento | Peso real   | Factible en ejecución | Diferencia respecto a benchmark |
|------|-----------|-------------|-----------------------|-------------|-----------------------|---------------------------------|
| 1    | 20        | 1000        | 0.25 in               | 527.2984 lb | Sí                    | 35.44 %                         |
| 2    | 30        | 2000        | 0.25 in               | 639.2428 lb | Sí                    | 64.19 %                         |
| 3    | 100       | 1000        | 0.26 in               | 415.6720 lb | Sí                    | 6.77 %                          |

El Caso 1 y el Caso 2 utilizan las restricciones originales del benchmark. Ambos generan diseños factibles, aunque con pesos superiores al valor de referencia. El mejor diseño factible manteniendo el límite original de desplazamiento es el del Caso 1, con 527.2984 lb.

El Caso 3, en cambio, utiliza una restricción de desplazamiento ligeramente más flexible. Con esta modificación, el algoritmo consigue acercarse bastante más al peso publicado en la literatura, alcanzando 415.6720 lb. Sin embargo, al no usar exactamente el mismo límite de desplazamiento, esta prueba debe considerarse como una comparación orientativa y no como una reproducción directa del benchmark.

### Discusión de los resultados WOA

Los resultados muestran que el modelo WOA implementado en NetLogo es capaz de generar diseños válidos para la celosía espacial de 72 barras. En los dos primeros casos, usando el límite de desplazamiento original de 0.25 in, los diseños encontrados son factibles, aunque bastante más pesados que el diseño de referencia.

También se observa que aumentar la población o el número de iteraciones no garantiza siempre una mejora directa. El Caso 2 usa más iteraciones y más población que el Caso 1, pero obtiene un peso mayor. Esto es algo normal en algoritmos metaheurísticos, donde el resultado depende de la población inicial, de los movimientos aleatorios y de la zona del espacio de búsqueda hacia la que converge el algoritmo.

El Caso 3 permite ver algo interesante: al relajar ligeramente la restricción de desplazamiento, el algoritmo encuentra un diseño mucho más ligero y cercano al valor del paper. Esto indica que el desplazamiento es una de las restricciones que más condiciona la búsqueda. El algoritmo puede reducir bastante el peso, pero llega un punto en el que pequeñas variaciones de sección provocan que el desplazamiento se acerque mucho al límite admisible.

En este caso, el diseño obtenido tiene un desplazamiento máximo de 0.258878 in, por lo que cumple el límite relajado de 0.26 in, pero no el límite original de 0.25 in. Por tanto, no puede considerarse una solución equivalente al benchmark original, aunque sí sirve para mostrar que el modelo se mueve en una región razonable del espacio de búsqueda.

En conjunto, estos resultados son coherentes con la dificultad del problema. La celosía de 72 barras es un problema discreto, con restricciones y múltiples combinaciones posibles de secciones. El algoritmo WOA consigue reducir el peso y encontrar diseños factibles, pero alcanzar exactamente el óptimo publicado requiere una búsqueda más fina y posiblemente una parametrización más ajustada o un mayor número de ejecuciones independientes.

### Conclusión parcial del modelo WOA

A partir de las pruebas realizadas, se puede concluir que el modelo WOA estructural funciona correctamente desde el punto de vista de simulación y evaluación. El modelo calcula el peso, comprueba restricciones, penaliza soluciones no válidas y permite visualizar la evolución del diseño.

Comparado con el benchmark de referencia, los resultados no alcanzan exactamente el peso óptimo publicado de 389.33 lb, pero sí muestran un comportamiento razonable. Con las restricciones originales se obtienen diseños factibles, aunque más pesados, y al relajar ligeramente el desplazamiento se obtiene un diseño mucho más cercano al valor del paper.

Por tanto, el modelo cumple su objetivo principal dentro del proyecto: servir como herramienta visual e interactiva para estudiar la aplicación de WOA a un problema realista de optimización estructural.

#### **6.4.2. GWO aplicado al canal de riego**

El segundo modelo específico evaluado fue el modelo de reparto de agua en un canal de riego mediante el algoritmo del lobo gris (Grey Wolf Optimizer). En este caso, cada lobo representa una posible configuración de aperturas de compuertas:

[TO1,TO2,TO3,TO4,TO5,TO6,CH1,CH2,CH3]

Los seis primeros valores corresponden a los puntos de entrega de agua, mientras que los tres últimos representan las compuertas de control del canal.

El modelo toma como referencia el artículo de Shahverdi y Maestre, donde GWO se aplica a la programación del reparto de agua en canales de riego utilizando indicadores de adecuación, eficiencia, equidad y fiabilidad. En el artículo original, el modelo se acopla con un simulador hidráulico completo, mientras que en este TFG se utiliza una versión simplificada implementada directamente en NetLogo. Por tanto, la comparación se interpreta a nivel de comportamiento general, no como una reproducción exacta del simulador hidráulico ICSS empleado en el paper.

La función de fitness utilizada es:

$$FF=(1-PA)+(1-PF)+PE+PD$$

donde:

- PA mide la adecuación del reparto.
- PF mide la eficiencia.
- PE mide la equidad espacial.
- PD mide la fiabilidad temporal.

Como el problema se plantea como una minimización, valores menores de fitness indican mejores soluciones. A partir de los valores de fitness, PA, PE y PD mostrados por el modelo, se puede obtener el valor de PF mediante:

$$PF=2-PA+PE+PD-FF$$

En todos los casos evaluados, PF resulta igual o prácticamente igual a 1, lo que indica que el modelo no está generando sobreentregas significativas.

#### Caso 1: demanda alta con caudal suficiente

En el primer caso se utilizó una demanda base de 0.20 y un caudal disponible de 1.5.

| Parámetro         | Valor   |
|-------------------|---------|
| num-turnouts      | 6       |
| base-demand       | 0.20    |
| available-flow    | 1.5     |
| section-decay     | 0.040   |
| demand-pattern    | uniform |
| population-size   | 9       |
| evaluation-budget | 10000   |
| Iteraciones       | 1112    |

El resultado obtenido fue:

| Métrica       | Valor   |
|---------------|---------|
| Mejor fitness | 0.02337 |
| PA            | 0.99564 |
| PF            | ≈ 1.000 |
| PE            | 0.00484 |
| PD            | 0.01418 |

Las entregas medias obtenidas fueron:

[0.199 0.2 0.198 0.2 0.2 0.198]

Las mejores aperturas encontradas fueron aproximadamente:

TO = [0.723 0.9 0.788 1 1 0.8]

CH = [0.391 0.532 0.315]

Este resultado es muy bueno, ya que las entregas medias están prácticamente igualadas con la demanda objetivo de 0.20. La adecuación es muy cercana a 1, la eficiencia calculada también es prácticamente 1, y los valores de equidad y fiabilidad son bajos. Esto indica un reparto suficiente, estable y bastante equilibrado.

#### Caso 2: demanda media con caudal suficiente

En el segundo caso se utilizó una demanda base de 0.18 y un caudal disponible de 1.2.

| Parámetro         | Valor   |
|-------------------|---------|
| num-turnouts      | 6       |
| base-demand       | 0.18    |
| available-flow    | 1.2     |
| section-decay     | 0.040   |
| demand-pattern    | uniform |
| population-size   | 9       |
| evaluation-budget | 10000   |
| Iteraciones       | 1112    |

El resultado obtenido fue:

| Métrica       | Valor                   |
|---------------|-------------------------|
| Mejor fitness | 0.00199                 |
| PA            | 0.99968                 |
| PF            | $\approx 1.000$         |
| PE            | $6.11706 \cdot 10^{-4}$ |
| PD            | 0.00106                 |

Las entregas medias fueron:

[0.18 0.18 0.18 0.18 0.18 0.18]

Las mejores aperturas fueron:

TO = [1 1 0.811 1 1 1]

CH = [1 0.304 0.037]

Este caso presenta un resultado prácticamente ideal. El fitness es muy bajo, la adecuación está casi en 1 y las entregas medias coinciden con la demanda de los seis puntos de entrega. Además, PE y PD son muy pequeños, lo que indica que el reparto es equilibrado y estable.

### Caso 3: demanda baja con caudal suficiente

En el tercer caso se utilizó una demanda base de 0.10 y un caudal disponible de 1.0.

| Parámetro         | Valor   |
|-------------------|---------|
| num-turnouts      | 6       |
| base-demand       | 0.10    |
| available-flow    | 1.0     |
| section-decay     | 0.040   |
| demand-pattern    | uniform |
| population-size   | 9       |
| evaluation-budget | 10000   |
| Iteraciones       | 1112    |



El resultado obtenido fue el mejor de todos los casos:

| <b>Métrica</b> | <b>Valor</b> |
|----------------|--------------|
| Mejor fitness  | 0            |
| PA             | 1            |
| PF calculado   | 1            |
| PE             | 0            |
| PD             | 0            |

Las entregas medias fueron:

[0.1 0.1 0.1 0.1 0.1 0.1]

Las mejores aperturas fueron:

TO = [1 1 0.857 0.942 1 1]

CH = [0.006 0.324 0.355]

En este escenario, el algoritmo consigue entregar exactamente la demanda requerida en todos los puntos. Por eso el fitness alcanza el valor 0. La adecuación y la eficiencia son máximas, y no aparece variabilidad espacial ni temporal en el reparto.

#### Caso 4: situación de escasez

En el cuarto caso se redujo el caudal disponible, generando una situación más exigente para el algoritmo. La demanda base fue de 0.12, mientras que el caudal disponible fue de 0.7.

| <b>Parámetro</b>  | <b>Valor</b> |
|-------------------|--------------|
| num-turnouts      | 6            |
| base-demand       | 0.12         |
| available-flow    | 0.7          |
| section-decay     | 0.040        |
| demand-pattern    | uniform      |
| population-size   | 9            |
| evaluation-budget | 10000        |
| Iteraciones       | 1112         |

El resultado obtenido fue:

| Métrica       | Valor           |
|---------------|-----------------|
| Mejor fitness | 0.09483         |
| PA            | 0.95554         |
| PF            | $\approx 1.000$ |
| PE            | 0.029           |
| PD            | 0.02137         |

Las entregas medias obtenidas fueron:

[0.113 0.111 0.12 0.113 0.116 0.115]

Las mejores aperturas fueron aproximadamente:

TO = [0.853 0.893 0.994 0.924 0.845 0.801]

CH = [1 1 0]

Este escenario es más complicado, ya que el caudal disponible no permite satisfacer perfectamente toda la demanda. Como consecuencia, el fitness aumenta respecto a los casos anteriores. Aun así, el algoritmo consigue mantener un reparto bastante razonable: ningún punto de entrega queda completamente desatendido y los valores de equidad y fiabilidad se mantienen en niveles moderados.

#### Comparación de los cuatro casos GWO

La siguiente tabla resume los resultados principales obtenidos:

| Caso | Demanda base | Caudal disponible | Fitness | PA      | PF              | PE                      | PD      |
|------|--------------|-------------------|---------|---------|-----------------|-------------------------|---------|
| 1    | 0.20         | 1.5               | 0.02337 | 0.99564 | $\approx 1.000$ | 0.00484                 | 0.01418 |
| 2    | 0.18         | 1.2               | 0.00199 | 0.99968 | $\approx 1.000$ | $6.11706 \cdot 10^{-4}$ | 0.00106 |
| 3    | 0.10         | 1.0               | 0       | 1       | 1               | 0                       | 0       |
| 4    | 0.12         | 0.7               | 0.09483 | 0.95554 | $\approx 1.000$ | 0.029                   | 0.02137 |

Los resultados muestran una tendencia clara. En los casos con caudal suficiente, el algoritmo encuentra configuraciones de compuertas capaces de satisfacer prácticamente toda la demanda. Esto se refleja en valores de PA y PF muy cercanos a 1 y en valores bajos de PE y PD.

El mejor caso es el tercero, con demanda base 0.10 y caudal disponible 1.0, donde el fitness llega a 0. En este escenario, el reparto es perfecto dentro de la simulación simplificada.

En cambio, el cuarto caso representa una situación de escasez. Aquí el fitness sube hasta 0.09483, la adecuación baja a 0.95554 y aumentan tanto la equidad como la fiabilidad. Este comportamiento es lógico: al reducir el caudal disponible, el algoritmo tiene menos margen para satisfacer todas las demandas de forma uniforme.

#### Discusión de los resultados GWO

Los resultados obtenidos son coherentes con lo esperado. En escenarios donde existe caudal suficiente, GWO encuentra configuraciones de compuertas que permiten entregar prácticamente la demanda requerida en todos los puntos. En escenarios más restrictivos, el reparto empeora, aunque se mantiene dentro de valores razonables.

Esta tendencia coincide con la idea general del artículo de referencia: GWO funciona especialmente bien en condiciones normales, mientras que en situaciones de escasez el problema se vuelve más difícil y los indicadores empeoran. En el paper, los autores también observan que las condiciones de falta de agua reducen la calidad del reparto y aumentan la dificultad del proceso de optimización.

Aun así, hay que recordar que el modelo desarrollado en este TFG no usa el simulador hidráulico ICSS, sino una representación simplificada del canal implementada en NetLogo. Por ello, los valores numéricos no deben compararse de forma directa con los del artículo, sino como una prueba de comportamiento. La comparación más importante no está en clavar exactamente los mismos números, sino en comprobar que el modelo responde de forma lógica: buenos resultados con caudal suficiente y peores resultados cuando el caudal disponible baja.

En ese sentido, el modelo cumple su objetivo. Permite visualizar cómo GWO

ajusta las aperturas de los turnouts y check gates, cómo evoluciona el fitness y cómo cambian los indicadores de reparto según las condiciones del problema.

#### **6.4.3. CSA aplicado al domo espacial de 120 barras**

El tercer modelo específico evaluado fue el correspondiente al algoritmo del camaleón (Chameleon Swarm Algorithm) aplicado al problema del domo espacial de 120 barras. Este problema consiste en minimizar el peso de una estructura espacial formada por 120 barras agrupadas en 7 grupos de diseño, cumpliendo restricciones dinámicas sobre las dos primeras frecuencias naturales:

$$f_1 \geq 9 \text{ Hz}$$

$$f_2 \geq 11 \text{ Hz}$$

Como referencia, el benchmark del domo espacial de 120 barras presenta un peso óptimo publicado cercano a 8888.74 kg, cumpliendo las restricciones de frecuencia. En este modelo, cada camaleón representa un vector de áreas:

$$[A1, A2, A3, A4, A5, A6, A7]$$

Una dificultad importante de este problema es el coste computacional. A diferencia de la función esfera o del canal simplificado, aquí cada evaluación puede requerir cálculos estructurales y cálculo modal. Por ese motivo, el modelo incorpora distintos modos de evaluación (el modo RAPIDO no tenía mucho sentido usarlo aquí para resultados donde no hay prisa en ver como corre, pero para ejecutarlo en directo sí):

- MIXTO, más rápido pero aproximado.
- PRECISO, más lento pero más fiable para validar resultados.

Se realizaron tres pruebas principales.

### Caso 1: modo mixto con 12000 rotaciones

En primer lugar, se ejecutó el modelo en modo MIXTO, con el objetivo de comprobar el comportamiento rápido del CSA antes de realizar ejecuciones más costosas.

| Parámetro          | Valor |
|--------------------|-------|
| evaluation-mode    | MIXTO |
| accurate-rotations | 12000 |
| population-size    | 10    |
| max-iterations     | 300   |
| csa-Pp             | 0.10  |
| csa-p1             | 0.85  |
| csa-p2             | 0.85  |
| csa-alpha          | 3.5   |
| csa-beta           | 3.0   |

El resultado obtenido fue:

| Métrica         | Valor             |
|-----------------|-------------------|
| Peso real       | 501.27276 kg      |
| g-best-val      | 501.27276         |
| f1              | 466.47652 Hz      |
| f2              | 1074.17237 Hz     |
| Margen f1       | 457.47652         |
| Margen f2       | 1063.17237 aprox. |
| Factible        | Sí                |
| Violación total | 0                 |

El vector de áreas obtenido fue:

A1=1

A2=1

A3=1

A4=1

117

A5=1

A6=1

A7=1

Aunque este resultado aparece como factible dentro del modo MIXTO, no debe interpretarse como una solución final válida frente al benchmark. El peso obtenido es extremadamente bajo y las frecuencias calculadas son muy elevadas, lo que indica que el modelo aproximado utilizado en este modo resulta demasiado optimista para este problema.

Por tanto, el modo MIXTO resulta útil para observar rápidamente la dinámica del algoritmo, pero no es suficiente para validar resultados finales del domo de 120 barras.

#### Caso 2: modo preciso con 12000 rotaciones

Después se realizó una ejecución en modo PRECISO, manteniendo 12000 rotaciones para el cálculo modal.

| Parámetro          | Valor   |
|--------------------|---------|
| evaluation-mode    | PRECISO |
| accurate-rotations | 12000   |
| population-size    | 10      |
| max-iterations     | 300     |
| csa-Pp             | 0.10    |
| csa-p1             | 0.85    |
| csa-p2             | 0.85    |
| csa-alpha          | 3.5     |
| csa-beta           | 3.0     |

El resultado obtenido fue:

| Métrica         | Valor                   |
|-----------------|-------------------------|
| Peso real       | 8718.12856 kg           |
| g-best-val      | 8727.42944              |
| f1              | 8.99916 Hz              |
| f2              | 11.00233 Hz             |
| Margen f1       | $8.37079 \cdot 10^{-4}$ |
| Margen f2       | 0.00233048 aprox.       |
| Factible        | No                      |
| Violación total | $9.30088 \cdot 10^{-5}$ |

El vector de áreas obtenido fue:

$$A1=19.8448$$

$$A2=39.2935$$

$$A3=9.6209$$

$$A4=21.0202$$

$$A5=10.2429$$

$$A6=12.9294$$

$$A7=14.9538$$

Este resultado es muy cercano a ser factible. La segunda frecuencia cumple la restricción, ya que:

$$f_2=11.00233 \text{ Hz}$$

Sin embargo, la primera frecuencia queda ligeramente por debajo del límite:

$$f_1=8.99916 \text{ Hz}$$

La diferencia respecto a los 9 Hz exigidos es muy pequeña, y por eso la violación total también es muy reducida. Este caso muestra que el algoritmo ya se encuentra en una zona muy próxima a la frontera de factibilidad.

### Caso 3: modo preciso con 20000 rotaciones

Finalmente, se realizó una ejecución más exigente en modo PRECISO, aumentando tanto el número de rotaciones del cálculo modal como el tamaño de población y las iteraciones. Esta ejecución fue la más costosa computacionalmente, llegando a requerir muchas horas de cálculo (más de 1 día).

| Parámetro          | Valor   |
|--------------------|---------|
| evaluation-mode    | PRECISO |
| accurate-rotations | 20000   |
| population-size    | 20      |
| max-iterations     | 500     |
| csa-Pp             | 0.10    |
| csa-p1             | 0.85    |
| csa-p2             | 0.85    |
| csa-alpha          | 3.5     |
| csa-beta           | 3.0     |

El resultado obtenido fue:

| Métrica         | Valor                         |
|-----------------|-------------------------------|
| Peso real       | 8715.13528 kg                 |
| g-best-val      | 8715.13528                    |
| f1              | 9.00029 Hz                    |
| f2              | 11.0002 Hz                    |
| Margen f1       | $2.89922 \cdot 10^{-4}$       |
| Margen f2       | $1.9749 \cdot 10^{-4}$ aprox. |
| Factible        | Sí                            |
| Violación total | 0                             |



El vector de áreas obtenido fue:

$$A1=19.0437$$

$$A2=41.0894$$

$$A3=10.8422$$

$$A4=21.1485$$

$$A5=10.4108$$

$$A6=11.7304$$

$$A7=14.5348$$

Este es el mejor resultado obtenido para el modelo CSA del domo. El diseño es factible, ya que cumple simultáneamente las restricciones:

$$f_1 \geq 9 \text{ Hz}$$

$$f_2 \geq 11 \text{ Hz}$$

Además, las dos frecuencias quedan muy próximas a sus límites mínimos. Esto es habitual en problemas de optimización estructural con restricciones activas: el algoritmo intenta reducir el peso todo lo posible, y para ello acaba llevando las restricciones justo hasta el borde admisible.

El peso obtenido fue:

$$\text{Weight}=8715.13528 \text{ kg}$$

Este valor es incluso inferior al peso de referencia de 8888.74 kg del benchmark. No obstante, esta comparación debe interpretarse con cuidado, ya que el modelo implementado en NetLogo puede diferir en algunos detalles numéricos respecto al modelo utilizado en la literatura. Por tanto, más que afirmar que se supera el benchmark, lo correcto es indicar que el resultado obtenido es comparable y coherente, y que dentro de la formulación implementada se ha alcanzado un diseño factible de peso muy próximo al valor de referencia.

La diferencia relativa respecto al benchmark es:

$$\frac{8715.13528 - 8888.74}{8888.74} \cdot 100 \approx -1.95\%$$

Es decir, el peso obtenido es aproximadamente un 1.95 % inferior al valor de referencia. Este resultado refuerza la idea de que el modelo preciso está funcionando de forma razonable y que el CSA ha sido capaz de encontrar una solución competitiva.

### Comparación de los casos CSA

La siguiente tabla resume los tres casos analizados:

| Caso | Modo    | Rotaciones | Población | Iteraciones | Peso   | f1     | f2     | Factible |
|------|---------|------------|-----------|-------------|--------|--------|--------|----------|
| 1    | MIXTO   | 12000      | 10        | 300         | 501.2  | 466.47 | 1074.1 | Sí       |
| 2    | PRECISO | 12000      | 10        | 300         | 8718.1 | 8.999  | 11.002 | No       |
| 3    | PRECISO | 20000      | 20        | 500         | 8715.1 | 9.000  | 11.00  | Sí       |

La comparación muestra claramente la diferencia entre los modos de evaluación. El modo MIXTO permite obtener resultados muy rápidos, pero su evaluación es demasiado aproximada para validar el problema del domo. En cambio, los modos PRECISOS proporcionan resultados mucho más realistas.

El Caso 2 queda prácticamente en la frontera de factibilidad, con una violación muy pequeña de la primera frecuencia. El Caso 3, al aumentar la precisión del cálculo y la capacidad de búsqueda del algoritmo, consigue finalmente una solución factible.

### Discusión de los resultados CSA

Los resultados del CSA muestran que el domo de 120 barras es el modelo más exigente de los desarrollados en el proyecto. La necesidad de calcular frecuencias naturales hace que cada evaluación sea costosa, especialmente cuando se usa el modo PRECISO con un número elevado de rotaciones.

Aun así, los resultados obtenidos son muy positivos. El modo PRECISO con 20000 rotaciones consiguió una solución factible con un peso de 8715.13528 kg, cumpliendo las dos restricciones dinámicas. Además, las frecuencias finales quedaron prácticamente ajustadas a los límites admisibles, lo que indica que el algoritmo está aprovechando al máximo las restricciones para reducir peso.

También se observa que el modo MIXTO no debe utilizarse para validar

resultados finales. Aunque resulta útil para pruebas rápidas y visualización, puede producir diseños aparentemente factibles pero físicamente poco realistas. Por ello, la validación final debe realizarse siempre en modo PRECISO.

Comparando con la literatura, el resultado obtenido se encuentra en el mismo orden de magnitud que el benchmark del domo de 120 barras. Esto permite considerar que la implementación es coherente y que el CSA ha sido capaz de adaptarse correctamente a un problema estructural con restricciones dinámicas.

#### **6.4.4. Conclusión general de los modelos específicos**

Los tres modelos específicos han permitido comprobar cómo se comportan los algoritmos bioinspirados cuando se aplican a problemas más complejos que una función matemática sencilla.

En el caso de la celosía de 72 barras, WOA fue capaz de generar diseños factibles y de aproximarse al benchmark al modificar las condiciones de búsqueda, aunque no llegó a reproducir exactamente el valor óptimo publicado con las mismas restricciones. Esto muestra la dificultad de los problemas estructurales discretos y la importancia del tratamiento de restricciones.

En el modelo del canal de riego, GWO obtuvo resultados muy buenos en escenarios con caudal suficiente y un comportamiento coherente en situaciones de escasez. Aunque el modelo hidráulico es simplificado, los indicadores evolucionan de forma lógica y permiten observar cómo el algoritmo ajusta las aperturas de las compuertas.

Finalmente, el modelo CSA aplicado al domo de 120 barras obtuvo el resultado más cercano a una validación completa frente a la literatura. Tras una ejecución precisa y costosa, el algoritmo consiguió un diseño factible con peso comparable al benchmark y con las frecuencias naturales prácticamente ajustadas a los límites requeridos.

En conjunto, los resultados muestran que los modelos específicos cumplen su objetivo principal: adaptar los algoritmos bioinspirados a problemas de ingeniería y permitir analizar visualmente su comportamiento. Aunque NetLogo no es la herramienta más eficiente para cálculo numérico intensivo, ha resultado útil para construir modelos interactivos que combinan optimización, visualización y análisis de resultados.

## **CAPITULO 7**

### **CONCLUSIONES Y TRABAJO FUTURO**

#### **7.1. Conclusiones generales**

En este Trabajo Fin de Grado se han desarrollado varios modelos interactivos en NetLogo 3D para simular y visualizar el comportamiento de algoritmos bioinspirados aplicados a problemas de optimización. En concreto, se han trabajado tres algoritmos: Whale Optimization Algorithm o WOA, Grey Wolf Optimizer o GWO, y Chameleon Swarm Algorithm o CSA.

El objetivo principal del proyecto era crear modelos que no solo ejecutaran los algoritmos, sino que también permitieran observar su comportamiento de una forma visual y comprensible. En este sentido, el trabajo ha buscado acercar algoritmos que normalmente se estudian mediante fórmulas, pseudocódigo o implementaciones numéricas a un entorno más interactivo, donde el usuario pueda modificar parámetros y ver cómo cambia el proceso de búsqueda.

A lo largo del proyecto se han desarrollado seis modelos: tres modelos básicos y tres modelos específicos. Los modelos básicos permiten estudiar el funcionamiento general de cada algoritmo sobre espacios de búsqueda sencillos, como la función esfera y paisajes aleatorios. Los modelos específicos aplican cada algoritmo a un problema de ingeniería diferente: una celosía espacial de 72 barras, un canal de riego y un domo espacial de 120 barras.

En conjunto, puede decirse que los objetivos principales del trabajo se han cumplido. Se han implementado los algoritmos seleccionados, se han creado interfaces interactivas, se han incluido visualizaciones tridimensionales y se han realizado pruebas para analizar su comportamiento. Además, los modelos desarrollados pueden servir como base para futuras ampliaciones o como material de apoyo en asignaturas relacionadas con optimización y computación inteligente.

Además, el estudio previo de los algoritmos no solo ha servido para poder implementarlos, sino también para identificar qué aspectos resultan más interesantes desde el punto de vista docente. Esta idea se ha materializado en una serie de actividades y pruebas guiadas, descritas en el Capítulo 5, que permiten al alumnado modificar parámetros, observar comportamientos y extraer conclusiones sobre cada algoritmo. Comprender cómo funcionan la exploración,

la explotación, la influencia de los líderes, el movimiento en espiral o las fases propias del CSA ha permitido proponer en la documentación de cada modelo una serie de pruebas recomendadas para el alumnado. De esta forma, el manual de usuario no se limita a explicar qué botón pulsar, sino que también orienta sobre qué parámetros modificar, qué comportamientos observar y qué conclusiones se pueden extraer de cada ejecución.

## **7.2. Conclusiones sobre los modelos básicos**

Los modelos básicos han permitido comprobar el comportamiento general de WOA, GWO y CSA sobre problemas sencillos de optimización. La función esfera ha sido especialmente útil para esta parte, ya que tiene un mínimo global conocido en el punto (0,0), lo que facilita interpretar si los algoritmos convergen correctamente.

A partir de las pruebas realizadas, los tres algoritmos fueron capaces de aproximarse al mínimo de la función esfera. Sin embargo, no todos lo hicieron con la misma rapidez ni con la misma precisión. En la comparativa realizada, GWO fue el algoritmo con mejor rendimiento global, obteniendo los mejores valores medios en la mayoría de configuraciones analizadas. CSA también mostró un comportamiento muy competitivo, llegando incluso a superar a GWO en algunas combinaciones concretas con población pequeña. WOA, por su parte, obtuvo resultados más modestos, aunque también mejoró claramente al aumentar el número de iteraciones.

Más allá de los resultados numéricos, los modelos básicos han cumplido una función importante dentro del proyecto: facilitar la comprensión visual de los algoritmos. Ver cómo se desplaza una población de agentes, cómo se agrupa alrededor de regiones prometedoras o cómo puede quedar atrapada en zonas locales ayuda mucho a entender conceptos como exploración, explotación, convergencia y óptimos locales.

También se ha comprobado que los paisajes aleatorios aportan una dificultad adicional interesante. Frente a la función esfera, que es suave y tiene un único

mínimo, los paisajes aleatorios permiten observar comportamientos más irregulares y situaciones donde los algoritmos pueden converger hacia zonas que no son necesariamente las mejores del espacio completo.

Estas observaciones han servido como base para plantear actividades docentes sobre los modelos básicos, descritas en el Capítulo 5. En ellas se propone que el alumnado compare algoritmos, modifique parámetros y relacione los resultados visuales con conceptos como exploración, explotación y convergencia.

### **7.3. Conclusiones sobre los modelos específicos**

Los modelos específicos han sido la parte más compleja del proyecto, ya que cada uno ha requerido adaptar un algoritmo bioinspirado a un problema de ingeniería distinto.

En el *modelo de la celosía espacial de 72 barras*, se aplicó WOA a un problema de optimización estructural con variables discretas, restricciones de tensión y restricciones de desplazamiento. El modelo fue capaz de generar diseños factibles y de evaluar correctamente el peso, las tensiones y los desplazamientos. Aunque los resultados obtenidos no alcanzaron exactamente el peso óptimo publicado en la literatura, sí mostraron un comportamiento coherente. Además, al relajar ligeramente la restricción de desplazamiento, el algoritmo consiguió acercarse bastante más al valor de referencia, lo que muestra la sensibilidad del problema frente a esta restricción.

En el *modelo del canal de riego*, se aplicó GWO a un problema de reparto de agua entre varios puntos de entrega. Aunque el modelo desarrollado no utiliza un simulador hidráulico profesional, sí reproduce de forma simplificada la lógica del problema: ajustar aperturas de compuertas para mejorar indicadores como adecuación, eficiencia, equidad y dependabilidad. Los resultados obtenidos fueron coherentes con lo esperado: en escenarios con caudal suficiente, el algoritmo consiguió repartos muy buenos, mientras que en situaciones de escasez el fitness empeoró y los indicadores reflejaron una mayor dificultad.

En el *modelo del domo espacial de 120 barras*, el CSA consiguió finalmente un diseño factible en modo preciso, con un peso de 8715.13528 kg y frecuencias  $f_1=9.00029$  Hz y  $f_2=11.0002$  Hz. Este resultado es comparable al valor de

referencia del benchmark y muestra que, aunque el cálculo modal es costoso, el modelo puede obtener soluciones coherentes cuando se utiliza una evaluación suficientemente precisa. En general, los modelos específicos muestran que los algoritmos bioinspirados pueden adaptarse a problemas muy diferentes, pero también dejan claro que cada problema tiene sus propias dificultades. No es lo mismo optimizar una función matemática sencilla que resolver un problema con restricciones estructurales, variables discretas, indicadores hidráulicos o cálculo modal.

Además, el análisis de estos modelos específicos ha permitido plantear actividades más cercanas a problemas reales de ingeniería, recogidas en el Capítulo 5. Estas actividades permiten observar no solo la convergencia del algoritmo, sino también el efecto de las restricciones, la factibilidad, los indicadores de rendimiento o el coste computacional de cada modelo.

#### **7.4. Aportación docente de las actividades propuestas**

Además de implementar y probar los modelos, el estudio de los algoritmos ha permitido identificar qué aspectos resultan más interesantes para trabajarlos desde un punto de vista docente. Esta parte es importante, porque uno de los objetivos del proyecto no era solo obtener resultados numéricos, sino crear una herramienta que ayudara al alumnado a entender cómo funcionan los algoritmos bioinspirados.

Por este motivo, a partir del análisis realizado se han propuesto una serie de pruebas o actividades recomendadas para cada modelo. Estas actividades están pensadas para que el usuario no se limite a pulsar un botón y ver una animación, sino que pueda modificar parámetros, comparar comportamientos y sacar conclusiones sobre exploración, explotación, convergencia, factibilidad o sensibilidad ante restricciones.

Las actividades concretas se han incluido en el Capítulo 5, dentro de la sección dedicada a la propuesta de actividades docentes y pruebas guiadas. En ellas se plantean ejercicios para los modelos básicos y específicos, incluyendo comparaciones sobre la función esfera, pruebas con paisajes aleatorios, análisis de restricciones estructurales, escenarios de escasez en el canal de riego y comparación de modos de evaluación en el domo de 120 barras.

De esta forma, el estudio teórico de los algoritmos se ha transformado también en propuestas prácticas para el alumnado. Esto refuerza el carácter docente del trabajo, ya que los modelos desarrollados no solo muestran resultados finales, sino que invitan a probar, comparar y razonar sobre el comportamiento de cada algoritmo.

### **7.5. Limitaciones del proyecto**

Aunque el proyecto ha cumplido sus objetivos principales, también presenta algunas limitaciones que conviene tener en cuenta.

La primera limitación está relacionada con el propio entorno de desarrollo. NetLogo 3D es muy útil para crear modelos visuales e interactivos, pero no está pensado principalmente para realizar cálculo numérico intensivo. Esto se nota especialmente en los modelos específicos, donde cada evaluación puede requerir resolver sistemas estructurales, calcular tensiones, simular repartos o estimar frecuencias naturales. En estos casos, herramientas como MATLAB o Python podrían ofrecer un rendimiento computacional mayor.

Otra limitación importante es el número de ejecuciones realizadas. En metaheurísticas, los resultados pueden depender bastante de la población inicial, de la semilla aleatoria y de los parámetros elegidos. Aunque se han realizado varias pruebas, un estudio estadístico más profundo requeriría muchas más ejecuciones independientes, más semillas y una comparación más extensa de parámetros.

En el caso del modelo del canal de riego, la principal limitación es que se trata de una simulación simplificada. El artículo de referencia utiliza un simulador hidráulico específico, mientras que en este trabajo se ha implementado una aproximación conceptual en NetLogo. Por tanto, los resultados del modelo no deben interpretarse como una validación hidráulica real, sino como una herramienta académica para estudiar el comportamiento del algoritmo GWO en un problema de reparto de recursos.

El coste computacional del modelo CSA fue una de las principales limitaciones del proyecto, ya que la ejecución precisa con 20000 rotaciones, población 20 y 500 iteraciones requirió muchas horas de cálculo. Para poder trabajar de forma interactiva se han utilizado modos de evaluación aproximados, lo que implica que los resultados rápidos o mixtos deben validarse posteriormente mediante una



evaluación más precisa.

Por último, otra limitación general del proyecto es que los modelos están pensados principalmente con una finalidad docente y experimental. Es decir, no pretenden sustituir implementaciones profesionales optimizadas ni competir directamente con algoritmos desarrollados en entornos de alto rendimiento.

## **7.6. Líneas de trabajo futuro**

A partir del trabajo realizado, se pueden plantear varias líneas de mejora y ampliación.

Una primera línea de trabajo futuro sería realizar una comparación experimental más amplia entre los algoritmos. En este proyecto se han probado los modelos básicos sobre la función esfera y algunos paisajes aleatorios, pero sería interesante incluir más funciones benchmark, como Rastrigin, Rosenbrock, Ackley o Griewank. Estas funciones permitirían analizar mejor la capacidad de exploración de los algoritmos en espacios con múltiples óptimos locales.

También sería recomendable aumentar el número de semillas y ejecuciones independientes. Esto permitiría obtener estadísticas más robustas, como medias, desviaciones típicas, mejores y peores resultados, y no depender tanto de ejecuciones concretas.

Otra mejora posible sería incorporar un sistema más completo de exportación automática de resultados. Aunque algunos modelos ya permiten guardar información en archivos CSV, se podría unificar este sistema para todos los modelos y generar tablas de resultados de forma más cómoda.

En los modelos estructurales, una línea interesante sería optimizar el cálculo numérico interno. Por ejemplo, podrían estudiarse formas más eficientes de ensamblar matrices, resolver sistemas o calcular frecuencias naturales. También sería posible trasladar parte del cálculo a Python o MATLAB y mantener NetLogo como entorno visual.

En el modelo de la celosía de 72 barras, se podría mejorar el tratamiento de restricciones. La función de penalización utilizada funciona, pero podrían probarse otros métodos, como penalizaciones adaptativas, reparación de soluciones o almacenamiento separado del mejor diseño factible. Esto último sería especialmente útil, ya que a veces el algoritmo encuentra diseños de bajo peso con pequeñas violaciones de restricciones.

En el modelo del canal de riego, una mejora clara sería acercar la simulación hidráulica a un modelo más realista. Se podrían incorporar retardos de transporte, ecuaciones hidráulicas más detalladas o incluso conectar el modelo con un simulador externo. Esto permitiría comparar los resultados de forma más directa con el artículo de referencia.

En el modelo del domo de 120 barras, sería interesante mejorar la eficiencia del modo preciso y estudiar con más detalle la relación entre los modos rápido, mixto y preciso. También podrían probarse otras configuraciones del CSA o incluso otros algoritmos para comparar su rendimiento en problemas con restricciones dinámicas.

Por último, desde el punto de vista docente, se podrían ampliar las actividades propuestas en este trabajo, convirtiéndolas en guías prácticas más completas. Por ejemplo, cada actividad podría incluir objetivos concretos, configuración inicial, preguntas de reflexión, tablas para anotar resultados y una pequeña rúbrica de evaluación. De esta forma, los modelos podrían utilizarse directamente como material de prácticas en asignaturas relacionadas con optimización, simulación o inteligencia

## **7.7. Conclusión final**

Como conclusión final, este Trabajo Fin de Grado ha permitido desarrollar un conjunto de modelos interactivos en NetLogo 3D para estudiar algoritmos bioinspirados aplicados a problemas de optimización. El proyecto ha combinado teoría, programación, simulación, visualización y experimentación, lo que ha supuesto un reto mayor que simplemente implementar un algoritmo aislado.

Los modelos básicos han servido para observar el comportamiento general de WOA, GWO y CSA en espacios de búsqueda sencillos. Los modelos específicos han permitido aplicar esos mismos algoritmos a problemas más cercanos a la ingeniería, mostrando las dificultades que aparecen cuando se introducen restricciones, variables discretas, indicadores de rendimiento o cálculos dinámicos.

Aunque los resultados no siempre igualan los valores publicados en la literatura, los modelos desarrollados cumplen su objetivo principal: facilitar la comprensión

y experimentación con algoritmos bioinspirados mediante simulaciones visuales e interactivas. Además, el análisis realizado durante el proyecto ha permitido transformar el estudio teórico de los algoritmos en propuestas prácticas para el alumnado, recogidas en la documentación de uso. Esto refuerza el carácter docente del trabajo, ya que los modelos no solo muestran resultados finales, sino que invitan a probar, comparar y razonar sobre el comportamiento de cada algoritmo.

En definitiva, el proyecto demuestra que NetLogo puede ser una herramienta útil para acercar la optimización bioinspirada a un contexto docente, permitiendo que el usuario no solo vea el resultado final, sino también el proceso de búsqueda que siguen los algoritmos para llegar hasta él.

## **CAPITULO 8 BIBLIOGRAFÍA**

- [1] Seyedali Mirjalili, Andrew Lewis (2016) The Whale Optimization Algorithm (WOA) Advances in Engineering Software, vol 95 pages 51–67. Brisbane, Australia. <https://doi.org/10.1016/j.advengsoft.2016.01.008>
- [2] Rezaei, H., Bozorg-Haddad, O., Chu, X. (2018). Grey Wolf Optimization (GWO) Algorithm. In: Bozorg-Haddad, O. (eds) Advanced Optimization by Nature-Inspired Algorithms. Studies in Computational Intelligence, vol 720. Springer, Singapore. [https://doi.org/10.1007/978-981-10-5221-7\\_9](https://doi.org/10.1007/978-981-10-5221-7_9)
- [3] Braik MS (2021). “Chameleon Swarm Algorithm: A bio-inspired optimizer for solving engineering design problems.” Expert Systems with Applications, 174, 114685. doi: 10.1016/j.eswa.2021.114685
- [4] A. Kaveh, Applications of Metaheuristic Optimization Algorithms in Civil Engineering (2017) Capitulo 4.4.1. A 120-Bar Dome Truss, pag 309. <https://doi.org/10.1007/978-3-319-48012-1>
- [5] Kazem Shahverdi, J. M. Maestre (2022) Gray Wolf Optimization for Scheduling Irrigation Water. [https://doi.org/10.1061/\(ASCE\)IR.1943-4774.0001688](https://doi.org/10.1061/(ASCE)IR.1943-4774.0001688)
- [6] A. Kaveh, Applications of Metaheuristic Optimization Algorithms in Civil Engineering (2017) Capitulo 16.4.4. Spatial 72-Bar Truss Problem, pag 51. <https://doi.org/10.1007/978-3-319-48012-1>
- [7] Ken Schwaber & Jeff Sutherland, La Guía Scrum (2020) Disponible en: <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-European.pdf>

[8] Wilensky, U. (1999). NetLogo. Center for Connected Learning and Computer-Based Modeling, Northwestern University. Evanston, IL. Disponible en: <https://www.netlogo.org/>

[9] J.A. Martín Melguizo. Algoritmos bioinspirados para NetLogo (2021)

## **ANEXO A. Manual de usuario de los modelos desarrollados**

### **A.1. Introducción**

Este anexo recoge un manual básico de uso de los modelos desarrollados en NetLogo 3D durante el trabajo fin de grado. La finalidad del manual es facilitar que un usuario, especialmente alumnado que se acerque por primera vez a estos algoritmos, pueda ejecutar los modelos, modificar sus parámetros principales e interpretar los resultados mostrados en la interfaz.

Los modelos se dividen en dos grupos:

- Modelos básicos, orientados a visualizar el funcionamiento general de los algoritmos.
- Modelos específicos, donde cada algoritmo se aplica a un problema de optimización de ingeniería.

Los modelos básicos permiten entender conceptos como *población*, *fitness*, *exploración*, *explotación*, *convergencia* y *óptimos locales*. Los modelos específicos muestran cómo esos mismos algoritmos pueden adaptarse a problemas más complejos, con variables, restricciones y funciones objetivo propias.

### **A.2. Recomendaciones generales de uso**

Antes de utilizar cualquiera de los modelos, se recomienda seguir estos pasos generales:

- 1- Abrir el archivo correspondiente en NetLogo 3D.
- 2- Revisar los parámetros disponibles en la interfaz.
- 3- Pulsar el botón de inicialización del modelo, normalmente llamado SETUP, SETUP GWO, SETUP CSA o similar.

- 4- Ejecutar el modelo:
  - a. con el botón GO o EJECUTAR, para ejecución continua
  - b. con el botón STEP o STEP BY STEP, para avanzar una iteración cada vez.
- 5- Observar la evolución en:
  - a. la vista 3D
  - b. los monitores
  - c. las gráficas de convergencia
  - d. los valores de fitness o mejor solución.

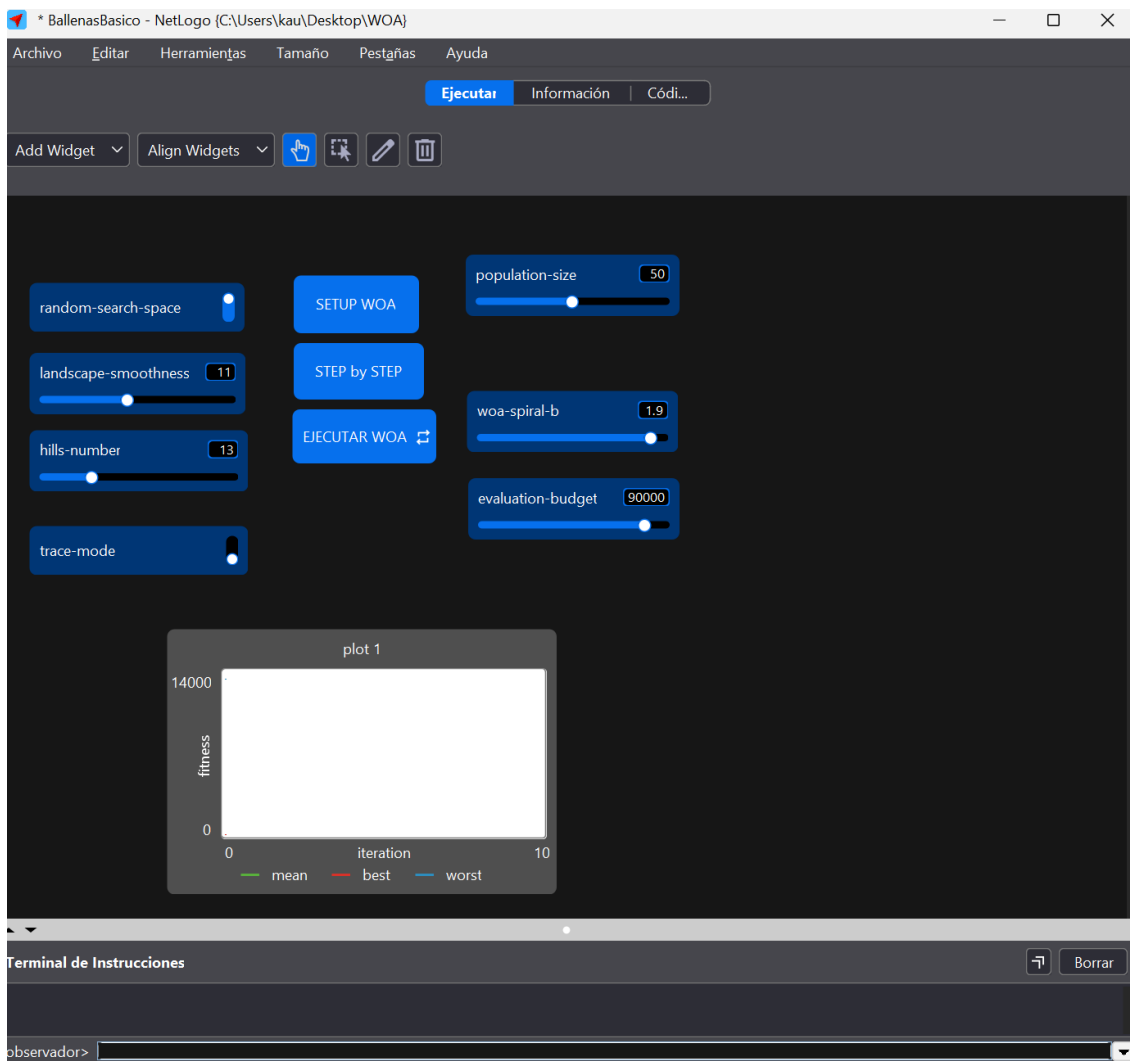
En general, si se modifica cualquier parámetro importante del modelo, es recomendable volver a pulsar SETUP antes de ejecutar de nuevo. Esto asegura que la población, el espacio de búsqueda o el problema específico se inicialicen correctamente con la nueva configuración.

### **A.3. Modelo básico WOA**

#### **A.3.1. Descripción del modelo**

El modelo básico WOA implementa el Whale Optimization Algorithm, inspirado en el comportamiento de caza de las ballenas jorobadas. Cada agente representa una ballena, es decir, una solución candidata dentro del espacio de búsqueda.

El modelo permite visualizar cómo las ballenas se desplazan intentando encontrar la mejor solución. En la función esfera, el objetivo es acercarse al mínimo global situado en el origen. En el modo de paisaje aleatorio, el comportamiento puede ser más irregular, ya que aparecen varias zonas atractivas en el espacio de búsqueda



### A.3.2. Parámetros principales

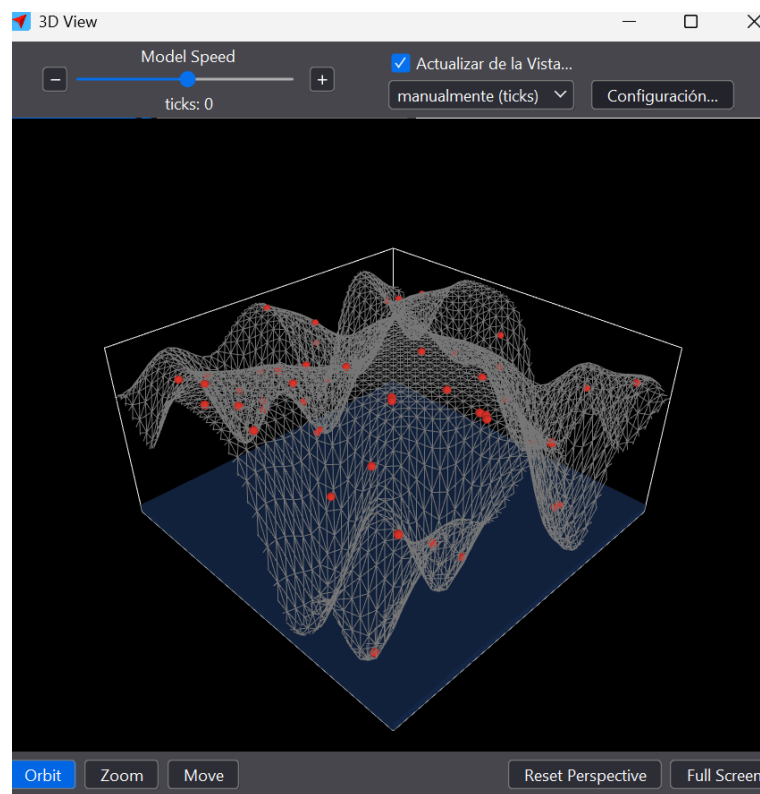
Los parámetros más importantes del modelo son:

- population-size: número de ballenas utilizadas en la simulación.
- random-search-space: permite activar un paisaje aleatorio en lugar de la función esfera.
- woa-spiral-b, si aparece en la interfaz: controla la forma de la espiral utilizada por el algoritmo.
- evaluation-budget: determina cuántas iteraciones puede ejecutar el modelo.



### A.3.3. Cómo ejecutar el modelo

- 1- Ajustar el tamaño de población.
- 2- Activar o desactivar random-search-space, según se quiera usar función esfera o paisaje aleatorio.
- 3- Pulsar SETUP.
- 4- Ejecutar:
  - a. STEP BY STEP, para avanzar una iteración;
  - b. GO, para ejecutar de forma continua.
- 5- Observar la vista 3D y la evolución del fitness.



### A.3.4. Interpretación de resultados

En la función esfera, un buen resultado se produce cuando las ballenas se acercan al centro de la superficie, donde el valor de la función es mínimo.

En paisajes aleatorios, el comportamiento puede variar más. Algunas ballenas pueden concentrarse en regiones locales que parecen buenas, aunque no sean necesariamente la mejor zona global.

Se recomienda observar:

- la posición final de las ballenas
- la evolución del mejor fitness
- si la población converge demasiado pronto
- cómo afecta el tamaño de población al resultado.

#### **A.3.5. Pruebas recomendadas**

Algunas pruebas sencillas son:

- Ejecutar el modelo con población pequeña y población grande.
- Comparar el comportamiento con random-search-space activado y desactivado.
- Ejecutar paso a paso para observar cómo se actualizan las posiciones.
- Repetir varias veces la simulación para ver cómo cambia el resultado debido a la aleatoriedad.

### **A.4. Modelo básico GWO**

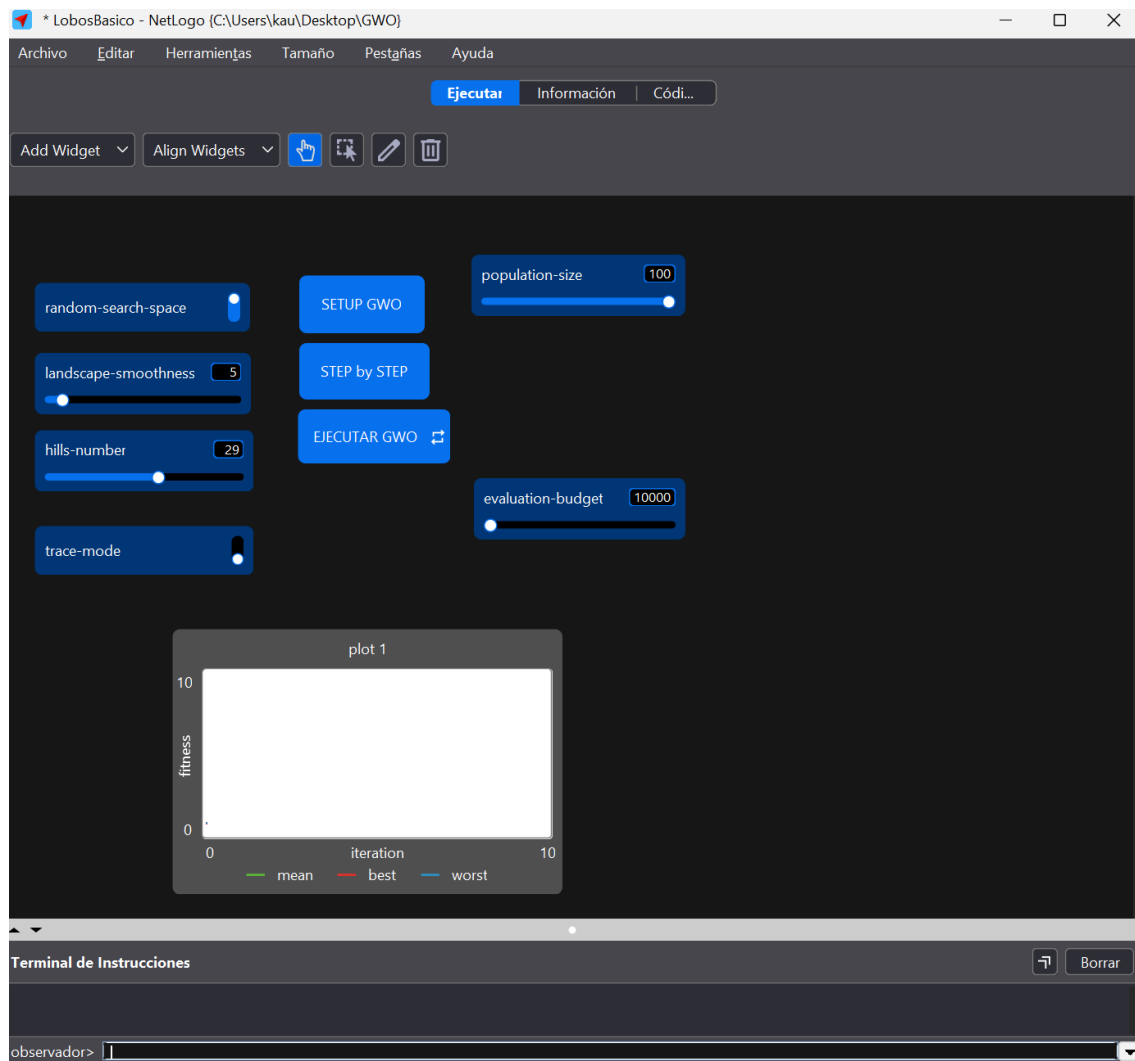
#### **A.4.1. Descripción del modelo**

El modelo básico GWO implementa el Grey Wolf Optimizer, inspirado en la jerarquía social y el comportamiento de caza de los lobos grises. Cada agente representa un lobo, es decir, una solución candidata.

La población se organiza mediante tres líderes principales:

- Alfa: mejor solución encontrada.
- Beta: segunda mejor solución.
- Delta: tercera mejor solución.
- Omega: resto de lobos.

El resto de la población actualiza sus posiciones tomando como referencia a estos tres líderes.



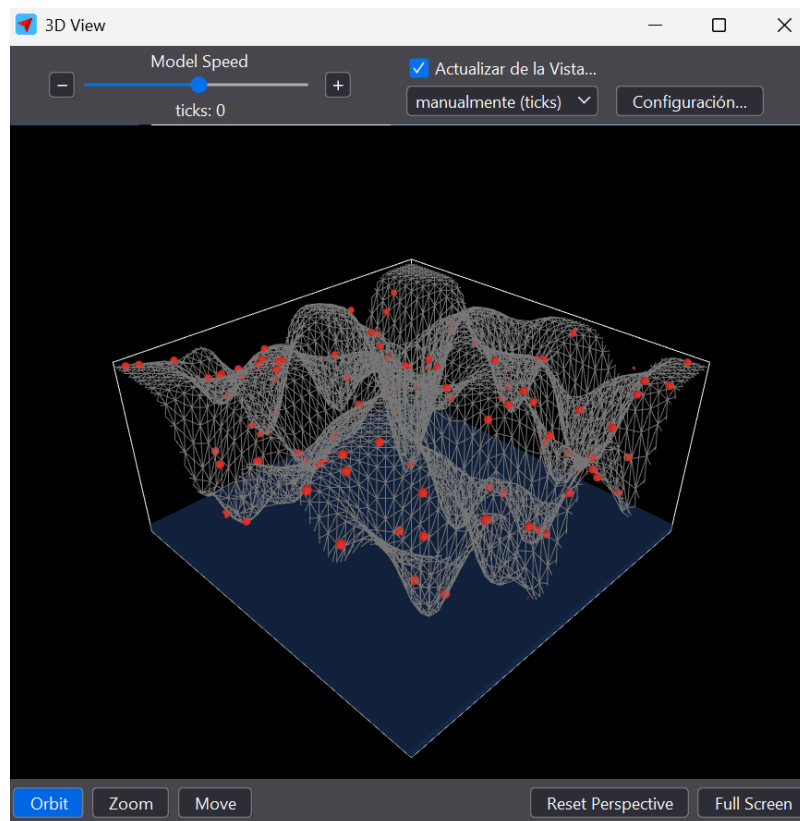
#### A.4.2. Parámetros principales

Los parámetros más importantes son:

- population-size: número de lobos.
- evaluation-budget: presupuesto máximo de evaluaciones.
- random-search-space: activa un paisaje aleatorio.
- landscape-smoothness: controla el suavizado del paisaje aleatorio.
- hills-number: número de elevaciones iniciales del paisaje.
- trace-mode: activa o desactiva el rastro de movimiento de los lobos.

#### A.4.3. Cómo ejecutar el modelo

- 1- Ajustar el tamaño de población.
- 2- Definir el presupuesto de evaluaciones.
- 3- Elegir si se quiere función esfera o paisaje aleatorio.
- 4- Pulsar CONFIGURAR o SETUP.
- 5- Ejecutar con PASO, STEP BY STEP, IR o GO, según la interfaz.
- 6- Observar la posición de los lobos y la evolución del fitness.



#### A.4.4. Interpretación de resultados

Al inicio de la simulación, los lobos suelen explorar regiones amplias del espacio de búsqueda. Conforme pasan las iteraciones, el parámetro de control disminuye y los lobos tienden a concentrarse alrededor de las mejores soluciones encontradas.

En la función esfera, lo esperable es que la población converja hacia el origen. En paisajes aleatorios, pueden aparecer estancamientos en óptimos locales.

Conviene observar:

- cómo cambian alfa, beta y delta
- si los lobos se agrupan demasiado pronto
- si el fitness baja de forma estable
- cómo afecta el presupuesto de evaluaciones.

#### **A.4.5. Pruebas recomendadas**

Se recomienda probar:

- poblaciones pequeñas frente a poblaciones grandes
- presupuestos de evaluación bajos y altos
- paisajes aleatorios con distinto número de colinas
- activación y desactivación del modo de rastro.

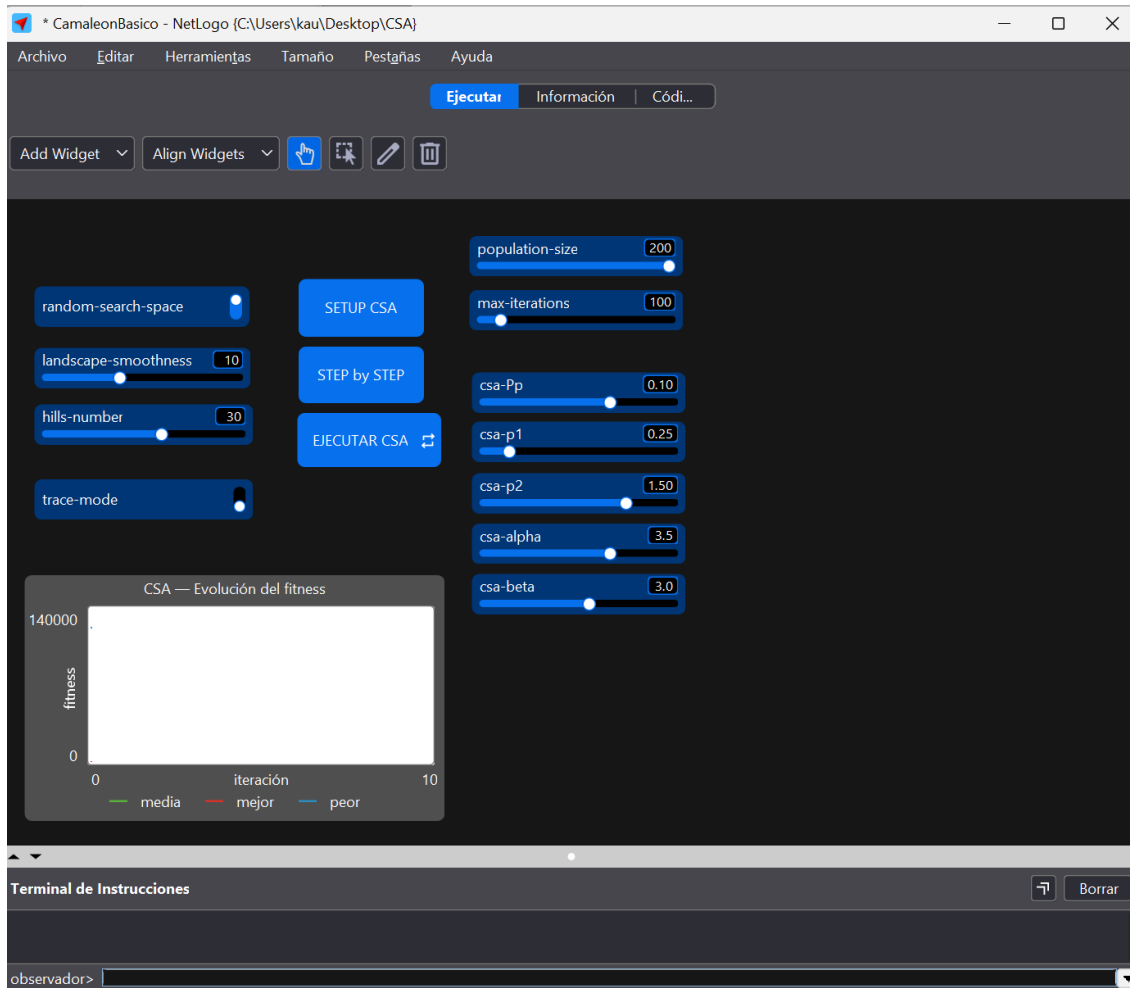
#### **A.5. Modelo básico CSA**

##### **A.5.1. Descripción del modelo**

El modelo básico CSA implementa el Chameleon Swarm Algorithm, inspirado en el comportamiento de los camaleones durante la búsqueda y captura de presas. Cada agente representa un camaleón. El algoritmo combina tres fases principales:

- 1- Búsqueda o percepción.
- 2- Rotación ocular 360 grados.
- 3- Lanzamiento de lengua o ataque.

El modelo permite observar cómo la población explora el espacio de búsqueda y cómo los camaleones se desplazan hacia regiones prometedoras.



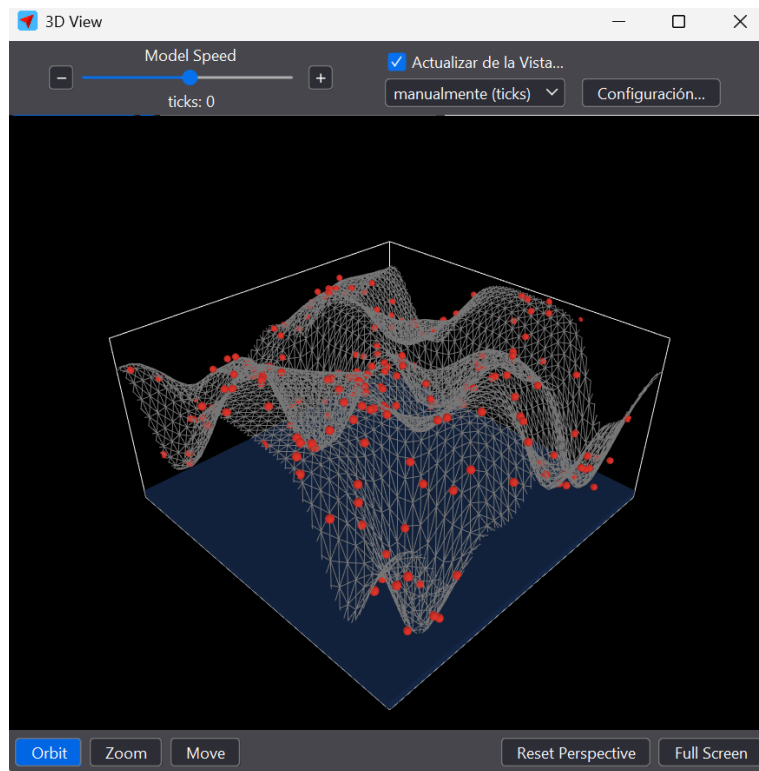
### A.5.2. Parámetros principales

Los parámetros más importantes son:

- population-size: número de camaleones.
- max-iterations: número máximo de iteraciones.
- csa-Pp: probabilidad asociada a la rama exploratoria.
- csa-p1: influencia del mejor personal.
- csa-p2: influencia del mejor global.
- csa-alpha: parámetro del decaimiento adaptativo.
- csa-beta: parámetro del decaimiento adaptativo.
- random-search-space: activa el paisaje aleatorio.
- trace-mode: muestra las trayectorias de los camaleones.

### A.5.3. Cómo ejecutar el modelo

- 1- Ajustar los parámetros del CSA.
- 2- Elegir función esfera o paisaje aleatorio.
- 3- Pulsar SETUP CSA.
- 4- Ejecutar con:
  - a. STEP BY STEP, para una iteración
  - b. EJECUTAR CSA, para ejecución continua.
- 5- Observar la evolución del enjambre en la vista 3D.



### A.5.4. Interpretación de resultados

En la función esfera, un buen resultado debe tener:

$$g\text{-best-vars} \approx [0 \ 0]$$

$$g\text{-best-val} \approx 0$$

En paisajes aleatorios, el objetivo es localizar la zona más favorable del paisaje.

En este caso, puede ocurrir que el enjambre se concentre en una región localmente buena pero no necesariamente global.

El gráfico de evolución del fitness ayuda a interpretar si el algoritmo está mejorando, convergiendo o estancándose.

#### **A.5.5. Pruebas recomendadas**

Se recomienda probar:

- aumentar y reducir population-size
- cambiar csa-Pp
- modificar csa-p1 y csa-p2
- probar distintos valores de csa-alpha y csa-beta
- activar trace-mode
- comparar función esfera y paisaje aleatorio.

#### **A.6. Modelo específico WOA - Celosía espacial de 72 barras**

##### **A.6.1. Descripción del modelo**

Este modelo aplica el Whale Optimization Algorithm a la optimización estructural de una celosía espacial de 72 barras.

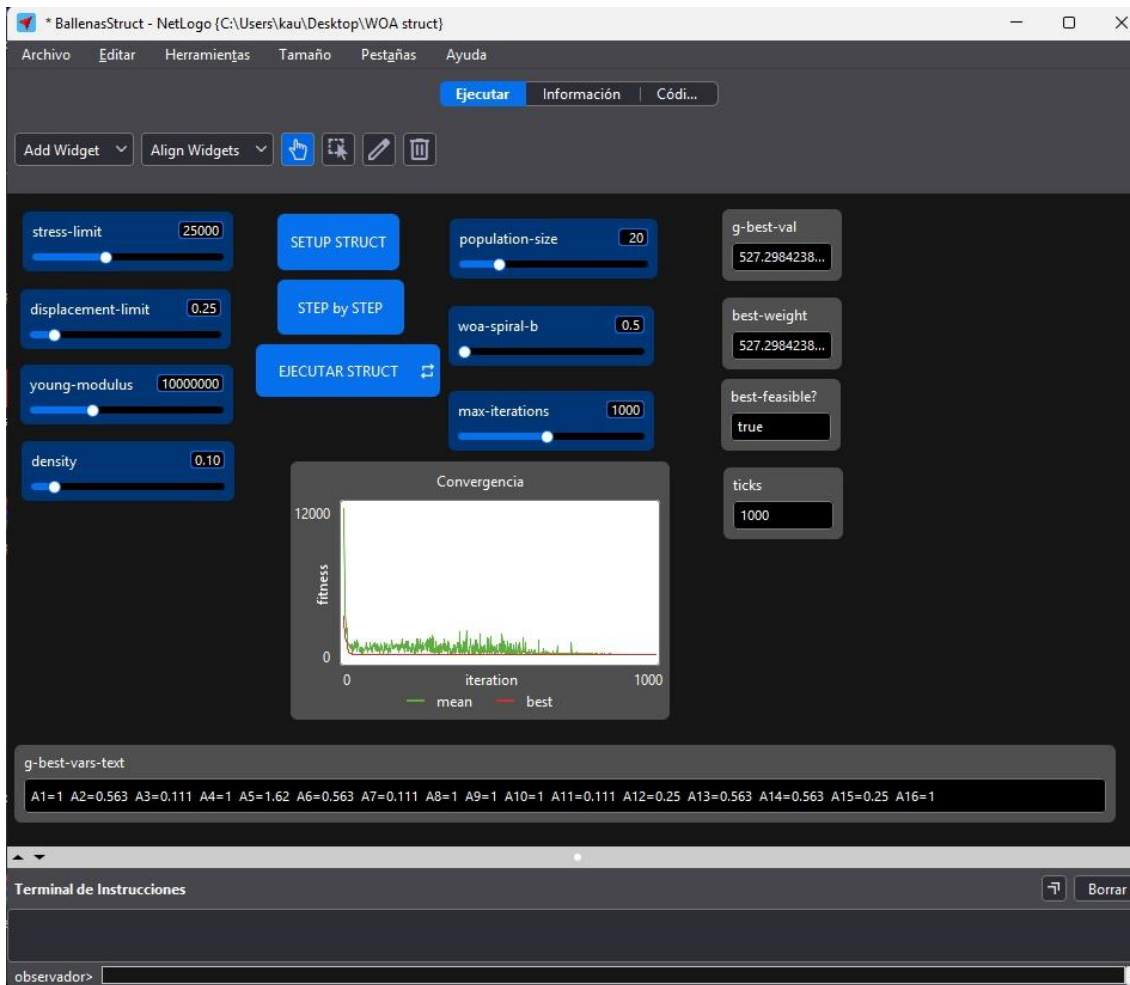
El objetivo es minimizar el peso de la estructura seleccionando áreas transversales para 16 grupos de barras. El diseño debe cumplir restricciones de tensión y desplazamiento.

Cada solución candidata representa un vector:

$$[A1 \ A2 \ A3 \ \dots \ A16]$$

Cada variable corresponde a un grupo de barras.





## A.6.2. Parámetros principales

### Parámetros estructurales

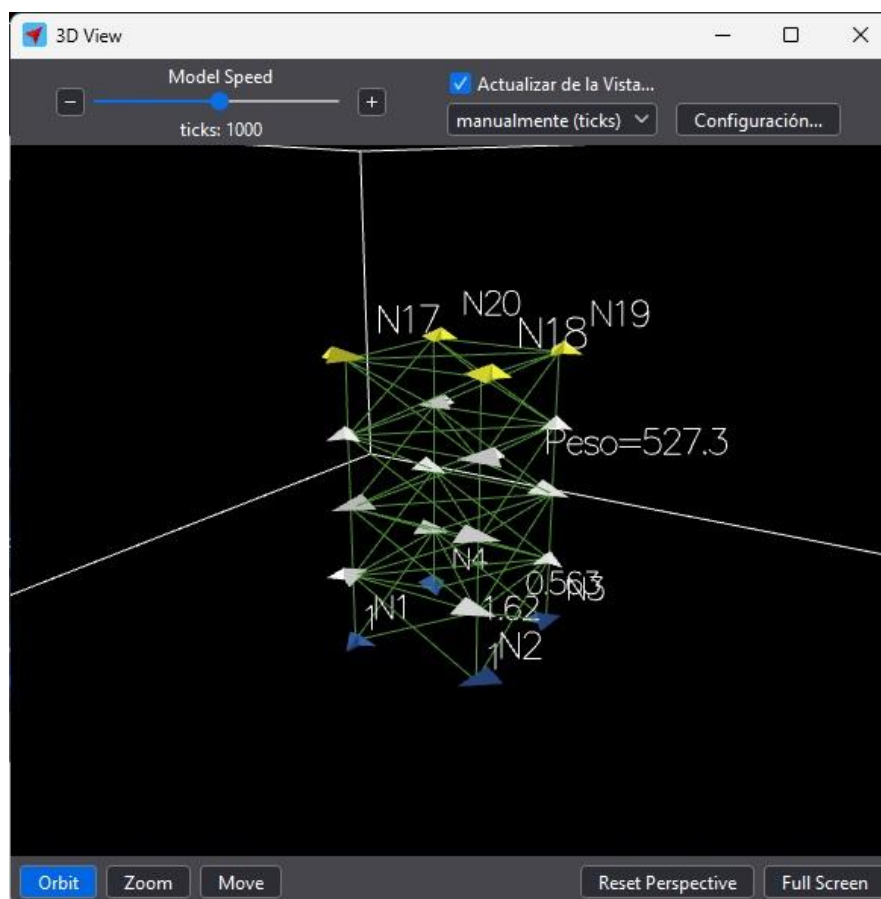
- stress-limit: límite de tensión admisible.
- displacement-limit: límite de desplazamiento admisible.
- young-modulus: módulo de elasticidad del material.
- density: densidad del material.

### Parámetros del algoritmo

- population-size: número de ballenas.
- max-iterations: número máximo de iteraciones.
- woa-spiral-b: parámetro de la espiral del WOA.

### A.6.3. Cómo ejecutar el modelo

- 1- Introducir los parámetros estructurales.
- 2- Definir tamaño de población e iteraciones.
- 3- Pulsar SETUP STRUCT.
- 4- Ejecutar:
  - a. STEP by STEP, para avanzar iteración a iteración
  - b. EJECUTAR STRUCT, para ejecutar de forma continua.
- 5- Observar el peso, la factibilidad y la estructura 3D.



#### **A.6.4. Qué muestran los resultados**

Los monitores principales son:

- g-best-val: mejor fitness encontrado.
- best-weight: peso real del mejor diseño.
- best-feasible?: indica si el diseño cumple las restricciones.
- ticks: iteración actual.
- g-best-vars-text: áreas seleccionadas para los 16 grupos.

La vista 3D muestra la estructura. Las barras pueden cambiar de color en función del nivel de tensión:

- verde: baja sollicitación
- amarillo/naranja: sollicitación intermedia o cercana al límite
- rojo: posible superación del límite.

#### **A.6.5. Interpretación de resultados**

Un buen resultado debe cumplir:

*best-feasible? = true*

*best-total-violation = 0*

Además, el peso debe ser lo más bajo posible.

Si un diseño tiene poco peso pero *best-feasible? = false*, no puede considerarse una solución válida, aunque pueda ser útil para analizar hacia dónde tiende el algoritmo.

#### **A.6.6. Pruebas recomendadas**

Se recomienda probar:

- población 20 y 1000 iteraciones
- población 30 y 2000 iteraciones
- población mayor, por ejemplo 100, si se desea explorar más
- distintos valores de woa-spiral-b
- comparar los resultados con el peso benchmark de referencia.

## A.7. Modelo específico GWO - Canal de riego

### A.7.1. Descripción del modelo

Este modelo aplica Grey Wolf Optimizer a un problema simplificado de reparto de agua en un canal de riego.

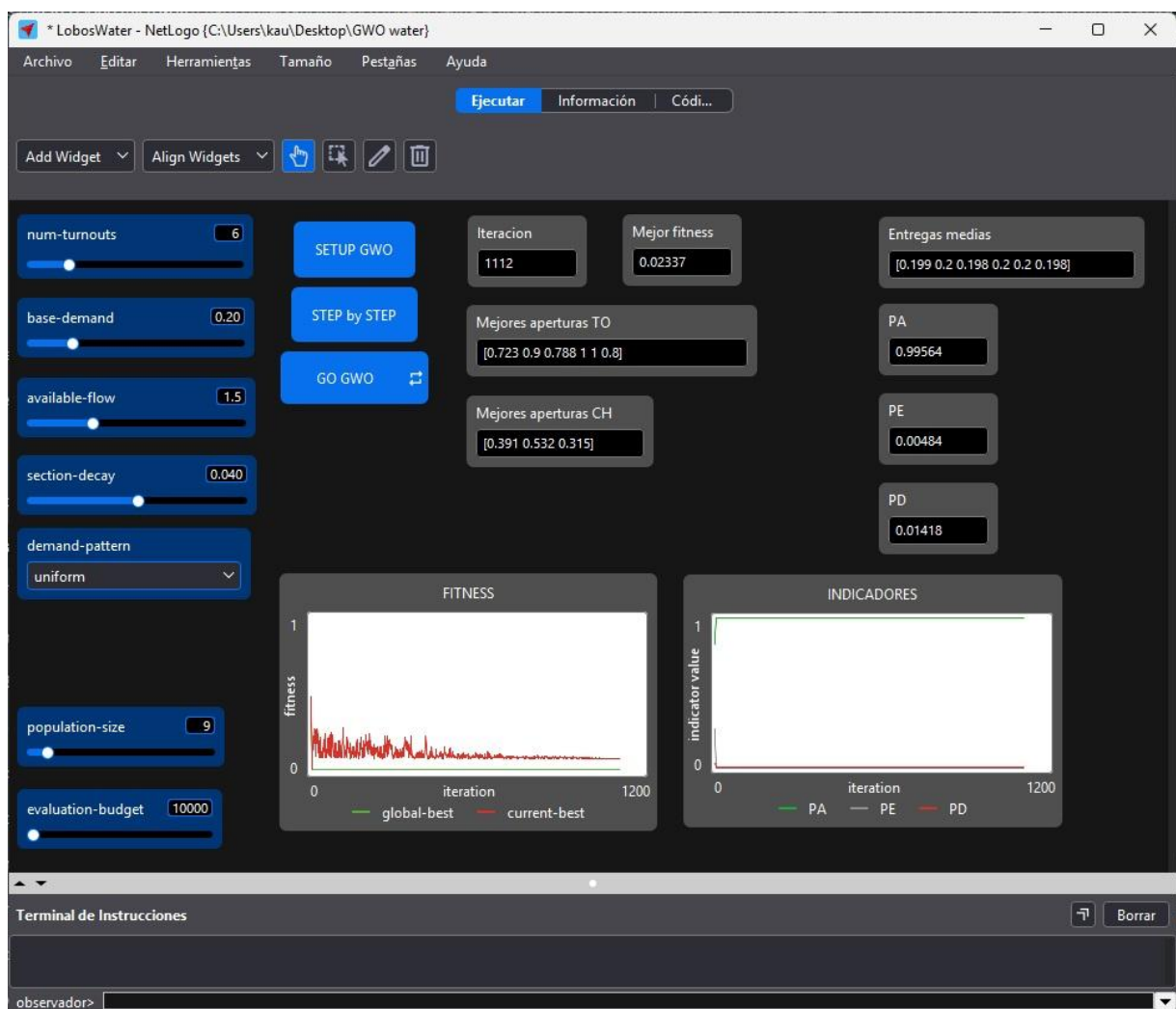
Cada solución candidata representa una configuración de aperturas:

*[TO1 TO2 TO3 TO4 TO5 TO6 CH1 CH2 CH3]*

Donde:

- TO representa puntos de entrega de agua.
- CH representa compuertas de control.

El objetivo es mejorar el reparto de agua entre los distintos puntos de entrega.



### **A.7.2. Parámetros principales**

#### Parámetros del problema

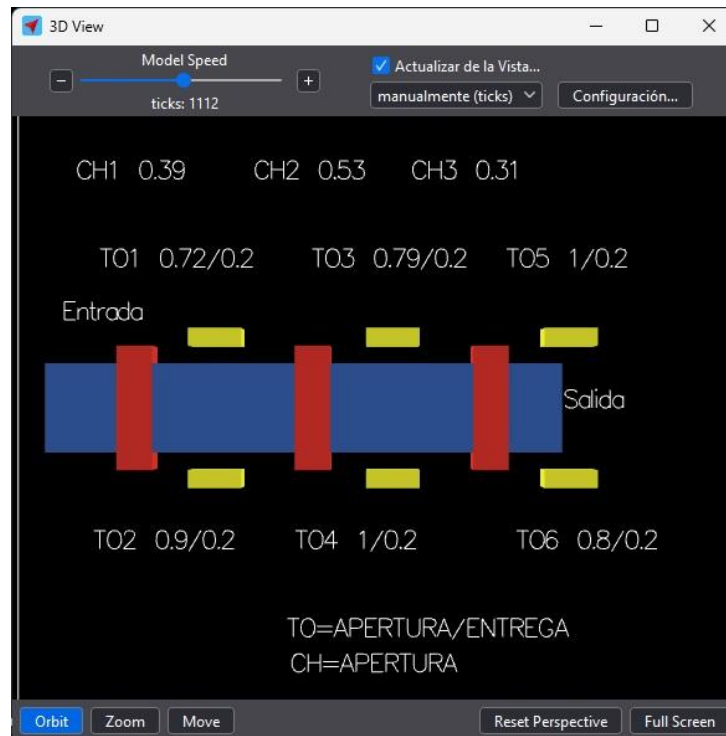
- num-turnouts: número de puntos de entrega.
- base-demand: demanda base por punto.
- available-flow: caudal disponible.
- demand-pattern: patrón de demanda.
- section-decay: heterogeneidad del canal.

#### Parámetros del algoritmo

- population-size: número de lobos.
- evaluation-budget: presupuesto de evaluaciones.

### **A.7.3. Cómo ejecutar el modelo**

- 1- Definir el número de turnouts.
- 2- Ajustar demanda base y caudal disponible.
- 3- Elegir patrón de demanda.
- 4- Definir población y presupuesto de evaluación.
- 5- Pulsar SETUP GWO.
- 6- Ejecutar con:
  - a. STEP by STEP
  - b. GO GWO.
- 7- Observar aperturas, entregas medias e indicadores.



#### A.7.4. Qué muestran los resultados

Los monitores principales son:

- Mejor fitness: valor de la función objetivo.
- Mejores aperturas TO: aperturas óptimas de los puntos de entrega.
- Mejores aperturas CH: aperturas óptimas de las compuertas de control.
- Entregas medias: caudal medio entregado en cada turnout.
- PA: adecuación.
- PE: equidad.
- PD: fiabilidad.
- (PF que seria eficiencia, no se muestra porque por la forma en que se desarrolla siempre será 1 o demasiado próximo a uno como para que merezca la pena ver la evolución del mismo)

### **A.7.5. Interpretación de resultados**

Un buen resultado debe tener:

*fitness bajo*  
*PA cercano a 1*  
*PF cercano a 1*  
*PE bajo*  
*PD bajo*

Si hay caudal suficiente, el modelo debería ser capaz de entregar valores cercanos a la demanda. Si hay escasez, es normal que el fitness aumente y que PA disminuya.

### **A.7.6. Pruebas recomendadas**

Se recomienda probar:

- demanda uniforme con caudal suficiente
- demanda baja y caudal suficiente
- demanda alta y caudal ajustado
- situación de escasez
- distintos valores de section-decay.

Ejemplos:

base-demand = 0.10

available-flow = 1.0

o bien:

base-demand = 0.12

available-flow = 0.7

## A.8. Modelo específico CSA - Domo espacial de 120 barras

### A.8.1. Descripción del modelo

Este modelo aplica el Chameleon Swarm Algorithm al problema del domo espacial de 120 barras.

El objetivo es minimizar el peso de la estructura cumpliendo restricciones dinámicas sobre las dos primeras frecuencias naturales:

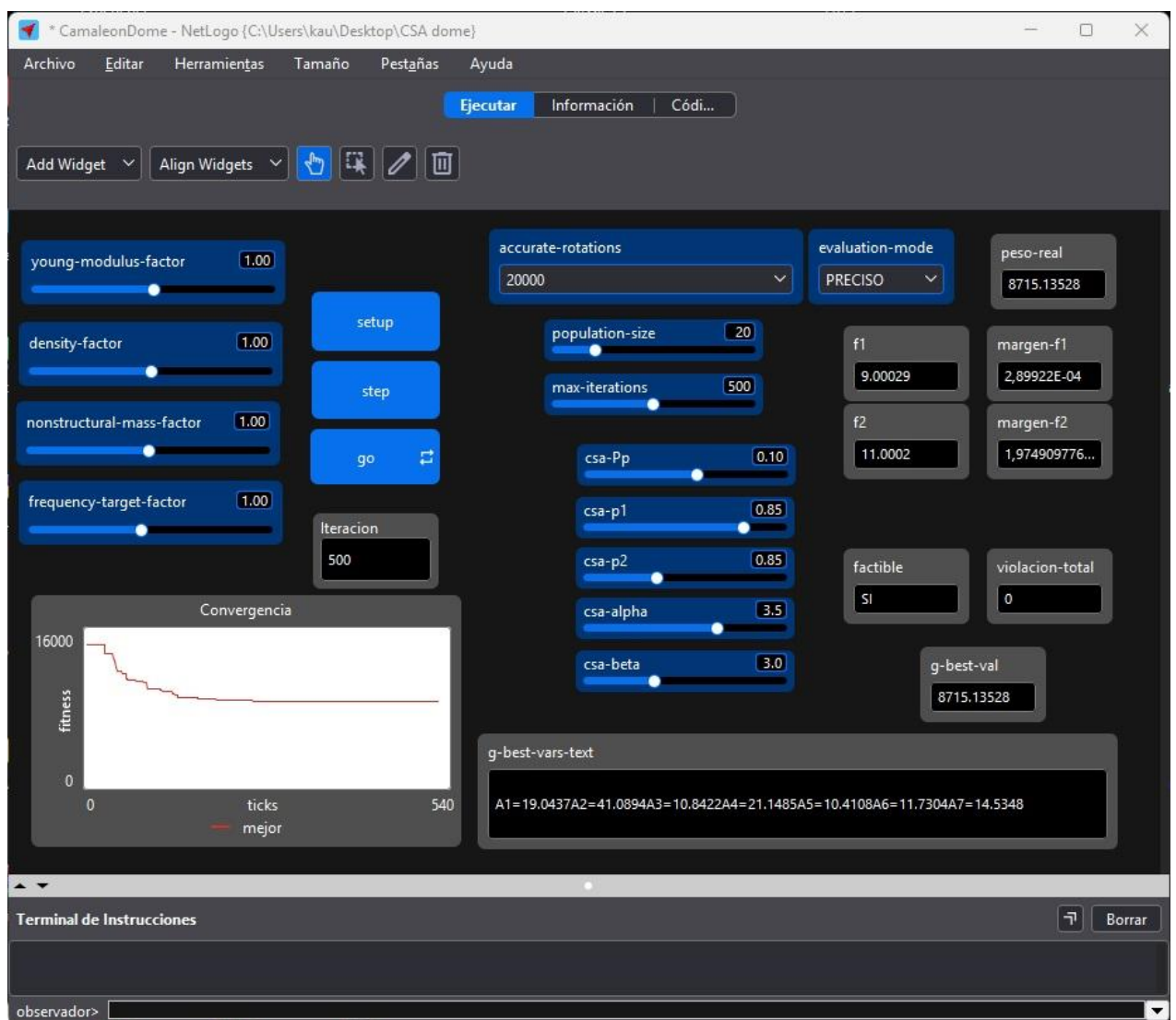
$$f_1 \geq 9 \text{ Hz}$$

$$f_2 \geq 11 \text{ Hz}$$

Cada solución candidata representa un vector de 7 áreas:

$$[A1 \ A2 \ A3 \ A4 \ A5 \ A6 \ A7]$$

Cada área corresponde a un grupo de barras del domo.





### A.8.2. Parámetros principales

#### Parámetros físicos

- young-modulus-factor: factor aplicado al módulo de Young.
- density-factor: factor aplicado a la densidad.
- nonstructural-mass-factor: factor aplicado a las masas no estructurales.
- frequency-target-factor: factor aplicado a las frecuencias mínimas.

#### Parámetros del algoritmo

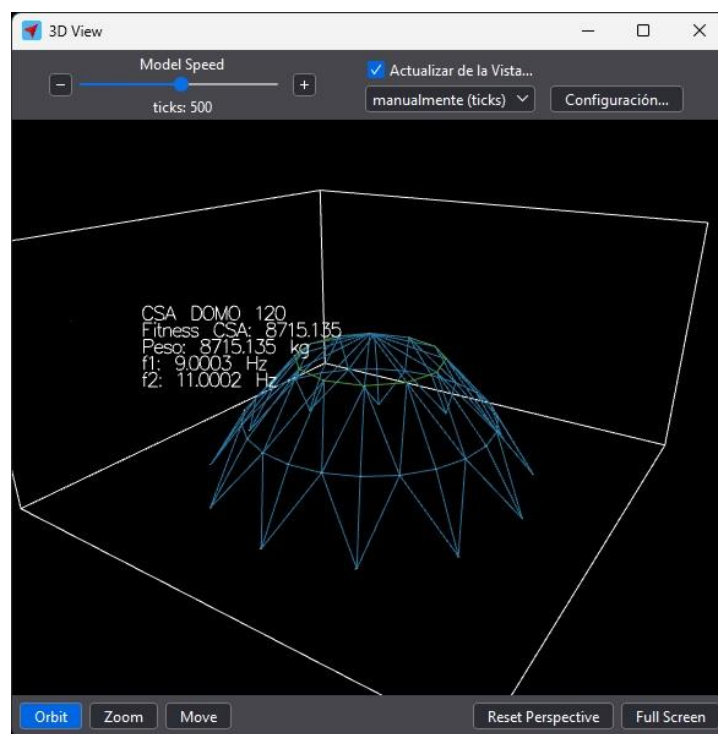
- population-size: número de camaleones.
- max-iterations: número máximo de iteraciones.
- csa-Pp: probabilidad de exploración.
- csa-p1: influencia del mejor personal.
- csa-p2: influencia del mejor global.
- csa-alpha: parámetro de decaimiento.
- csa-beta: parámetro de decaimiento.

#### Parámetros de evaluación

- evaluation-mode
  - RAPIDO
  - MIXTO
  - PRECISO
- accurate-rotations:
  - controla el número de rotaciones de Jacobi usadas en evaluación precisa.

### A.8.3. Cómo ejecutar el modelo

- 1- Ajustar factores físicos a 1.0 para el caso base.
- 2- Seleccionar modo de evaluación.
- 3- Definir población e iteraciones.
- 4- Ajustar parámetros CSA.
- 5- Pulsar setup.
- 6- Ejecutar con:
  - a. step;
  - b. go.
- 7- Observar peso, frecuencias, factibilidad y vector de áreas.



### A.8.4. Modos de evaluación

#### Modo RAPIDO

Sirve para observar el comportamiento del algoritmo de forma ágil. No debe utilizarse para validar resultados finales.

### Modo MIXTO

Usa una aproximación intermedia. Es útil para pruebas rápidas y para ver tendencias, pero puede ser demasiado optimista.

### Modo PRECISO

Realiza una evaluación más fiable. Es el modo recomendado para validar resultados finales, aunque puede requerir mucho tiempo de ejecución.

#### **A.8.5. Qué muestran los resultados**

Los monitores principales son:

- peso-real: peso físico del diseño.
- g-best-val: fitness del mejor diseño.
- f1: primera frecuencia natural.
- f2: segunda frecuencia natural.
- margen-f1: diferencia entre  $f1_{f1}$  y el límite.
- margen-f2: diferencia entre  $f2_{f2}$  y el límite.
- violacion-total: incumplimiento total de restricciones.
- factible: indica si el diseño cumple las restricciones.
- g-best-vars-text: áreas del mejor diseño.

#### **A.8.6. Interpretación de resultados**

Un diseño válido debe cumplir:

$$\begin{aligned} \text{factible} &= SI \\ \text{violacion-total} &= 0 \\ f1 &\geq 9 \\ f2 &\geq 11 \end{aligned}$$

Un buen resultado tendrá además un peso lo más bajo posible.

Si el modo MIXTO muestra pesos muy bajos y frecuencias muy altas, el resultado debe interpretarse con cuidado. Para obtener conclusiones fiables, es necesario validar en modo PRECISO.

### **A.8.7. Pruebas recomendadas**

Para pruebas rápidas:

- evaluation-mode = MIXTO
- population-size = 10
- max-iterations = 300
- accurate-rotations = 12000

Para validación final:

- evaluation-mode = PRECISO
- population-size = 20
- max-iterations = 500
- accurate-rotations = 20000

Parámetros recomendados del CSA:

- csa-Pp = 0.10
- csa-p1 = 0.85
- csa-p2 = 0.85
- csa-alpha = 3.5
- csa-beta = 3.0

### **A.9. Problemas frecuentes y recomendaciones**

#### El modelo no cambia al modificar parámetros

Si se modifica un parámetro importante, hay que pulsar de nuevo SETUP. Si no se reinicia el modelo, la simulación puede seguir usando la configuración anterior.

#### El algoritmo no mejora

Puede deberse a:

- pocas iteraciones;
- población demasiado pequeña;
- paisaje de búsqueda complicado;
- convergencia prematura;
- parámetros demasiado exploratorios o demasiado conservadores.

#### El resultado tiene buen peso pero no es factible

Esto ocurre especialmente en los modelos estructurales. Un diseño puede ser ligero, pero incumplir tensión, desplazamiento o frecuencia. En ese caso no debe considerarse una solución válida.

#### El modelo CSA tarda mucho

Es normal en modo PRECISO. El cálculo modal del domo es costoso. Para pruebas iniciales conviene usar modo MIXTO y reservar el modo PRECISO para validación final.

#### Los resultados no son exactamente iguales al paper

Esto también es normal. Los modelos de este trabajo están implementados en NetLogo y tienen una finalidad principalmente docente y visual. Además, algunos modelos, como el canal de riego, son simplificaciones del problema original.

### **A.10. Cierre del manual**

Este manual pretende facilitar el uso de los seis modelos desarrollados. Los modelos pueden emplearse para realizar pruebas sencillas, observar la evolución de los algoritmos y comprender mejor cómo funcionan las metaheurísticas bioinspiradas.

La recomendación general es comenzar siempre por los modelos básicos y, una vez entendido el comportamiento de cada algoritmo, pasar a los modelos específicos. De esta forma, resulta más fácil interpretar los resultados cuando aparecen restricciones, variables discretas, indicadores hidráulicos o cálculos dinámicos.